

1 ЛЕКЦИЯ

**1. Что такое "вычислительная платформа" с точки зрения пользователя?
Каков её состав?**

Вычислительная платформа - такой набор инструментов, которые позволяют организовать какие-то вычисления, вычислительный процесс.

Классические примеры - C, Python и так далее. Еще более традиционный инструментарий - система команд процессора.

На каждой вычислительной платформе предоставляем пользователю инструментарий, чтобы он мог описать вычислительную модель и потом ее запустить.

4 компонента вычислительной платформы:

- Язык. В рамках которого мы формулируем вычислительный процесс(синтаксис Си)
- Наборы типов. Данные, которыми мы оперируем в рамках языка (машинные слова, int, char, pointer)
- Инстансы. Набор примитивных кусочков из которых составляется вычислительный процесс (операция "+", вызов функции и так далее)
- Паттерны. Элемент, объединяющий предыдущие три. Элемент, выводящий инструмент с уровня технических ресурсов на уровень человеческой деятельности. Те принципы и идеи, как реализовывать или описывать те или иные алгоритмы

2. Какие уровни абстракций относятся к дисциплине "Архитектура компьютера"? Почему границы определены именно так?

Application Software	программы
Операционные системы	Драйверы устройств
Архитектура	Регистры инструкций
Микро - архитектура	контроллеры каналы передачи данных
Логика	Сумматоры память
Цифровые схемы	AND вентили NOT вентили
Аналоговые схемы	усилители фильтры
Устройства	транзисторы диоды
Физика	Электроны

К нашей дисциплине относятся уровни примерно с середины операционных систем до цифровых схем. Все что ниже нас не очень интересует, потому что мы не собираемся тут проектировать аналоговые схемы и не очень углубляемся в физику. А операционные системы целиком нам не очень интересны, потому что это уже достаточно высокий уровень и изучать его надо отдельно.

3. В чём заключается отличие Platform-Based Design от проектирования для конкретной платформы?

Традиционно как мы пишем софт? Приходим, берем ЯП, берем либы и фреймворк, переиспользуем этот код и реализовываем свой софт. Но при разработке низкоуровневых систем, с высокими требованиями, ограничением по батарее, сложными условиями эксплуатации мы так сделать не можем - потому что каждый раз, когда мы фиксируем универсальную "вычислительную платформу" у этой универсальности есть большая цена - потому что появляется куча уровней абстракции, внутренних процессов, которые мы не можем контролировать.

Platform-Based Design - реализует другой подход, когда разработка системы идет двунаправленная: берем какую-то низкоуровневую вычислительную платформу, на базе нее строим тот самый вычислитель, ту самую платформу, которая будет закрывать наше пространство задач, что называется системной платформой (по сути также вычислительная платформа, выращенная с какой-то базы), это одна сторона. С другой стороны используем эту платформу и ее механизмы под наши задачи.

Если мы говорим о разработке встроенных систем, то такая концепция получается by design (с одной стороны разработать железку, с другой софт который будет этим рулить)

2 ЛЕКЦИЯ

4. Почему большинство современных компьютерных систем считаются системами с преобладающей программной составляющей? Приведите примеры.

Если есть компьютерная система и в процессе ее разработки огромное влияние оказывает разработка программной составляющей, то компьютерная система - программная (с преобладающей программной составляющей). Значительная часть затрат на систему приходится на разработку программного обеспечения.

Причина в том, что тотально снижаются затраты на "железную" составляющую. За счет ее переиспользования, очень редко она разрабатывается под систему.

За редчайшими исключениями любая компьютерная система на самом деле программная система.

Пример: даже когда мы говорим про производство и покупку процессоров, там от 50 до 75% это уже затраты на ПО. Или ноутбуки Apple, железная составляющая занимает сильно меньшую составляющую, чем операционная система, которая стоит над этим железом и позволяет так классно им пользоваться.

Программная система — это система, состоящая из программного обеспечения, аппаратных компонентов и данных, основная ценность которой обеспечивается за счёт выполнения программных функций.

5. Что такое информационная и управляющая система? Каковы их отличия (функции, ПО, аппаратура)?

Информационная система - те системы, задача которых: получать данные, преобразовывать/накапливать и выдавать в измененном/обработанном виде (бд в простейшем случае, google, zoom, telegram).

Такие системы не взаимодействуют напрямую с физическим миром и ориентированы на работу в цифровой информационной среде.

Характерные особенности

1. Производительность --- приоритет на скорость обработки данных.
2. Спекулятивные вычисления --- предсказание ветвлений и выполнение наперёд.
3. Параллелизм --- одновременное выполнение множества операций.

Важный акцент на скорости - в общем случае чем быстрее – тем лучше.

Управляющая система - те системы, которые направлены и сконцентрированы на взаимодействие с реальным физическим миром с целью контроля или управления

объектами или процессами посредством получения данных, преобразования/накопления, и выдачи в измененном/обработанном виде (например, автопилот, встроенные системы в бытовой технике).

Такие системы всегда работают в условиях реального времени и должны учитывать физические параметры объекта управления.

Характерные особенности

1. Встроенное исполнение

- УС интегрируются в объект управления.
- Минимизация задержек и устойчивость к помехам.

2. Ограниченность ресурсов

- Главный ограничитель --- энергия и тепло, а не производительность.
- УС проектируются под конкретные условия эксплуатации.

4. Работа в реальном времени

- УС должны выполнять действия в соответствии с физическим временем, а не только с логикой программы.
- Жёсткие временные ограничения.

6. Что такое "требование реального времени"? Примеры систем.

Если система работает в режиме реального времени, это не значит, что она работает быстро, не значит, что работает абсолютно точно, не значит, что работает без отказов. Это значит, что система работает предсказуемо и в срок

Пример: система управления ГЭС, а именно системы водосброса. Есть системы, которые должны обеспечить открытие водосброса в случае, если внутренние бассейны критически переполняются. Если датчик регистрирует превышение допустимого уровня воды, то в течение 7 минут мы следим за значением, и если оно стабильно высокое и стабильно превышает, только после этого мы считаем, что вода действительно переполняет и в течение 7 минут мы поэтапно открываем водосброс.

7. Каковы этапы эволюции информационных систем: от пакетной обработки до облачных систем?

Эволюция архитектур

- Централизованные ЭВМ --- один мощный компьютер обслуживает пользователей. Как пакетная обработка: задачи формируются одна за одной, передаются в систему и обрабатываются также одна за одной
- Распределённые системы --- несколько машин обрабатывают задачи параллельно. Мы уходим от единого сервера в сторону нескольких

небольших узлов, которые могут обрабатывать задачи и быть использованы независимо

- ПК и смартфоны --- перенос вычислений к пользователю. ПК — это в целом то, на чем мы есть сейчас. Появляется классическая "рабочая станция"
- Клиент-серверная модель --- взаимодействие локальных клиентов с удалёнными сервисами.
- Облачная инфраструктура - вычислительные мощности более не ценность сама по себе, теперь это сервис, услуга

8. Каковы этапы эволюции управляющих систем: от ИУС до КФС?

Этапы эволюции:

ИУС - информационно управляющие системы.

Объект управления и управляющая система непосредственно разделены и могут быть представлены как два прямо отдельных объекта

ВсС - встроенные системы

Когда компьютеры стали достаточно маленькие их стало возможно конструктивно объединить с объектом управления. В целом все осталось также, просто физически одно располагается внутри другого.

РВсС - распределенные встроенные системы

Пример: наушники с подавлением шума. Мы в принципе не можем себе позволить разделить объект управления с управляющей системой (будут адские задержки) и нам придется поставить контроллер в наушник, который будет заниматься шумоподавлением, максимально близко к управляемому объекту. Теперь мы разрабатываем систему как распределенную систему:

- Управляющие блоки встроены в оборудование --- микроконтроллеры, процессоры.

- Система разделилась на компоненты, размещённые ближе к объекту управления.

- Это позволило снизить помехи, повысить точность и надёжность.

ВК КФС - киберфизические системы. Объект и система управления неразделимы (единое целое). Пример: гироскутер, вращающиеся двери

Во всех прошлых у нас есть объект управления и мы им как-то управляем, а здесь в целом сама система не может существовать без системы управления.

Аппаратная составляющая в таких системах стала сложнее и важнее.

9. Каковы задачи и предмет дисциплины "Системная инженерия"? Какова роль системных инженеров?

Системная инженерия - пытается ответить на вопрос а как делать успешные системы?

Системная инженерия занимается разработкой больших систем.

Большая система — это когда мы без гениального инженера не можем собрать систему у кого-то в голове целиком.

Предмет системной инженерии - Audi, Boeing 787, нефтедобывающая платформа. Словом - предмет системной инженерии — это большие системы

Роль системного инженера – человек, который согласует составные части больших систем (но не как менеджер), отвечает за целостность системы, согласовывает разработчиков разного профиля, согласовывает процессы, команды разработки, ведет документацию.

10. Что такое успешная система? Какие точки зрения (заинтересованные стороны) необходимы для построения успешной системы?

Успешная система - эта та, при реализации которой удовлетворены все значимые заинтересованные стороны (стейкхолдеры).

Под сторонами понимаются все люди, которые имеют власть над нашим проектом, в широком смысле от разработчиков до государства и сертифицирующих органов. Те, кто может повлиять на наш проект.

Как пример - активисты, которые просто против всего, тоже будут являться заинтересованной стороной и иногда бывает полезно учитывать в некотором объеме

Точки зрения на понятие системы:

- система как совокупность частей, которые при этом (!) дают какое-то новое качество, новые свойства, которые позволяют получить что-то новое, чего раньше не было

- система как функциональное место - где мы можем приткнуть эту систему и получить какую-то пользу

- система как жизненный цикл - откуда система появляется и что нам делать потом

Разработка успешной системы требует:

1. Рассмотрения её структуры
2. Рассмотрения её функционального места
3. Рассмотрения её операционного окружения
4. Рассмотрения её жизненного цикла
5. Рассмотрения её обеспечивающих систем

11. Как можно рассматривать систему с точки зрения структуры? Каковы причины множественности структуры?

Система — это комбинация взаимодействующих элементов, организованных для достижения одной или нескольких поставленных целей.

Система как совокупность частей (с точки зрения структуры) - мы всегда можем построить "диаграмму деления" - декомпозировать систему на кусочки, на этажи, выстраивая иерархию и дерево этих подсистем и из составных частей (верхний уровень --- сама система, средний --- её подсистемы, а нижний --- составные части. Это отражает ключевой принцип системной инженерии --- разбиение сложной системы на управляемые компоненты для анализа, проектирования и управления)

В реальной жизни систему можно рассматривать как несколько подсистем, которые с разных точек зрения принимают различные структуры и взаимодействия.

И для каждого заинтересованного лица система может принимать свою структуру, это и есть причина ее множественности. Пример: дом как система может быть рассмотрен с точки зрения электрики, с точки зрения вентиляции, с точки зрения расстановки мебели и техники. Это все одна и та же система, но на каждом слое ее составные части взаимодействуют по-разному и для электрика не подойдет структура, рассказанная для строителя. Можно сделать "гипер структуру", которая будет содержать все, о чем было упомянуто и сразу, но это не будет иметь смысла ввиду сложности такой структуры.

12. Как можно рассматривать систему с точки зрения функционального места? Что такое операционное окружение?

Система определяется как то, что она делает, а не только как то, из чего она состоит.

Идентификация системы и определение системы идет не от внутренней конструкции, а от того функционального места, для которого она сделана, куда она будет размещена/поставлена. Т.е. рассматриваем, как мы ее применяем, и не важно, как она устроена внутри.

Пример с насосной станцией: у нас есть станция, у нее два насоса. Один из них ломается. Мы говорим "поменяй насос 2", а не "поменять составляющую системы с серийным номером 123". Хотя именно серийный номер идентифицирует сломанную часть системы. Более того, мы можем поменять насос на немного другой, оставив внешний интерфейс таким же. Или вообще поменять тип насоса. Тогда, рассматривая систему с точки зрения структуры это новая система, но с точки зрения функционального места это та же самая система.

Функциональное место определяется:

- стейкхолдерами (те самые заинтересованные стороны)
- точками зрения

Существует операционное окружение, в котором система будет развернута. Это окружение определяет интерфейсы и взаимодействия, которые система должна

реализовать, чтобы быть полезной и эффективной, выполняя свою функциональность. При этом системы, определённые в этом окружении, также являются частями своей технологической экосистемы. Для процессора это система питания и вся периферия, с которой он вынужден работать.

13. Кто такие заинтересованные стороны (Stakeholders) системы и какова их роль в разработке?

Стейкхолдеры (stakeholders) — это заинтересованные стороны: физические лица, группы или организации, которые влияют на деятельность компании/проекта или испытывают влияние от ее результатов

Стейкхолдеры — это не только заказчики, но и все, кто взаимодействует с продуктом: от разработчиков до конечных пользователей и даже контролирующих органов.

Роль стейкхолдеров заключается в активном участии в жизненном цикле продукта для обеспечения его успешности. Их основные функции включают: определение целей, бюджета и необходимости создания системы, формирование требований, оценивают промежуточные результаты, проверяют систему на соответствие требованиям.

14. Как можно рассматривать систему с точки зрения жизненного цикла? Что такое обеспечивающая система?

Системная организация и её операционная среда определяют действующую систему. Но этого недостаточно для успешной системы. Целостное представление должно включать жизненный цикл системы и её обеспечивающие системы

Жизненный цикл системы — это эволюция интересующей системы со временем: от концепции до вывода из эксплуатации.

Этапы жизненного цикла:

1. Концептуальный этап. Для компьютерных систем на этом этапе проводится анализ требований и архитектурное проектирование.
2. Этап разработки включает создание проектной документации. Для инженерных систем это могут быть схемы и чертежи, а для программного обеспечения --- исходный код, архитектура
3. Этап производства связан с изготовлением или сборкой системы.
4. Этап утилизации охватывает эксплуатацию системы в условиях реального использования, вплоть до исчерпания её ресурсов. В этот период система должна сохранять работоспособность, эффективность и устойчивость в заданной среде.
5. Этап поддержки направлен на обеспечение стабильной работы системы после запуска. Он включает техническое обслуживание, обновление компонентов, устранение ошибок и адаптацию к новым условиям.

6. Этап вывода из эксплуатации --- финальный этап, включающий отключение системы, демонтаж оборудования, завершение сопутствующих процессов.

Для каждого этапа притягивается обеспечивающая система - система, которая дополняет данную на этапе жизненного цикла, но может не вносить непосредственный вклад в ее функционирование. Причем у каждой из обеспечивающих систем есть свой жизненный цикл.

Примеры обеспечивающих систем: команды разработчиков, промышленные производства и т.д.

15. Каковы цели архитектурного проектирования компьютерных систем?

Главная цель архитектурного проектирования системы в первую очередь снижение рисков, что включает в себя обнаружение ошибок на ранних этапах, обеспечение целостности системы, структуризация и поддержка процессов системы на всех ее жизненных циклах.

Также архитектурный подход обеспечивает предсказуемость сроков и бюджета, и по сути все что мы получаем на практике – повышение вероятности того, что эти предсказания будут верны.

В цели не входит "сделать дешевле", "сделать быстрее"

16. Что такое архитектура по Гради Бучу? Что представляют собой логическая и физическая структура?

Архитектура - логическая и физическая структура компонентов системы и их взаимосвязи, сформированные всеми стратегическими (это важно в долгосрок) и тактическими (сейчас так удобно) проектными решениями, применяемыми во время разработки.

Логическая структура - устанавливает существование и роль ключевых абстракций и механизмов, которые будут определять архитектуру и общий дизайн системы

Физическая структура - описывает конкретный программный и аппаратный состав реализации системы. Зависит от конкретной технологии

Это хорошее определение, но немного устаревшее для современных реконфигурируемых и облачных систем, где аппаратное обеспечение может динамически изменяться в зависимости от текущих требований. Архитектура, согласно этому подходу, не статична, так как она эволюционирует по мере появления новой информации.

Таким образом, логический аспект отвечает за структурное и абстрактное понимание устройства системы, а физический -- за её реализацию в конкретной технической среде.

17. Что такое архитектура согласно ISO 42010? Что такое архитектурное описание?

Архитектура --- фундаментальные концепции или свойства системы в ее среде, воплощенные в ее элементах, взаимосвязях и принципах ее проектирования и развития.

Описание архитектуры --- рабочий продукт, используемый для воплощения архитектуры.

Это определение рекомендуется для использования в разработке программного обеспечения и систем.

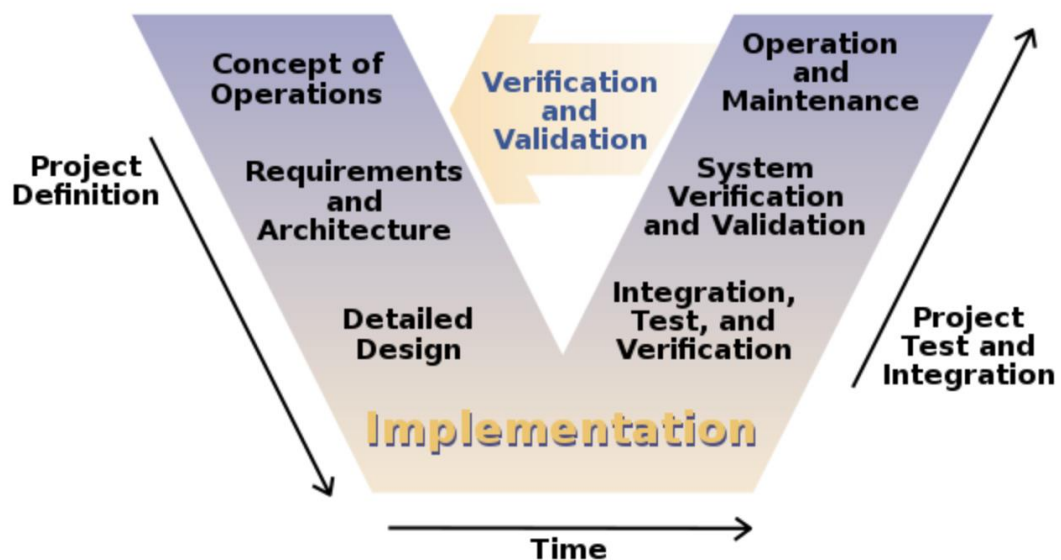
Главная его сложность --- высокая абстрактность формулировки, поэтому важно помнить, что оно не даёт точного ответа, что считать архитектурой, но указывает, где её искать и на что обращать внимание. Это приводит к следующим важным выводам:

1. Аппаратное и программное обеспечение являются компонентами компьютерной системы. Это означает, что при анализе компьютерной системы вы не можете абстрагироваться ни от одного из них.
2. Понятие "архитектура" включает в себя технические вопросы и вопросы, связанные с проектом. Невозможно спроектировать архитектуру вычислительных систем только с технической точки зрения.
3. Система имеет архитектуру вне зависимости от ее описания.

18. Как архитектурные решения влияют на проектные метрики? Что такое V-диаграмма и какую особенность разработки она демонстрирует?

В целом архитектура — это совокупность проектных решений, которые определяют каркас системы и её поведение. Именно на этом уровне закладываются фундаментальные концепции, взаимосвязи компонентов и принципы функционирования системы в окружении.

Ошибки на архитектурном уровне особенно критичны, так как они могут привести к тому, что система окажется нефункциональной или вовсе непригодной для эксплуатации. Именно поэтому архитектура --- зона наивысших проектных рисков.



Левая часть «V» описывает этапы нисходящего проектирования: от формулировки требований --- к архитектуре, детальному проектированию и реализации. Правая часть отражает соответствующие этапы валидации: модульное тестирование, интеграционные испытания, верификация подсистем и, наконец, проверка всей системы на соответствие исходным требованиям.

Ключевая идея здесь в соответствии уровней проектирования и проверки, то есть чем выше по «V» находится ошибка, тем дороже её исправление, например проблема, заложенная на уровне архитектуры, обнаружится только при сборке всей системы, когда затраты на переделку могут быть критичными. А локальные ошибки на нижнем уровне --- например, в реализации отдельных функций --- могут быть устранены с минимальными издержками.

Таким образом, качественная архитектура позволяет минимизировать стоимость изменений, снижает технические риски и обеспечивает управляемость сложных систем в долгосрочной перспективе.

19. Почему плохой менеджмент может увеличить бюджет проекта быстрее, чем другие факторы? Проблема передачи информации.

Менеджмент - коммуникация между разными элементами систем проектов.

Ключевая проблема - проблема передачи информации.

Менеджер (нетехнический специалист) формулирует задачу разработчику, у них сильно отличается круг важных вопросов, часть задачи в целом не понимаема разработчиком.

Пример с металлическими уголками, где думали, что производство будет делать лазером, и сделали информационные отверстия в уголках, а оказалось, что на производстве делают это вручную, что оказалось гораздо сложнее из-за этих отверстий. Срок выполнения увеличился за просто так. Пример с контроллером освещения, где из-за разности расположения платы на столе разработчика и при

реальной эксплуатации расположение фаз получилось в обратном порядке. Придется потом переделывать.

Задача менеджера --- обеспечить передачу необходимой информации, организовать консультации и сделать так, чтобы каждый участник получил те сведения, которые ему действительно нужны. Это требует высокой квалификации, поскольку любые пробелы в понимании со стороны разработчика приводят к принятию неверных решений и, как следствие, к срыву задачи.

Если постановку задачи осуществляет нетехнический специалист, риски увеличиваются. Именно здесь особенно важны архитектурный подход, системное мышление и эффективное управление.

3 ЛЕКЦИЯ

20. Сравните подходы к реализации вычислений в арифмометре и на логарифмической линейке. Аналогии с современными компьютерами.

Вычисления на логарифмической линейке осуществляются путем заранее, по определенной схеме сделанных насечек, которые ввиду математических свойств их размещения позволяют получить значение какой-либо функции. В компьютерной технике подобный подход применяется в формате кеширования значения функции, получения ее значения по заранее вычисленным таблицам. Позволяет сильно упростить и зачастую ускорить алгоритм, зачастую в ущерб занимаемой памяти

В случае арифмометра мы не пытаемся представить всевозможные решения в виде данных. Мы анализируем вычислительный процесс, бьем его на некоторые повторяемые кусочки, воспроизводимые шаги, чтобы получить результат для входных данных из области определения.

А еще арифмометр - цифровое устройство, а линейка - аналоговое.

+ Есть номограммы (слева первый множитель, справа второй, по центру результат произведения). Идеи номограммы и линейки уже в виде табличек делают.

1. Рост области определения функций:

- для арифмометра: медленный рост сложности устройства, средний рост длительности расчёта (зависит от функции);
- для линейки/таблицы: быстрый рост размера линейки, минимальный рост длительности расчёта.

2. Рост области значений функции:

- для арифмометра: медленный рост сложности устройства, средний рост длительности расчёта (зависит от функции);
- для таблицы: средний рост размера таблицы, минимальный рост длительности расчёта.
- для линейки: быстрый рост размера линейки или погрешности, минимальный рост длительности расчёта.

21. Каков был подход к расчёту артиллерийских таблиц группой людей? Аналогии с современными компьютерами (параллелизм уровня инструкций, надёжность).

В начале XX века перед армиями стояла сложная вычислительная задача --- расчёт артиллерийских таблиц. Эти таблицы содержали параметры для стрельбы: количество пороха, угол наклона ствола и поправки на ветер, рельеф местности и другие факторы.

Расчёт требовал многократного решения сложных уравнений с разными входными данными. Однако компьютеров ещё не существовало, а ручные вычисления были ненадёжными. Риски:

- ошибки выбора операции --- могла происходить путаница между сложением и вычитанием.
- арифметические ошибки --- из-за усталости или невнимательности
- сокрытие ошибок --- если расчёт занимал много времени, исполнитель мог подкорректировать результат, чтобы не переделывать работу.

В основе архитектуры лежало представление расчетной формулы в виде графа, где вершины соответствовали элементарным математическим операциям, а ребра --- потокам данных между ними.

На каждую вершину графа назначался обученный грамоте солдат, способный безошибочно выполнять только одну определенную математическую операцию - будь то сложение, вычитание или поиск значений тригонометрических функций в таблицах.

Вычислительный процесс был организован как строго последовательный поток слева направо. Промежуточные результаты передавались между операторами --- фактически, "бумажка" с результатом просто переходила от одного вычислителя к следующему. Такой подход минимизировал возможность потери или искажения данных.

Для обеспечения максимальной надёжности результатов применялась система дублирования вычислений. Две идентичные роты независимо выполняли одни и те же расчеты. На выходе системы результаты сравнивались, и в случае расхождения требовался повторный расчет проблемного фрагмента. Вероятность одновременного совершения одинаковой ошибки в обеих группах была статистически меньше.

Преимущества:

1. Организационная эффективность: Чёткое разделение ролей между компонентами системы позволяет каждому сосредоточиться на своей задаче.
2. Рост производительности за счёт параллелизма: Параллельное исполнение независимых операций значительно ускоряет обработку данных.

3. Конвейеризация расчётов ещё больше повышает производительность. После подсчёта одного значения и его передачи он может сразу считать следующее входные данные. Таким образом, количество параллельно выполняемых расчётов соответствует числу стадий в конвейере: чем больше стадий, тем выше уровень параллелизма.

22. Каково устройство электрического реле? Какие виды реле существуют и каковы области их применения?

Электрическое реле - автоматизированная кнопка, может быть нормально разомкнутой, нормально замкнутой. Если подан ток - катушка создает магнитное поле и одни контакты замыкаются, другие размыкаются, обеспечивая передачу сигнала или энергии через цепь.

Физическая конструкция:

1. Электромагнитная катушка --- при подаче тока создаёт магнитное поле, которое воздействует на стержень и якорь.
2. Стержень --- усиливает магнитное поле катушки, повышая эффективность работы реле.
3. Якорь --- металлическая пластина, которая под действием магнитного поля притягивается к стержню, замыкая или размыкая контакты.
4. Неподвижные контакты: N.C. (Normally Closed) --- контакт, замкнутый в исходном состоянии (ток идёт, пока реле не активировано) и N.O. (Normally Open) --- контакт, разомкнутый в исходном состоянии (ток появляется только при срабатывании реле).
5. Основание --- корпус реле, обеспечивающий механическую фиксацию всех элементов.
6. Пружина --- обеспечивает возврат якоря в исходное положение при отключении питания.

Области применения электрического реле:

- когда-то для компьютеров
- там, где транзисторы не выдерживают, силовые шкафы управления, неблагоприятная среда
- в ПЛК

Виды:

- Механические (концевой выключатель в лифте)
- Тепловые (в чайнике)
- Оптические (реагируют на освещение)
- Реле времени (отключают по прошествии времени)

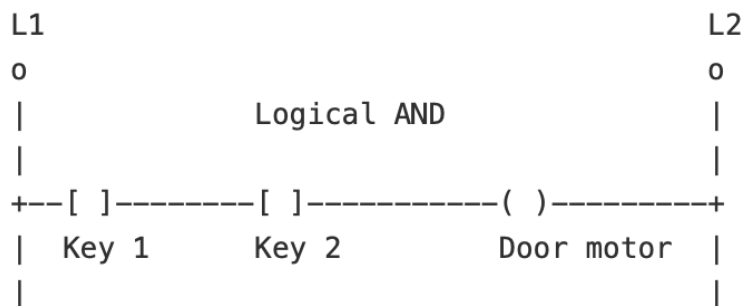
23. Как реализуется булев базис на электрических реле?

В зависимости от типа контактов реле делятся на два класса:

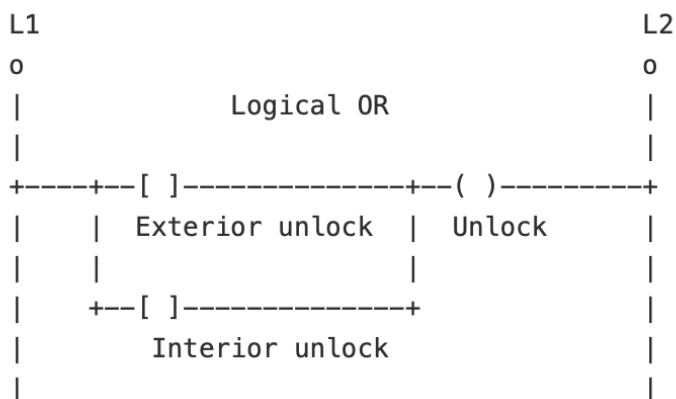
- Нормально разомкнутые (NO, Normally Open), обозначаемые как $-[]-$. В исходном состоянии (без подачи тока на катушку) их контакты разомкнуты, и ток через них не проходит. При активации реле контакты замыкаются, обеспечивая прохождение сигнала.
- Нормально замкнутые (NC, Normally Closed), обозначаемые как $-(\backslash)-$. В исходном состоянии их контакты замкнуты, пропуская ток, а при активации реле --- размыкаются, прерывая цепь.

Дополнительными элементами диаграмм являются актуаторы --- катушки реле, обозначаемые как $-()-$ (нормально неактивные) и $-(\backslash)-$ (нормально активные), а также шины питания: L1 --- шина с фазой (линейное напряжение) и L2 --- шина с нейтралью (нулевой провод), обеспечивающие энергоснабжение системы.

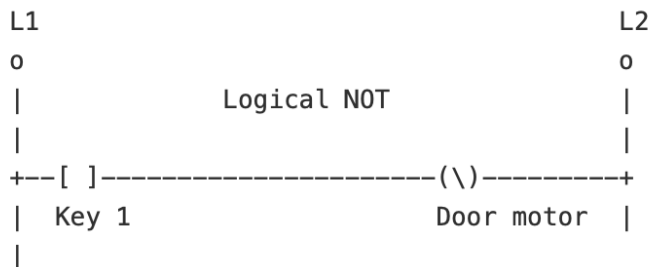
Логическое "И" (AND). Реализуется последовательным соединением нормально разомкнутых контактов. Например, в схеме с двумя ключами, подключёнными между шиной питания L1 и Door motor, ток проходит только при одновременной активации обоих реле. Например: дверь открывается только при повороте двух ключей.



Логическое "ИЛИ" (OR). Контакты реле соединяются параллельно. В такой конфигурации, как схема с аварийной кнопкой (Exterior unlock) и датчиком дыма (Interior unlock), ток поступает на нагрузку (Unlock) при замыкании любого из контактов. Это позволяет активировать сигнализацию либо вручную, либо автоматически при обнаружении возгорания



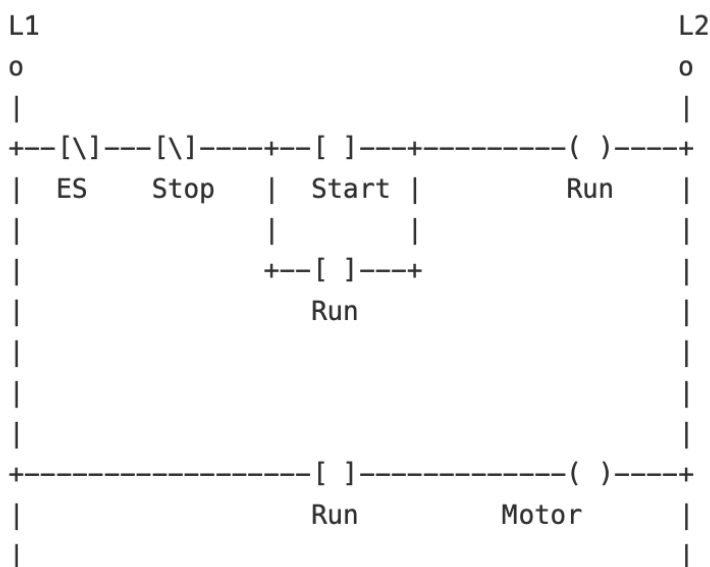
Логическое "НЕ" (NOT). Используется нормально замкнутый контакт. В состоянии покоя цепь замкнута, и ток поступает на нагрузку (например, индикатор «Дверь открыта»). При активации реле контакт размыкается, прерывая цепь. Таким образом, индикатор горит, пока дверь закрыта, и гаснет при её открытии, обеспечивая инверсию сигнала.



24. Опишите классическую схему включения электродвигателя с аварийной остановкой. Объясните, как работает обратная связь.

Основными компонентами системы являются:

- Нормально разомкнутая кнопка Start.
- Нормально замкнутая кнопка Stop.
- Самоподдерживающееся реле.
- Аварийный выключатель EMG Stop с нормально замкнутыми контактами.



При нажатии кнопки Start ток протекает через кнопку Stop, активируя катушку, которая замыкает свои контакты. Это позволяет реле оставаться включённым даже после отпускания кнопки Start, что обеспечивает работу двигателя.

Для остановки системы достаточно нажать кнопку Stop, что приводит к разрыву цепи и отключению катушки, в результате чего двигатель прекращает работу. В случае аварийной ситуации кнопка EMG Stop физически разрывает цепь,

обеспечивая отключение системы даже в случае, если реле зафиксировано в активном состоянии.

При нажатии пользователем на кнопку Start наблюдается небольшая задержка, после которой возникает сигнал Run, а спустя некоторое время активируется Motor. Следует отметить, что уровень тока остается неизменным даже после отпускания кнопки Start.

Обратная связь - реле run начинает самоподдерживать свое состояние

4 ЛЕКЦИЯ

25. В чём заключается принцип развития иерархических систем Седова? Иллюстрируйте применительно к вычислительным платформам.

Принцип развития иерархических систем Седова - для того, чтобы у нас появилось многообразие на следующем уровне организации системы, нам необходимо ограничить его на нижележащих уровнях. Пример: чем больше операционных систем у нас будет, тем меньше разнообразия хорошего софта мы получим; чем больше языков программирования, тем меньше у нас будет библиотек для них.

Это означает, что упорядоченность и стандартизация базовых компонентов позволяют системе достигать большей сложности, гибкости и эффективности на уровне надсистем.

26. Что такое программируемые логические контроллеры (ПЛК) и какова область их применения?

ПЛК - специальная разновидность ЭВМ, в котором как правило есть много клеммников и разъемов. Задача (и один из вариантов использования) в том, что на компьютере есть возможность программировать данные контроллеры как схемы реле или функциональных блоков без изучения кода и программирования.

Применяются в системах управления в промышленности, в системах с традиционно высокими требованиями к надежности. Можно применять для модернизации областей, где применялись релейные шкафы (огромные релейные схемы, вот их можно заменить куда меньшей по габаритам схемой. Кроме того эта схема будет надежнее за счет отсутствия механических частей)

27. Какова структура программируемых логических контроллеров (ПЛК)? Причины отделения инструментальной составляющей?

Модули ввода. Эти модули преобразуют поступающие сигналы от различных сенсоров (температуры, давления, положения и др.) в цифровой формат, понятный процессору. ->

Центральный процессор представляет собой вычислительное ядро системы. Он обрабатывает полученную информацию по заданной программе, реализуя алгоритмы управления технологическим процессом. ->

Модули вывода преобразуют управляющие сигналы от процессора в форму, пригодную для воздействия на исполнительные механизмы --- клапаны, двигатели, сигнальные устройства.

Каждая стрелка снабжена оптической изоляцией. Позволяет отделить контроллер от объектов, с которыми он интегрирован (если что-то пойдет не так на входе или выходе, это не выведет из строя сам контроллер). Вся система имеет источник питания.

Инструментальная составляющая (ПК с ПО для разработки) отделен от ПЛК для безопасности и надежности, разделения среды разработки и исполнения, гибкости разработки, да и просто в этом и есть вся разница между ПЛК и реле, есть возможность написать софт на виртуальном железе и зашить эту логику в шкаф. Ну и просто это в целом не нужно вообще.

28. Каковы особенности аппаратного обеспечения программируемых логических контроллеров (ПЛК)? Объясните причины таких особенностей.

Предназначены для работы в агрессивных средах, имеют защищенные корпуса. Имеют гальваническую развязку модулей ввода и гальваническую развязку модулей вывода. Нет механических элементов. Часто устойчивы к электромагнитным помехам, механическим повреждениям, имеют защищенные выводы, внутренние системы, перезагружающие схему, если она (вдруг) зависнет

Причины очень просты - они разрабатывались под конкретный контекст их применения. А применять их планировалось (так и получилось в целом) как раз в агрессивных средах, в системах, где очень важна надежность, где есть требование к устойчивости к электромагнитным помехам, где схема должна отрабатывать с минимальными рисками и максимально предсказуемо.

Их ключевое преимущество --- предсказуемость работы, достигаемая благодаря чисто аппаратной реализации.

29. Каковы особенности программного обеспечения программируемых логических контроллеров (ПЛК)? Объясните причины таких особенностей.

Современные ПЛК программируются с помощью:

1. Релейно-контактные схемы (LAD, Ladder Diagram) --- визуальный язык, имитирующий электрические схемы на базе реле, удобен для инженеров-электриков, так как сохраняет привычную логику проектирования цепей.
2. Функциональные блоки (FBD, Function Block Diagram) --- позволяют работать с дискретными сигналами, аналоговыми величинами, временными параметрами и событийными состояниями.
3. Последовательные функциональные диаграммы (SFC, Sequential Function Chart) --- применяются для описания сложных технологических процессов с параллельными ветвями выполнения.

4. Структурированный текст (ST, Structured Text) --- текстовый язык для алгоритмически сложных задач с гарантией выполнения.

Ограниченный набор языков программирования (по сравнению с универсальными языками типа C или Python) является осознанным решением, обеспечивающим несколько ключевых преимуществ. Во-первых, это гарантирует надежную остановку системы и предотвращение зависаний. Во-вторых, позволяет точно оценить время выполнения алгоритма, что критически важно для систем реального времени. В-третьих, упрощает верификацию программного обеспечения.

30. Что такое булев базис и какова его роль в вычислительной технике? Приведите примеры.

Основой современной вычислительной техники являются следующие принципы, построенные на булевой алгебре:

двоичная логика - все можно представить в виде некоторого количества единиц и нулей

полный набор булевых функций, комбинируя которые мы можем реализовать любое преобразование из формата А в формат Б: И, ИЛИ, НЕ, штрих Шеффера (х и у сначала подвергаются операции И, а затем отрицаются), стрелка Пирса (выполнение операции ИЛИ над х и у, а затем выполнение обратной операции)

комбинационные схемы - из булевых функций собираем преобразователи, которые работают за один прогон

триггер - хранение состояния

Булев базис - минимальный набор логических операций, через которые можно выразить любую булеву функцию.

Роль:

Основа цифровой логики (все процессоры и микросхемы работают на базе булевых операций)

Минимизация аппаратной реализации (любую схему можно собрать на булевом базисе)

Универсальность (с помощью булева базиса можно описывать арифметические операции, условные переходы)

Примеры:

RS-триггер(собирается на NOR и NAND)

Во всяких сумматорах, вентилях, схемах

31. Что такое двоичное кодирование сигналов? Каковы его достоинства и недостатки? Что такое запретная зона?

У нас есть электрический сигнал - мы говорим о том, что некоторый диапазон значений — это единица, другой интервал — это логический ноль. Далее этот сигнал передается из источника по проводнику в приемник, где мы декодируем его по другим, немного расширенным диапазонам. Те значения, которые не входят ни в один из диапазонов, считаются запретной зоной - те значения, которые, как мы считаем, не должны быть в нашей системе в норме.

И тут мы по сути совершаем метасистемный переход, где грубо ограничиваем весь аналоговый мир цифрой, булевыми значениями.

Достоинства - надежно и помехоустойчиво (сигнал разделяется на логический 0 и логическую 1, с четко определёнными интервалами напряжения), простая арифметика (двоичную арифметику проще реализовать на аппаратном уровне, чем десятичную), погрешности можно рассчитать изначально в процессе проектирования

Недостатки - нечитаемое представление (например, бинарные часы используют формат отображения времени, который существенно отличается от привычного человеку. Он требует специальной подготовки или визуального привыкания); простые десятичные дроби представляются как бесконечные двоичные (это требует специальной обработки при хранении и расчётах и может привести к потере точности)

32. Какова роль машинного слова в устройстве процессора? Что такое Big- и Little-endian?

Один бит — это не интересно и не прикольно. Мы хотим представлять числа покрупнее и поинтереснее, поэтому очень быстро приходим к тому, что составляем биты в некоторые конструкции.

Машинное слово - некоторая совокупность бит, с которым компьютер может "спокойно работать", условно обработать их совокупность одновременно (напр. сложить числа). Оперативная память адресует и хранит данные в виде машинных слов. При обращении программа читает или записывает значения по адресам, оперируя целыми словами. Команды процессора (инструкции) также хранятся как машинные слова и содержат:

- Код операции --- указывает, какую операцию выполнить (например, сложение, переход).
- Операнды --- над какими данными выполняется операция.
Процессор считывает слово из памяти, интерпретирует его и выполняет соответствующее действие

Big endian: представляет собой метод хранения данных. В этом режиме старший байт данных хранится по младшему адресу, а младший байт --- по старшему адресу. Он обычно используется в сетевых протоколах, некоторых форматах файлов и т. д.

Little endian: в отличие от big endian, младшие байты данных хранятся по младшим адресам, а старшие байты --- по старшим адресам (как если бы они хранились справа налево (сначала младшие байты)). Это может быть проще в аппаратной реализации, поскольку CPU начинает с малого адреса при считывании памяти байт за байтом

Сравнение:

- Big Endian ближе к человеческому восприятию --- сначала записываются старшие байты, как мы читаем числа.
- Little Endian --- удобен для процессоров, так как позволяет начинать обработку с младших байтов и экономит на сдвигах.

33. Какие существуют способы кодирования целочисленных данных? Что такое позиционное кодирование, дополнительный код, код Грея, BCD?

В позиционной системе счисления положение числа определяет величину представляемого им значения. Каждая цифра имеет соответствующий вес. Числа в разных разрядах умножаются на соответствующие им веса в соответствии с их позициями, а затем складываются, чтобы получить фактическое значение, представленное числом (на первом месте – 1, на втором – 2, на третьем – 4 и тд).

Метод расчета обратного кода: обратный код положительного числа = ему самому. Обратный код отрицательного числа основано на исходном коде. Знаковый бит остается неизменным, а остальные биты инвертируются.

Дополнительный код - некоторое отображение позиционной системы счисления, для работы (причем удобной работы) с отрицательными числами

Код Грея — это двоичная система кодирования, в которой каждая последующая комбинация отличается от предыдущей ровно одним битом. Такой принцип минимизирует вероятность ошибок, особенно в тех случаях, когда значения переходят из одного состояния в другое (например, при сканировании датчиков или инкременте счётчиков).

BCD - записываем десятичное число по разрядам, каждый разряд десятичного числа - 4 бита. В отличие от обычной двоичной системы, BCD не преобразует всё число целиком, а кодирует каждую цифру по отдельности. Это удобно интерпретировать пользователю, но занимает больше места.

34. Какие существуют способы кодирования бинарных данных? Что такое Base64, Base58?

Base64 — это способ представления двоичных данных в текстовом виде, при котором каждые 3 байта (24 бита) исходной информации кодируются в 4 текстовых символа из специального алфавита из 64 символов. Этот формат широко используется для передачи и хранения данных в системах, где допустим только текст (например, в email, JSON, XML, URI и т. п.). Мы просто берем данные по 6 бит, сопоставляем с алфавитом. Все. Если надо дополнить есть символ "=" которыми можно дополнить до кратности длины до 4 символов.

Недостатки Base64:

- Увеличение объёма данных: Кодировка увеличивает объём примерно на 33%, поскольку каждые 3 байта данных превращаются в 4 символа.
- Отсутствие защиты информации: Несмотря на то, что Base64 делает данные визуально нечитаемыми, она не обеспечивает ни шифрование, ни аутентификацию. Закодированную строку можно легко декодировать обратно — это не средство безопасности.

Base58 — это модифицированная версия кодировки Base64, разработанная для удобного отображения данных в человекочитаемом виде.

В ней исключены символы, которые легко перепутать визуально: 0 (ноль), O (заглавная o), l (заглавная и), I (строчная эль), + и /. Такой подход делает Base58 особенно удобной для ручного ввода и копирования в интерфейсах с участием человека.

Наиболее известное применение --- адреса в криптовалютных системах, таких как Bitcoin

Недостатки Base58:

- Отсутствие стандартизации: В отличие от Base64, кодировка Base58 не имеет единого официального стандарта. Разные реализации могут использовать различный алфавит, правила проверки или структуры, что вызывает проблемы совместимости между системами.
- Отсутствие механизмов безопасности: Base58 --- это метод кодирования, а не защиты. Она не обеспечивает шифрование, контроль целостности или аутентификацию.
Любой, кто получает строку, закодированную в Base58, может её легко декодировать.

35. Что такое комбинационные схемы? Что такое переходный процесс и с чем связаны задержки?

Комбинационная схема — это логическая схема, составленная из набора вентилей, которая реализует заданную таблицу истинности.

Логические вентили являются основными блоками для реализации двоичных логических операций. Они выполняют логическую обработку входных двоичных сигналов (0 и 1, которые обычно соответствуют низкому и высокому уровням в схеме) и выводят соответствующие двоичные сигналы.

Комбинационные схемы строятся из элементарных логических компонентов (вентилей), таких как AND, OR, NOT, BUF (буфер просто повторяет вход, надо чтобы усиливать какой-то сигнал и повторять его дальше). Каждый компонент обрабатывает бинарные входы (0 или 1) и выдаёт бинарный выход, согласно определённой логической функции.

Каждая функция — это некоторое отображение одного множества в другое. Любое значение любой сложности фиксированного диапазона фиксированной точности может быть представлено в виде совокупности бит. Следовательно любую функцию можно реализовать как отображение одного бинарного значения в другое. Значит можно построить комбинационную схему, которая это реализовывает.

Можно построить через таблицу истинности (черный ящик) Там просто строим таблицу истинности, записываем это как совокупность булевых функций (в канонической форме ДНФ, КНФ) и дальше там можно минимизировать, оптимизировать

Через алгоритмизацию (белый ящик, структурность/этапность расчета) Может быть проще и компактнее, но это творческий процесс, нет четкого алгоритма

Переходный процесс - кратковременный ложный сигнал на выходе комбинационной схемы, возникающий из-за разной задержки распространения сигналов через логические элементы.

Задержки и накопления ошибки:

Каждый логический элемент вносит задержку (в сложных схемах задержки суммируются)

Накопление ошибки в физическом процессе, что может привести к ошибке на логическом уровне. Проблему решают буферы/повторители.

36. Как связаны комбинационные схемы с параллелизмом уровня бит?

Параллелизм уровня бит - узлы с параллельным включением работают параллельно. Применимо буквально везде в цифровой схемотехнике и комбинационных схемах.

Параллелизм уровня битов — это способность обрабатывать сразу несколько бит данных в пределах одного машинного слова.

В итоге комбинационные схемы физически реализуют принцип параллелизма на уровне бит, преобразуя входные данные в выходные мгновенно для всех бит слова.

37. Какие состояния существуют в комбинационных схемах (0, 1, x, z) и что они означают?

0 и 1, в цифровых схемах 0 и 1 являются наиболее базовыми и понятными логическими состояниями, обычно соответствующими низкому уровню и высокому уровню, и являются основным представлением цифровых сигналов.

z --- Состояние, указывающее на отключенное или плавающее состояние, то есть источник данных (провод) не подключен к допустимому сигналу и не сохраняет логическое значение 0 или 1. В общем случае состояние не кодируется как сигнал нулевого уровня. В это время линия не оказывает существенного влияния на другие цепи, что эквивалентно «изоляции» от цепи.

x --- неизвестное состояние, представляющее неопределённое или произвольное значение. Оно может возникнуть при выполнении некорректной или неразрешённой операции, например делении на 0 или попытке вычислить квадратный корень из отрицательного числа.

5 ЛЕКЦИЯ

38. Каковы особенности реализации "условного оператора" в комбинационных схемах?

Глобально есть два сценария, как мы можем получать условный оператор. Эти два сценария покрывают практически все наши потребности, это:

Есть два входа и мы хотим выбрать, из какого входа получить данные

Решение - мультиплексор, у него просто три входных переменных (I0, I1, select), одна выходная. Один из входов выбирает, какой из других двух входов будет транслироваться на выход

Есть один вход и мы хотим выбрать, на какой выход передать данные. На какой из двух приемников отправить сигнал

Тут у нас проблема - мы не можем остановить распространение сигнала и "не передать" что-то куда-то. Появляется D-триггер, который может "защелкнуть" значение по клок. И если нам надо выбрать, в какой триггер принять данные, то мы втыкаем "И", которое пропускает клок только если это то устройство, которое должно принять этот сигнал

39. Что такое триггеры в цифровых схемах? Каковы варианты их использования?

Триггер - класс электронных устройств, обладающих способностью длительно находиться в одном из двух устойчивых состояний и уметь менять их под действием внешних факторов

Позволяют зафиксировать состояние линии на длительное время (всевозможные ячейки памяти). Сократить задержку в комбинационной схеме, что позволит одной части комбинационной схемы работать уже над другими входными данными, пока остальная схема доделывает прошлые. Позволяют создавать многотактовые схемы

Использование триггеров позволяет:

- разделить большие комбинационные схемы на этапы (аналогично буферу);
- хранить состояние внутри схемы и выполнять операции пошагово;
- реализовать счётчики, регистры, флип-флопы и другие компоненты цифровой логики.

40. Что такое D-триггер и RS-триггер? Какие существуют варианты условия изменения состояния?

D-триггер

- Входы:
 - Терминал данных D (ввод данных)
 - Терминал тактового сигнала CLK (ввод такта)
- Выход:
 - Терминал Q (вывод состояния)
- Режим работы:
 - На переднем фронте тактового сигнала (CLK) значение с входа D передается в выход Q.
 - На заднем фронте такта состояние Q остается неизменным (запоминается).

RS-триггер

- Входы:
 - Терминал установки S (Set)
 - Терминал сброса R (Reset)
 - Терминал тактового сигнала CLK
- Выход:
 - Терминал Q (вывод состояния)
- Режим работы:
 - Требуется тактовый сигнал (синхронный триггер).
 - На заднем фронте такта (CLK) состояние выхода Q обновляется в соответствии с входами:
 - Если $S = R = 0 \rightarrow Q$ остается прежним.
 - Если $S = 0, R = 1 \rightarrow Q = 0$ (сброс).
 - Если $S = 1, R = 0 \rightarrow Q = 1$ (установка).
 - Если $S = R = 1 \rightarrow$ состояние нестабильно (запретное состояние).

Есть D-триггер - запоминает состояние входа и дает его на выход. RS - есть два входа один устанавливает выходное значение, другой сбрасывает

Условия изменения состояния:

По уровню: изменяет состояние, пока входной сигнал удерживается на определенном уровне (высоком или низком).

Пример:

В триггере SR, если тактовый сигнал CLK удерживается на высоком уровне, любые изменения на входах S и R немедленно отражаются на выходе Q. Когда CLK становится низким --- триггер "замораживается".

Особенности:

- Более чувствителен к шумам и дребезгу контактов.
- Используется в простых схемах, где важна экономия ресурсов.

По фронту: реагирует только в момент изменения уровня сигнала --- при нарастающем (↑) или спадающем (↓) фронте.

Пример: D-триггер срабатывает только в момент перехода CLK с 0 на 1 (нарастающий фронт) и сохраняет значение входа D в выходе Q.

Особенности:

- Менее подвержен влиянию шума, так как реагирует только на короткий момент.

41. Что такое пространственные и временные вычисления? Как они могут быть использованы для оптимизации процессоров?

Суть в том, что мы, например, можем сделать один большой сумматор, на 16 бит, который сразу их сложит (в один такт), но займет больше места (требует больше логических вентилях). Либо сделать сумматор на 8 бит, который делит операцию на два такта: младшие 8 бит → старшие 8 бит и требует дополнительной системы управления (контроль переноса, синхронизация стадий), тогда будет вычисляться больше времени.

Возможность перехода от временных вычислений к пространственным позволит организовать параллелизм уровня задач, инструкций, бит. И он же позволит сделать конвейерную организацию работ за счет многотактовости

Конвейер --- это архитектурный подход, при котором разные стадии обработки данных выполняются одновременно, но над разными входами.

42. Что называют синхронной схемотехникой? Каковы её достоинства и недостатки?

В синхронных схемах все элементы работают под управлением единого тактового сигнала. Это позволяет:

- строго упорядочить операции;
- синхронизировать все модули схемы;
- упростить прогнозирование и отладку.

Синхронной схемотехникой называют такую схемотехнику, в которой изменение элементов памяти происходит одновременно по флону, т.е. возможно существование многотактовых схем

Синхронные схемы позволяют:

- реализовать конвейерные архитектуры, где каждое звено работает независимо, но с общей тактовой координацией;
- повышать тактовую частоту, разбивая сложную комбинационную логику триггерами;
- упростить проектирование, поскольку вся схема "живёт" в унисон с одним источником времени.

Недостатки:

Тактовая частота схемы определяется самым медленным блоком

43. Почему цифровая схемотехника оперирует уровнями, а не сигналами? Какие возможности это открывает и какие проблемы создаёт?

Сигнал это что-то мгновенное во времени (буквально аналогия отправить запрос на сервер, он мгновенно ушел и мы когда-то получим ответ). Уровень же восстанавливается, и пока его никто не трогает он будет стоять.

Цифровая схемотехника использует дискретные уровни напряжения(0/1) вместо аналоговых сигналов.

Почему уровни а не сигналы (в понимании пакета данных) здесь речь о том, что уровень, например, нельзя не послать в отличие от сигнала.

Возможности:

- Высокая надежность (возможна работа с помехами)
- Автоматизация проектирования
- Можно строить кеши, конвейеры

Проблемы:

- Точный контроль уровней(если напряжение проваливается, то будут ошибки (пысы: можно пофиксить буферами, повторителями и не делать длинных проводников))
- Задержки распространения сигналов
- Потребление энергии при переключении синхросигнала

44. Каковы ключевые тенденции в производстве радиоэлектронной аппаратуры и связанные с этим проблемы?

Монтаж радиоэлектронной аппаратуры - процесс сборки отдельных частей (реле, транзисторы) в одно целое как нам надо

Тенденции:

1. Гонка за интеграцией. Это постоянная борьба за размещение максимального количества функциональных элементов в минимальном

объеме. Яркий пример --- современные беспроводные наушники, где в крошечном корпусе умещаются несколько микрофонов, радиомодули, излучатели и тд

2. Снижение затрат на производство изделия. Стоимость изделия радикально зависит от тиража. При массовом производстве (от 10 000 шт) это выгодно
3. Эволюция производственных цепочек. Рост сложности производства является неизбежной платой за прогресс.

45. Какие виды монтажа электроники существуют? Как они соотносятся по гибкости и сложности производства?

Навесной монтаж - способ монтажа электронных схем, при котором расположенные на изолирующем шасси радиоэлементы соединяются друг с другом проводами или непосредственно выводами. (буквально детали прикручены к шасси и соединены проводами)

Монтаж на печатную плату - есть печатная плата — это пластина из диэлектрика, на поверхности и/или в объеме которой сформированы электропроводящие цепи электронной схемы. Электрическое и механическое соединение компонентов (буквально детали вставляются в отверстия и запаиваются, механическое соединение деталей и электронных компонентов в последовательности, обеспечивающий их требуемое расположение и взаимодействие для тех. требований)

Поверхностный монтаж (разновидность монтажа на печатную плату) - маленькие детали припаяны к поверхности платы без отверстий. (буквально фиксируем электронные компоненты непосредственно на внешние стороны платы, выводы компонентов монтируются на металлизированные площадки на печатной плате, тем самым получаем электрические цепи). Ключевым элементом процесса является паяльная паста --- специальная смесь припойного порошка и флюса, которая наносится на контактные площадки через трафарет.

Штырьевой монтаж - детали с ножками вставляются в отверстия платы и запаиваются с обратной стороны.

Гибкость:

Навесной монтаж - прост в производстве, в подготовке производства и гибок (но он никак не поддается автоматизации, так как вручную собирается монтажниками)

Монтаж на печатную плату - гибкость в ручной сборке и ремонте

Поверхностный монтаж - гибкость в автоматизации

Каждый способ имеет свои плюсы и минусы => получаем гибкость и разнообразия выбора под разные цели и задачи

46. Какова производственная цепочка поверхностного монтажа? Каковы её этапы?

Производство плат с поверхностным монтажом начинается с нанесения паяльной пасты через трафарет. Затем автомат расставляет компоненты на свои места. Плата отправляется в печь, где паста плавится и припаивает детали. После этого идёт проверка: сначала автоматика осматривает плату, потом тестирует её работу. Для сложных чипов вроде процессоров дополнительно используют рентген, чтобы проверить невидимые соединения под корпусом.

47. Какова производственная цепочка кремниевого производства? Каковы особенности формирования цены изделия?

Интегральные схемы --- микроэлектронное изделие, выполняющее определенную функцию преобразования, обработки сигнала, накопления информации и имеющее высокую плотность электрически соединенных элементов, которые с точки зрения требований к испытаниям, приемке, поставке и эксплуатации рассматриваются как единое целое.

Производство интегральных микросхем — это сложный и многогранный процесс, требующий высокой точности и продвинутых технологий.

Он очень похож на производственный процесс печатных плат, но в сильно уменьшенном размере, очень плотно это все.

Этапы:

1. Кремниевый песок -> выращиваем монокристалл кремния, высочайшей степени очистки (буля), потому что на мельчайших размерах (4нм) даже небольшое искривление может привести к большим проблемам
2. Пилим булю очень точной плитой на пластины ваферы (wafer)
3. Далее на кремниевую подложку (вафер) наносят материал для рисунка:
 - нанести фоторезист
 - экспонировать через фотошаблон
 - удаляем то, что не закреплено
4. После делаем проводящие, непроводящие, полупроводниковые слои (выстреливая ионами в кремний) и так далее
5. Дальше вафер режем на чипы
6. Корпусируем (выводим микро проводки на более толстые, погружаем в корпус)

Про цену: Вообще сделать маску для 3 этапа стоит дорого

6 ЛЕКЦИЯ

48. Какие трудности связаны с производством аппаратного обеспечения (подготовка производства, элементная база)?

Логистика

Склады

Специалисты

Производственная цепочка (**github actions**)

Тестирование

Упаковка

Дистрибуция (**docker hub**)

Гарантийный ремонт (**support**)

(жирным выделено то, что есть и в производстве программного продукта)

49. Какие трудности связаны с эксплуатацией аппаратного обеспечения (амортизация, физический доступ, среда эксплуатации)?

Амортизация. Выход оборудования из строя (программы сами по себе не деградируют, не выходят из строя)

Необходимость физического доступа

Особенности среды эксплуатации

Долгосрочные эффекты (ну типа вот этих усов, которые олово само из себя выкристаллизовывает)

Поддержка, отладка и восстановление

Обновление: сложность изменений (например, обновить аппаратное обеспечение на вояджере, который уже за пределами солнечной системы. Или на маяке посреди океана)

50. Какие подходы к решению проблемы "устаревающей аппаратуры" существуют с точки зрения аппаратуры и ПО?

Аппаратное обеспечение со временем стареет двумя способами – физически и морально. Физическое старение связано с износом деталей: компоненты со временем теряют свойства, ломаются или выходят из строя из-за усталости материалов, перегрева, коррозии и других факторов. Моральное устаревание наступает, когда появляются новые, более дешёвые или производительные аналоги устройств; даже полностью исправное оборудование перестаёт удовлетворять текущим требованиям по скорости и функциям. Таким образом, срок службы "по факту" часто оказывается короче теоретически возможного.

Техническое обслуживание (ТО) – первоочередной способ продления: регулярная очистка от пыли, своевременная замена термопасты и вентиляторов, проверка питания и охлаждения позволяют предотвратить перегрев и преждевременный износ. Перегрев электронных компонентов является одной из основных причин отказов: при избыточной температуре материал деградирует, возникают трещины, течёт ток утечки (особенно в конденсаторах). Поэтому эффективные методы предупреждения перегрева (качественные системы охлаждения, правильная

вентиляция корпуса) широко обсуждаются как ключевые для долгосрочной надёжности.

Кроме регулярной профилактики, продлению службы способствуют модернизация и апгрейд. Замена умирающих компонентов (например, обновление оперативной памяти, улучшение блока питания) позволяет сохранить актуальность системы. Кроме того, правильное хранение и эксплуатация (защита от влажности, перепадов напряжения, электростатических разрядов) предотвращают физическое старение. Виртуализация приложений позволяет запускать устаревший софт на современных устройствах без конфликтов: например, бухгалтерское приложение, написанное для старой ОС, может работать в виртуальной оболочке на современной машине независимо от реальной среды. Это избавляет от необходимости сохранять устаревшие компьютеры ради старого ПО.

51. Что такое концепция 2-этапного производства? Какие подходы к "конфигурированию" продукции существуют?

Сначала производство "универсальной" компьютерной системы, т.е. не специально под задачу, а пытаемся сделать так, чтобы ее можно было конфигурировать под решение задачи.

А сверху над этим строим прикладное ПО, которое заставляет железо работать над нашей задачей, уже конкретной.

Количество промежуточных уровней оно произвольно в целом. Есть подходы к обеспечению этой концепции: закладываем при проектировании, например, чтобы можно было вставить платы расширения, можно было конфигурировать там и все такое.

- Сборка аппаратной платформы: Процесс физического соединения основных компонентов вычислительной системы:

Центральный процессор (CPU)

Оперативная память (RAM)

Материнская плата

Жёсткие диски (HDD) и твердотельные накопители (SSD)

Платы расширения

Блок питания и корпус

Система охлаждения

Разъёмы и кабели

- Комплектация: процесс предварительной подготовки наборов комплектующих для сборки изделий. На производстве заранее собирают "наборы" – комплекты деталей для конкретной модели. Это ускоряет сборку

- Реконфигурация: после физической сборки этап программной настройки (реконфигурации) расширяет возможности платформы без изменения "железа" (конфигурирование данных)
- Программирование: кусок инструкции, который определяет, как должен работать вычислитель прямо здесь и сейчас

52. Каково определение программной системы согласно OMG Essence? Каковы её части?

Программная система - система, собранная из программного обеспечения, аппаратного обеспечения и данных, которые являются частью этой системы (дают ей первичную ценность).

Пример данных - шрифты

53. Что такое программное и аппаратное обеспечение? Что означают понятия Hardware и Software? Сопоставьте их.

Аппаратное обеспечение - электронные и механические части вычислительного устройства, входящие в состав системы или сети, исключая программное обеспечение и данные.

Другими словами: аппаратное обеспечение — это железка из каких-то *обычно* радиоэлектронных элементов, которые, взаимодействуя между собой, позволяют реализовать какую-то прикладную задачу.

Программное обеспечение - совокупность программ, системы обработки информации и программных документов, необходимых для эксплуатации. Позволяет аппаратному обеспечению вычислительно системы выполнять вычисления или функции управления

Software - что-то гибкое, изменяемое, мягкое. Легко/быстро/дешево поменять

Hardware – что-то твердое, тяжело/долго/дорого поменять

Мы не ставим соответствия ПО и software. Пример, Minix - популярное ОС, которая стоит внутри процессоров, это ПО, но это hardware.

54. Какие возможности открывает программное обеспечение (цикл разработки, гибкость, контроль, и т.д.)?

Быстрый цикл разработки

Легко заменяется прямо у пользователя

Пользователь как Beta-тестер

Возможно удаленное обновление, в том числе без информирования согласия

Сервис - вершина владения компьютерной системы

Процесс создания и внедрения ПО автоматизируется (CI/CD)

Высокая сложность программ

55. Чем отличается типовое проектирование Hardware/Software от совместного (CoDesign) проектирования Hardware/Software? Каковы достоинства и недостатки?

Типовое HW/SW проектирование:

- Проектируем аппаратуру: система команд, интерфейсы, схемотехника
- Пишем программу (используя инструментарий аппаратуры): прикладная задача язык и библиотеки

Проблемы:

Шаблонное проектирование (разделение аппаратчиков и программистов приводит к шаблонным решениям, потому что одни не знают что делать, а другие не знают, что им надо, типа "что дали, то и хорошо")

Высокие интеграционные риски (может получиться, что HW+SW вместе просто не заработают, и такое часто бывает)

Последовательная разработка (растягивает сроки)

Совместное (CoDesign) проектирование Hardware/Software:

Пробуем работать без жесткого разделения на HW SW и это разделение оставить на поздние этапы проектирования. Как можно дольше остаёмся на уровне моделей, где можно двигать границу Hw/Sw. На этом выехал Node.js на котором можно написать и фронт и бэк, что стерло между ними границу и позволило удобнее работать

56. Что такое "Модель вычислений"? В чём назначение моделей вычислений? Приведите примеры.

Модель вычислений - средство формального описания вычислительного процесса. Это абстрактное, упрощенное представление компьютера или вычислительного процесса, определяющее допустимые операции, структуры данных и правила их обработки. Она абстрагируется от конкретного «железа» (процессоров, архитектур), позволяя анализировать алгоритмы на основе набора элементарных действий. Используется для формальных верификаций, теоретического доказательства алгоритмов, в дизайне языков программирования. Она определяет возможности вычислительной машины, характеризует, как выполняется программа: возможные состояния вычислителя, их последовательность, возможность переходов.

Примеры: машина Тьюринга, лямбда-исчисления

57. Что такое последовательные модели вычислений? Приведите примеры. Как в них представляется вычислительный процесс?

Последовательные модели вычислений - такие модели вычислений, в которых инструкции, события, действия идут строго последовательно, одна за одной (последовательность переходов состояния).

Очень яркий пример - конечные автоматы: задаем состояния и условия перехода состояний между ними; машина Тьюринга: есть лента с ячейками, читает символ, меняет его по правилу, переходит в новое состояние

58. Что такое машина Тьюринга и почему она важна для теории вычислений?

Машина Тьюринга – абстрактная вычислительная модель. Содержит следующие элементы:

- неограниченная двусторонняя лента (возможны машины Тьюринга, имеющие несколько бесконечных лент), разделенная на ячейки
- устройство управления, которое может находиться в конечном числе состояний.

Управляющее устройство может перемещаться по ленте влево и вправо, читать и записывать в ячейки символы некоторого конечного алфавита. Выделяется специальный пустой символ, который заполняет все ячейки ленты, кроме ячеек с входными данными.

Управляющее устройство работает в соответствии с правилами перехода, представляющими собой алгоритм машины Тьюринга. Каждое правило перехода предписывает машине, в зависимости от текущего состояния и символа, наблюдаемого в текущей ячейке, записать в эту ячейку новый символ, перейти в новое состояние и переместиться на одну ячейку влево или вправо. Некоторые состояния машины Тьюринга могут быть помечены как конечные. Переход в них означает окончание алгоритма.

Основной частью машины Тьюринга является явное разделение потока управления (управляющее устройство) и потока данных (лента).

Она не может быть реализована на практике из-за того, что бесконечных лент не найти, обладает полнотой по Тьюрингу (можно реализовать любой известный алгоритм), есть проблема остановки (нет возможности понять для любого алгоритма, остановится ли он когда-нибудь)

59. Что такое Random Access Machine? Каково её устройство и место сегодня? Какова её связь с машиной Тьюринга?

Random Access Machine - берем ленту из машины Тьюринга и заменяем ее адресуемой памятью с конечным или бесконечным количеством регистров. В итоге получаем то же самое (только можно читать и писать куда нам надо в память, а не последовательно как на ленте). Есть инпут, есть РС с помощью которого берем из памяти что нам надо, выполняем это действие, записываем в регистры и что-то отдаем на аутпут. В целом все современные процессоры описываются этой random access machine (с небольшими оговорками)

А связана она с машиной Тьюринга тем, что обе модели формально эквивалентны и могут решать одни и те же задачи. Ну и Random Access Machine это своего рода

математическая абстракция, но просто очень и очень приближенная к реальным процессорам.

60. Что такое функциональные модели вычислений? Приведите примеры. Как в них представляется вычислительный процесс?

Функциональные модели вычислений – представление вычислительного процесса как совокупности символов и правил их преобразований. Не задают жестко последовательность вычислений

Примеры - арифметика, лямбда исчисление, Рефал

Лямбда исчисление:

4 вида выражений: переменные, константы, комбинация, абстракция

3 вида преобразований: переименование переменных, переименование функции, сокращение аргумента

Если в последовательных моделях вычислений, у нас есть какая-то машина, которая шаг за шагом выполняет какие то действия, то в функциональных моделях мы больше работаем по математическим законам и правилам. В случае функциональных моделей данные и программы неразличимы, и мы можем оперировать данными как программами и наоборот.

61. Что такое параллельные модели вычислений? Приведите примеры. Как в них представляется вычислительный процесс?

Параллельные модели вычислений – системы: включающие несколько взаимодействующих процессов:

Kahn process networks - однонаправленные каналы с неограниченными буферами (пример яп Go)

Акторные модели (отправка асинхронных сообщений в почтовые ящики, пример банковские транзакции да и в целом в распределенных системах)

События в заданные моменты времени (процессы выполняются в заданные моменты времени, реагируя на события)

Общая память (потoki/процессы шарят общие переменные)

Синхронизированные потоки данных (обмен данными по тактам)

Вычислительный процесс представляется: взаимодействие – через общую память, события; параллелизм – несколько ядер цпу/гпу, чередование потоков на одном ядре

7 ЛЕКЦИЯ

62. Что такое Model-Driven Engineering (MDE)? Чем цепочка трансформации в MDE отличается от языков высокого уровня? Приведите примеры.

MDE - подход к разработке программных компьютерных систем, который заключается в том, что не надо давать программисту использовать япы, а надо дать использовать набор моделей, в рамках которых он сможет описать ту прикладную задачу, те процессы, с которыми он непосредственно работает.

Отвечая на вопрос о цепочке трансформации: использование моделей подразумевает отвязку от вычислительной платформы. Есть ведь императивное (когда мы говорим как делать), а есть декларативное (когда мы говорим что делать) программирование. Цепочка трансформации состоит в том, что в модели мы разрабатываем безотносительно того где и как это будет выполняться, появляется этот доп слой, который уже трансформируется в что-то, что будет выполняться на конкретной вычислительной платформе

Пример case-технологии – набор инструментов и методов программной инженерии для проектирования ПО, обеспечивающих качество, надёжность и поддерживаемость продуктов

Другой пример - предметно-ориентированные языки (Domain Specific Language). Это подход, позволяющий писать на языке, разработанном специально для нашей предметной области. Например, все современные ЯПы, это DSL специально для алгоритмов. SQL - язык для доступа к реляционным БД.

63. Что такое универсальный процессор? Каковы его особенности и свойства?

В основе всего лежит двухэтапное производство - когда мы разрабатываем некоторый универсальный вычислитель, а потом программируем его.

Информационный процессор - абстракция, которая позволяет обозначить устройство, которое может преобразовывать информацию. Включает в себя:

1. Ввод
2. Процессор
3. Хранилище
4. Вывод

Универсальный процессор - информационный процессор, который позволяет решать широкий круг задач, настройка которых производится после производства. (как раз подходит для двухэтапного производства)

Особенности:

- Универсальность определяется широтой круга решаемых задач, а не вычислительной полнотой.
- То, что универсально в теории, не всегда универсально в практике. Теоретически универсальные системы (например, Машина Тьюринга, язык Brainfuck) могут выполнять любые вычисления, но их практическая применимость ограничена.

- Универсальность противоречит эффективности (чем универсальнее платформа, тем менее она эффективна. Реализация "под задачу" всегда будет эффективнее универсального решения)
- Современный процессор - совокупность software и hardware

Свойства:

- можно использовать в 2-этапном производстве
- полнота по Тьюрингу (почти всегда для процессора)
- Отсутствие "серьезных" ограничений на "объем" программы
- Изменяемость ПО — это необходимое условие для двухэтапного производства

64. В чём заключается противоречие между универсальностью и эффективностью в разных видах процессоров (СБИС, FPGA, CGRA, GPU, DSP, CPU)?

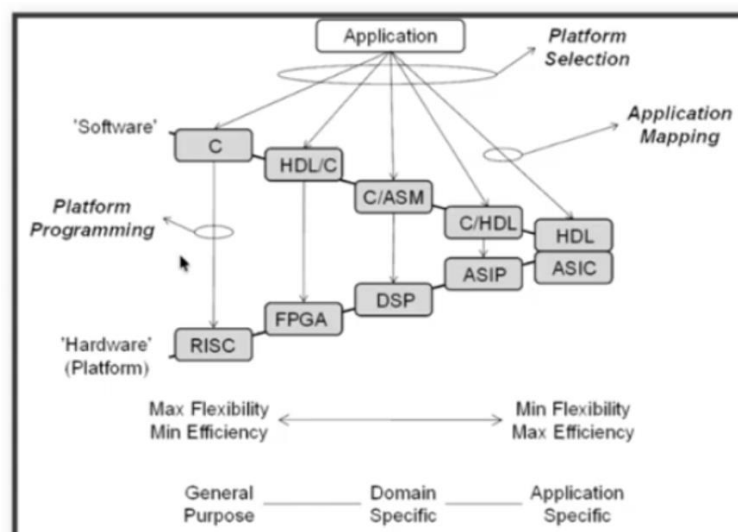
Когда мы делаем универсально, мы жертвуем эффективностью и наоборот - эффективное решение зачастую очень не универсально и пишется буквально по месту, максимально "одноразово" и под задачу

Наблюдается общая зависимость: увеличение универсальности приводит к росту энергопотребления, в то время как специализация повышает энергоэффективность.

Специализированные архитектуры демонстрируют высокую производительность в расчёте на затраченную энергию.

Универсальные платформы требуют больше энергии для выполнения аналогичного объёма операций.

65. Как соотносятся вычислительные платформы и языки описания вычислительного процесса? Как это связано с эффективностью и областью применения?



Вопрос заточен под эту картинку. Сверху перечислены языки описания вычислительного процесса, каждому из которых соотносится вычислительная платформа.

Вторая часть вопроса про конфликт гибкости (=то, насколько широкая область применения у инструмента) и эффективности. Потому что слева у нас максимально широкая область применения - ну си он просто везде используется, можно компилировать, запускать где угодно и на очень многих вычислительных платформах, однако мы теряем в эффективности (энергоэффективности, скорости и так далее) за счет того, что наше решение отдаляется от вычислительной платформы (например, может не учитывать приколы конкретного процессора, особенности работы с ним).

Справа же рассматриваются вещи, которые очень эффективны, но заточены под конкретную прикладную задачу с очень узкой областью применения.

RISC (Reduced Instruction Set Computer) — Компьютер с сокращённым набором команд. Это универсальные процессоры (как в iPhone или современных ноутбуках), которые умеют выполнять любые задачи за счёт простых и быстрых инструкций.

FPGA (Field-Programmable Gate Array) — Программируемая логическая интегральная схема (ПЛИС). «Чип-конструктор», внутреннюю структуру которого можно перенастроить программно уже после его изготовления.

DSP (Digital Signal Processor) — Цифровой сигнальный процессор. Специализированный чип, «заточенный» под быструю математическую обработку потоков данных (звук, видео, радиосигналы).

ASIP (Application-Specific Instruction set Processor) — Процессор с системой команд под конкретное приложение. Это нечто среднее: база от обычного процессора, но с добавлением уникальных команд для ускорения специфических задач (например, шифрования).

ASIC (Application-Specific Integrated Circuit) — Интегральная схема специального назначения. Чип, созданный на заводе под одну-единственную задачу (например, чип в банковской карте). Его нельзя перепрограммировать, но он работает быстрее всех.

66. Что такое архитектура и микроархитектура процессора? В чём различие между ними?

Архитектура процессора отличается от архитектуры систем (например, программных) и включает:

1. Набор инструкций, способ их выполнения.
2. Набор правил и ограничений, которые программисту необходимо учитывать для того, чтобы процессор мог эффективно и автономно выполнять поставленные задачи.

3. Установленный уровень абстракции, который ограничивает спектр проектных решений, например, x86 (CISC), ARM (RISC).

Архитектура процессора создает экосистему и сильно влияет на нее: формирует инструментальную цепочку, требует длительной разработки современных компиляторов, создает зависимость всей экосистемы от выбранной архитектуры.

В отличие от архитектуры процессора, микроархитектура определяет технические способы реализации этого интерфейса. Микроархитектура процессора описывает внутреннюю организацию его аппаратных компонентов, включая состав функциональных блоков и принципы реализации выполнения инструкций.

Архитектура процессора - то, как видит компьютер программист. Практически всегда это система команд. В целом включает в себя еще и регистры и память (как место нахождение операндов) и вычислительные механизмы (типа кешей, прерывания)

Альтернатива и по сути антонимичное понятие: микроархитектура процессора - соединение простейших цифровых элементов в логические блоки, предназначенные для выполнения команд определенной архитектуры.

По сути для заданной архитектуры есть практически бесконечное количество возможных микроархитектур

67. Что такое система команд и какова её роль в архитектуре процессоров? Что определяет система команд?

Система команд по сути "словарный запас" программиста.

Формальное определение:

Система команд - абстрактная модель процессора, формирующая интерфейс взаимодействия между программным обеспечением и процессором. Формальный язык, описывающий правила вычислений и то, как они должны выполняться.

Включает в себя:

типы данных и значений, которые она способна обработать

набор инструкций

механизмы обработки прерываний и исключений

методы ввода и вывода (как часть информационного процессора)

При этом часто не включает в себя энергопотребление, производительность и задержки (для современного процессора практически невозможно понять, какое время будет выполняться та или иная команда)

68. Что такое машина фон Неймана? Какова её связь с машиной Тьюринга? Каковы её ключевые принципы?

В основе машины фон Неймана лежит Central Process Unit (CPU) --- универсальный процессор, способный решать любые задачи благодаря максимальной гибкости и программируемости. Структура машины фон Неймана:

- Control Unit --- блок управления, координирующий выполнение инструкций.
- Arithmetic/Logic Unit (ALU) --- арифметико-логическое устройство, обрабатывающее данные.

Машина фон Неймана повторяет машину Тьюринга, но имеется отличие --- наличие Memory Unit (RAM) вместо бесконечной ленты. Он даёт возможность адресовать ячейки памяти случайным образом, а также объединять данные и инструкции.

Главное достоинство архитектуры --- простота разработки и реализации (есть память и процессор). Благодаря этому машина фон Неймана быстро стала доминирующей и сохранила влияние до настоящего времени.

Особенности:

1. Использование двоичного кодирования.

Встречается еще троичное и двоично-десятичное кодирование

2. Программное управление.

Команды выполняются последовательно

3. Однородность памяти для данных и программ.

Инструкции и данные хранятся в едином адресном пространстве, что упрощает универсализацию. Однако на практике процессор разделяет их для повышения эффективности.

4. Ячейки памяти компьютера имеют адреса. Random-Access Memory.

Ячейки памяти доступны через прямую адресацию, исключая необходимость последовательного перебора.

5. Возможность условного перехода.

При использовании условного перехода алгоритм перестает быть строго линейным, позволяя изменять ход выполнения программы в зависимости от условий.

Принцип, на котором стоит эта машина - хранит инструкции в памяти, и есть дисциплина исполнения: читаем инструкцию, делаем ее, переходим к следующей. Короче линейность выполнения (с поправкой на переходы).

69. Что такое микропроцессор и микроконтроллер? В чём их отличия?

Микропроцессор (CPU) : это вычислительный модуль, отвечающий за выполнение инструкций, взаимодействие с оперативной памятью и управление потоком данных. Он не включает интерфейсы ввода-вывода.

Микроконтроллер (например, Arduino, STM32) : это микросхема, в которой память интегрирована непосредственно в процессор, что делает ее самодостаточной для решения задач без внешней памяти. Регистры при этом являются внутренними аппаратными элементами процессора и не заменяются внешними компонентами.

70. Что такое Control Unit и Data Path? Каковы их назначение и принципы взаимодействия?

Логика работы процессора разделена на два взаимосвязанных блока.

DataPath : это вычислительная часть процессора (АЛУ, регистры, шины). Выполняет чтение и запись в память и порты ввода-вывода, пересылку между регистрами, арифметические и логические операции. Представляет собой конструкцию, внутри которой данные перемещаются и преобразуются.

Control Unit : это управляющая часть процессора (CPU). Определяет последовательность выполнения операций в DataPath, управляет потоком команд, генерацией управляющих сигналов и синхронизацией работы всех компонентов. Является конечным автоматом.

Control Unit и Datapath тесно взаимосвязаны. Такая связь необходима в том числе для реализации условных переходов и других операций, зависящих от результатов вычислений. DataPath непосредственно выполняет операции над данными: включает регистры, АЛУ, мультиплексоры и другие логические блоки, формирует флаги состояния, такие как флаг переполнения, которые Control Unit использует для принятия решений. Сам Control Unit не анализирует данные напрямую, а опирается на информацию, предоставленную DataPath. То есть Control Unit --- это менеджер, полностью зависящий от информации, поступающей от исполнителя --- Datapath.

71. Какие виды инструкций существуют в машине фон Неймана? Приведите примеры. Каковы принципы кодирования?

Control Unit должен понимать, какие действия ему необходимо осуществлять. Для этого используются инструкции. Они определяют взаимодействие процессора с памятью, портами ввода-вывода и логику выполнения операций.

Инструкции разделяют по назначению:

Ориентированные на передачу информации между разными видами памяти.

Направленные на преобразование данных (арифметические, логические, битовые операции).

Определяющие поток управления программы (последовательное выполнение, условный переход, вызов и возврат из подпрограмм).

Направленные на взаимодействие с сопроцессорами.

Инструкция для RISC процессора представлена в виде 32-битного машинного слова. Возьмем в качестве примера инструкцию addi:

1. Код операции (opcode) --- определяет тип выполняемой операции (в нашем случае --- сложение регистра и константы).
2. Адресация регистров:
 - Addr 1 содержит значение, к которому прибавляется константа
 - Addr 2 используется для записи результата операции
3. Immediate Value --- константа, зашитая в саму инструкцию, которая участвует в операции сложения.

При возникновении переполнения, регистр, в котором происходит сложение, остается без изменения, устанавливается флаг Overflow. Однако в архитектуре RISC-V флаг Overflow отсутствует.

8 ЛЕКЦИЯ

72. Сравните принстонскую и гарвардскую архитектуры: особенности, достоинства, недостатки и области применения.

Принстонская – фон-Неймана. Особенности были в вопросе 66.

Основные недостатки классической архитектуры фон Неймана:

1. Узкое место – совместная память: доступ к инструкции и данным по очереди по одному каналу
2. Использование единой шины для инструкций и данных ограничивает производительность, снижая пропускную способность.
3. Ограничения стиля программирования.
 - Последовательное выполнение --- пошаговая обработка инструкций затрудняет эффективный параллелизм.
 - Общий доступ к памяти --- осложняет параллельные вычисления.
 - Единое адресное пространство позволяет интерпретировать данные как исполняемый код, что создаёт уязвимости безопасности.

Достоинства:

Простота проектирования

Универсальность

Эффективное использование памяти

Легкость программирования

Основная сфера — универсальные компьютеры, персональные компьютеры (PC), серверы и системы, где критична простота программирования.

Гарвардская архитектура. Она предполагает физическое разделение памяти на две части:

- Память инструкций --- подключена к Control Unit для интерпретации инструкций.
- Память данных --- подключена к DataPath для работы с данными.

Достоинства Гарвардской архитектуры:

- Два физических канала --- разделение каналов для инструкций и данных обеспечивает более эффективную работу, чем один общий канал.
- Одновременный доступ к памяти --- можно одновременно читать операнд для одной инструкции и загружать следующую инструкцию, не дожидаясь освобождения канала данных.
- Разная ширина машинного слова и адреса для данных и программ --- возможность использовать более широкий формат для инструкций, что позволяет эффективно кодировать код операции и константы
- Изоляция --- чёткое разделение данных и инструкций повышает безопасность и предотвращает случайное выполнение данных как инструкций.

Недостатки:

- Сложность и стоимость реализации --- раздельная организация памяти усложняет проектирование процессора и увеличивает его стоимость. Процессор с двумя шинами требует более сложной архитектуры чем тот, который использует одну.

Основные области применения — микроконтроллеры, цифровые сигнальные процессоры (DSP), встроенные системы и кэш-память первого уровня.

73. Как организовано адресное пространство в гарвардской и принстонской архитектурах?

Принстонская архитектура - единое адресное пространство, код и данные в одной памяти и используют общую шину.

Гарвардская - раздельные адресные пространства, память команд и память данных, для каждой отдельные шины.

Принстонская - универсальность, Гарвардская - быстродействие и надежность

74. Какие существуют варианты обхода ограничений гарвардской архитектуры, включая "Модифицированную гарвардскую архитектуру"?

В Принстонской архитектуре выполняются программы без необходимости переходных шагов. В таком процессоре нет проблемы с перемещением данных между разными типами памяти, что упрощает процесс работы.

Гарвардская архитектура, напротив, сталкивается с проблемой: когда данные и инструкции хранятся в разных типах памяти, возникает необходимость в

дополнительных шагах для передачи данных из одной памяти в другую. Это требует дополнительных решений, которые, в большинстве случаев, являются неоптимальными и усложняют систему. Существуют следующие подходы для решения этой проблемы:

- Память инструкций как данные --- инструкции могут быть интерпретированы как данные и загружены в память, что позволяет работать с ними как с обычными данными.
- Память данных как инструкции --- аналогичный подход, но данные воспринимаются как инструкции.

Однако у таких решений есть серьёзные недостатки. При объединении памяти для инструкций и данных утрачиваются ключевые преимущества Гарвардской архитектуры, в частности изоляция инструкций и данных и наличие двух независимых путей доступа к памяти.

Модифицированная Гарвардская архитектура: с точки зрения адресации, вся память воспринимается как единая, но используются разные способы получения данных. Это достигается благодаря системам кэширования, которые позволяют читать данные и инструкции из различных мест. Для самого процессора эта организация выглядит как Гарвардская архитектура, однако на уровне архитектуры процессора это не всегда явно проявляется.

75. Что такое "аккумуляторная система команд"? Каковы основные элементы? Приведите примеры. Каковы достоинства и недостатки?

Аккумуляторная архитектура: для хранения одного из операндов операции в процессоре имеется выделенный регистр --- аккумулятор. В этот же регистр заносится и результат операции. Изначально оба операнда хранятся в основной памяти, и до выполнения операции один из них нужно загрузить в аккумулятор. После выполнения команды обработки результат находится в аккумуляторе и, если он не является операндом для последующей команды, его требуется сохранить в ячейке памяти.

Основные компоненты такой архитектуры включают:

- Память --- для считывания и записи данных;
- АЛУ (арифметико-логическое устройство) --- для выполнения операций;
- Аккумулятор --- специальный регистр процессора, предназначенный для временного хранения промежуточных результатов вычислений.

Преимущества аккумуляторной архитектуры:

- Простота системы команд. Архитектура подразумевает минимальный набор инструкций: аккумулятор (неявный аргумент) и адрес памяти (явный аргумент).
- Компактность инструкций. Инструкции имеют фиксированный размер и малый объём.

- Простота аппаратной реализации.

Одним из ограничений аккумуляторной архитектуры является то, что в ней есть только один аккумулятор, а значит можно одновременно хранить лишь одно промежуточное значение. Это создаёт трудности при вычислении выражений, где требуется несколько операций с промежуточными результатами.

Примеры: БЭВМ, DSP

76. Что такое Register-to-Memory системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

Логичное развитие аккумуляторной архитектуре --- это Register-to-Memory. По своей сути, между ними нет принципиальной разницы, за исключением одного важного улучшения --- наличия большего количества регистров. И над ними можем выполнять операции (соответственно с этими регистрами).

Основные элементы: регистры общего назначения, АЛУ, память

Основные особенности:

- Увеличение числа регистров. Множество регистров позволяет снизить нагрузку на память. Нет необходимости постоянно считывать и записывать данные между памятью и аккумулятором. Данные обрабатываются непосредственно в регистрах.
- Расширение системы команд. Архитектура поддерживает более сложные команды, включающие два или три операнда. Один операнд указывает на регистр, с которым происходит операция. Два операнда позволяют разделить регистр-источник и регистр-назначение, а иногда третий --- дополнительный источник данных, повышая гибкость операций.

Недостаток – на каждом такте выполняется доступ к памяти, что вызывает задержку, так как память работает медленнее процессора и регистров.

Пример: мотолла (m68k), CISC

77. Что такое Register-to-Register системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

Процесс:

1. В первую очередь, все необходимые данные предварительно загружаются из оперативной памяти в регистры процессора. Это позволяет минимизировать количество обращений к относительно медленной памяти во время выполнения операций.
2. Далее, все арифметические и логические вычисления происходят исключительно между регистрами, что существенно ускоряет обработку данных за счёт высокой скорости работы с регистрами.
3. Завершающим этапом является сохранение полученных результатов обратно в память, если они необходимы за пределами текущих вычислений.

Основные элементы: регистры общего назначения, АЛУ, память

Преимущества Register-to-Register архитектуры:

- Высокая скорость выполнения операций. Операции на регистрах происходят значительно быстрее, так как регистры имеют мгновенный доступ.
- Минимизация времени выполнения инструкций. Инструкции, работающие только с регистрами, могут выполняться за очень малое количество тактов

Минус в том, что больше инструкций надо выполнить, чтобы сделать то же самое по сравнению с прошлой системой команд

Пример: RISC

78. Что такое Memory-to-Memory системы команд? Каковы основные элементы? Приведите примеры. Каковы их достоинства и недостатки?

В отличие от Register-to-Register подхода, где все вычисления происходят между регистрами, эта архитектура позволяет инструкциям напрямую взаимодействовать с ячейками оперативной памяти. Каждая операция здесь может включать сразу несколько этапов: считывание исходных данных из одной области памяти, выполнение вычислений внутри процессора и запись полученного результата обратно в другую область памяти. Таким образом, одна инструкция совмещает в себе сразу три действия: загрузку, обработку и сохранение --- что делает машинный код более компактным.

Преимущества Memory-to-Memory архитектуры:

- Удобство для низкоуровневого программирования. При разработке на ассемблере Memory-to-Memory архитектура позволяет писать код, который напрямую отражает алгоритмическую суть программы.
- Эффективность инструкций. Одна инструкция выполняет сразу несколько операций, что уменьшает накладные расходы на кодирование и интерпретацию инструкций.
- Программы на такой архитектуре легко читаются

Недостаток непосредственно в том, что для каждой инструкции аж три цикла обращения к памяти, которая обычно медленная, время доступа которой может отличаться на порядки в зависимости от например кэширования.

Пример: VAX, ПДП

79. Что такое CISC процессоры? Каковы их ключевые особенности, достоинства и недостатки?

Complex Instruction Set Computing (CISC) --- архитектура процессора с полным набором команд, в которой отдельные инструкции способны выполнять несколько операций (например, загрузку из памяти, арифметические вычисления и

сохранение обратно в память) или многошаговые процедуры в рамках одной инструкции. Типичный пример CISC-процессоров является x86

Основная идея CISC заключалась в стремлении обеспечить максимально эффективное управление процессором. Для этого систему команд усложнили, чтобы минимизировать количество лишних операций при выполнении задач.

Одним из ключевых факторов, способствовавших развитию архитектуры CISC, стало внедрение микропрограммного управления. Оно позволило реализовывать сложные команды и расширенные режимы адресации без существенных затрат на аппаратную часть.

В то время высокоуровневые языки ещё не были широко распространены, и основное программирование велось на ассемблере. Это делало наличие расширенного набора команд особенно актуальным: он упрощал разработку, повышал удобство. Расширенные инструкции позволяли выполнять сложные действия за одну команду, снижая количество строк кода, экономя память и увеличивая производительность, которая в тот период была ограниченным и дорогостоящим ресурсом.

Преимущества архитектуры CISC

- Удобство использования ассемблера для написания программ.
- Снижение затрат на доступ к памяти команд благодаря более коротким программам.

Недостатки архитектуры CISC

- Сложность системы команд: Большое количество команд делает их полное освоение и эффективное использование сложной задачей для программистов и компиляторов.
- Усложнение устройства процессора и блока управления: Наличие множества инструкций, способов адресации и аргументов приводит к значительному усложнению схем управления процессором.
- Повышенные требования к разработчикам: Разработка и сопровождение сложных систем команд требуют глубоких знаний архитектуры процессора.

80. Какие виды адресации аргументов инструкции в CISC процессорах?

Их может быть вообще великое множество чисто из философии CISC-like процессоров. Например, из того, что реализовано во *wrench* (архитектура m68k):

Прямая регистровая - значение из регистра

Непосредственная (Immediate) - значение напрямую

Косвенная - берем значение по адресу, который задан значением в регистре

Косвенная с пост-инкрементом/пре-декрементом – когда после инструкции адрес будет смещен

Косвенная со смещением - как косвенная, но с добавлением некоторой константы

Косвенная с индексом и смещением - как прошлая, но еще плюс добавить значение некоторого регистра

81. В каких случаях CISC инструкции эффективны? Приведите примеры.

CISC инструкции эффективны за счет того, что они достаточно узкоспециализированы (тот самый конфликт гибкости и эффективности), каждая из них делает очень конкретную задачу, но с очень узкой областью применения.

Пример:

`move.l 4(A0, D1), D0`

В m68k за счет применения CISC одной инструкцией делает: загрузить значение из адреса (адрес + 4 + D1) в регистр D0

В RISC это были бы отдельные операции: Загрузить значение A0 куда-то. Прибавить ему 4, сохранить. Прибавить ему D1, сохранить. Обратиться к памяти и загрузить значение по полученному адресу. Сохранить это значение в D0 (примерно)

82. В чём выражается комплексность систем команд CISC?

Комплексность в CISC процессорах - принцип построения системы команд, при котором:

1. одна инструкция выполняет сложную операцию, заменяя несколько других
2. Поддерживаются разные режимы адресации
3. Инструкции имеют переменную длину и формат

83. Какие существуют подходы к реализации Control Unit? Достоинства и недостатки Hardwired и микропрограммного управления?

При проектировании CISC-процессоров основное внимание уделяется устройству процессора и блоку управления (Control Unit), поскольку реализация конечного автомата, способного обрабатывать сложную систему команд, представляет собой чрезвычайно сложную задачу с точки зрения аппаратной реализации.

Существует два основных подхода к реализации Control Unit:

- Hardwired (жёстко заданный) --- управление реализуется через комбинационные схемы, декодирующие инструкции в последовательности управляющих сигналов. Блок управления использует фиксированный набор логических вентилей и схем, при этом каждая инструкция имеет заранее заданную жёсткую реализацию. Hardwired-блоки просты и быстры, однако они обладают малой гибкостью и трудны для модификации.
- Microcoded (микропрограммный) --- управление осуществляется посредством исполнения микропрограммы, управляющей выполнением инструкций. Внутри процессора размещается очень быстрая память, в

которой хранится микрокод, управляющий всеми процессами на уровне выполнения инструкций.

Преимущества микропрограммного управления:

- Гибкость. Возможность описания произвольной системы команд и модификации процессора без изменения схемотехники.
- Программируемость системы команд. Позволяет настраивать поведение процессора под конкретные задачи или оптимизировать под целевые приложения.

Недостатки микропрограммного управления:

- Необходимость хранения микрокода. Требуется быстрая и дорогая память внутри процессора
- Ограничения по производительности. Сложные алгоритмы декодирования могут затруднять точную оценку длительности выполнения инструкции.

84. Что такое микропрограммное управление? Что такое микроинструкция? В чём отличия инструкций и микроинструкций?

Микропрограммное управление - способ реализации управления процессором, при котором каждая инструкция выполняется как последовательность микроинструкций.

Микроинструкция - элементарная команда, управляющая работой компонентов процессора (открытие/закрытие защелок, чтение/запись и тд)

Отличия:

инструкция — это то, с чем работает программист

микроинструкция — это то, во что должна быть транслирована инструкция в момент исполнения.

Одна инструкция может требовать исполнения множества последовательных микроинструкций, но ни одна микроинструкция не может исполняться параллельно с другой.

85. Как можно оптимизировать инструкции при помощи микропрограммирования? Приведите пример.

Можно в теории исправить ошибки в реализации, закрыть уязвимости уже после производства. Можно создавать очень сложные инструкции даже на простой архитектуре, которые комбинационной схемой в лоб было бы куда сложнее реализовать. Привносит понятности в Control Unit.

86. Какие существуют подходы к поиску первой микроинструкции для заданной инструкции?

1. Алгоритмически: сопоставление инструкции с условием + условный переход в рамках микропрограммы

2. Отображение кода операции на адрес микроинструкции:

Прямое отображение - опкод (код операции) выступает в роли адреса. К примеру опкод 0101 - 0x5 адрес микроинструкции

Непрямое отображение - требует использования look-up-table для поиска нужного адреса (LUT - таблица, где каждому опкоду сопоставлен нужный адрес)

87. Что такое архитектура NISC? Каковы область её применения, достоинства и недостатки?

No instruction set computing (NISC) --- архитектура процессора без фиксированного набора инструкций, где управление выполняется напрямую микрокодом или конфигурационными битами, а не машинными командами

Если мы посмотрим на процессор и зададимся вопросом: «А зачем нам вообще нужны инструкции?», становится очевидным, что процессор в реальности управляется инструкциями. Тогда возникает идея: отказаться от этапа компиляции в машинные инструкции и вместо этого компилятору сразу выдавать множества микроинструкций, которые напрямую загоняются в процессор для управления.

В классическом CISC-процессоре мы видим:

- Program Counter
- Program Memory
- Micro Program Counter
- Micro Program Memory
- Управляющие слова

Если удалить этап с Micro Program Counter и Micro Program Memory, останется только Program Counter и Program Memory. Это позволяет значительно упростить процессор.

Преимущества такого подхода:

- Значительное упрощение аппаратной части.
- Отсутствие необходимости в проектировании и поддержке ISA, так как управляющие сигналы напрямую задаются железом.

Однако у такого подхода есть и очевидные недостатки:

1. Для исполнения кода на другом процессоре потребуются полное совпадение их внутреннего устройства. Отсутствие разделения между архитектурой процессора и его микроархитектурой приводит к тому, что компилятор, программист и процессор должны разрабатываться совместно и не могут развиваться независимо.
2. Управляющие слова содержат большое количество "пустых" нулей. Память внутри процессора оказывается перегруженной хранением лишних данных,

и эти нули приходится передавать по шинам, что увеличивает накладные расходы.

Применение:

Применяется в ускорителях, в высокоуровневом синтезе (HLS), спец вычислителях

Проект NITTA - CGRA процессор, где вычислительные блоки управляются в стиле NISC.

9 ЛЕКЦИЯ

88. Что такое стековые системы команд? Каковы основные элементы? Что такое неявная адресация аргументов? Каковы их достоинства и недостатки?

В её основе лежит принцип использования стековой памяти, размещённой непосредственно внутри процессора, вместо традиционного набора регистров. Стек представляет собой совокупность связанных между собой ячеек памяти, функционирующих по принципу «последним вошёл — первым вышел» (LIFO, Last In First Out). При этом операции обычно выполняются над двумя верхними элементами стека, что позволяет обходиться без явного указания аргументов — они неявно берутся с вершины стека.

Состав: стек, АЛУ, память

Преимущества стековой архитектуры:

- Аргументы функций передаются посредством стека без необходимости использования ограниченного числа регистров, что особенно актуально при глубокой вложенности вызовов.
- Стековая архитектура характеризуется простотой реализации и минималистичным машинным кодом, что упрощает проектирование процессоров.

Недостатки стековой архитектуры:

- В случае необходимости обращения к элементам стека, расположенным не на вершине, возникает необходимость выполнения дополнительных операций, что приводит к увеличению количества команд. Для частичного решения этой проблемы могут использоваться несколько стеков или дополнительные регистры.
- Ограничение работы только с вершиной стека затрудняет распараллеливание операций, что снижает производительность и ограничивает возможности ускорения выполнения программ.

89. Какие виды стеков есть в стековых процессорах? Их количества?

У нас может быть один и несколько стеков, что позволяет решить некоторые вопросы параллелизма (если например у нас есть два стека, то переключаясь

между ними мы нативно реализовываем поддержку thread'ов), или как в f32a удобнее реализовать поддержку процедур (подпрограмм)

Виды стеков:

стек в памяти, когда есть регистр который указывает на вершину стека и собственно память где хранятся значения

Набор связанных регистров

Либо еще как-то аппаратно-реализованная история, которая обычно позволяет за одни такт прочитать вершину стека, а также следующее после вершины стека значение

В f32a один стек возврата и один стек данных

90. Каковы основные принципы описания алгоритмов для стековых процессоров? Что такое Forth и High-level language computer?

Forth - императивный (описывающий, как что надо делать) язык программирования на основе стека. Очень хорошо компилируется под стековые архитектуры и крайне эффективен для них. Еще и процедуры поддерживает и с рекурсией и все это без жутких соглашений и сложностей на уровне процессора (все очень нативно). Особенности: возможно исследовать и изменять структуру программы во время выполнения, последовательное программирование

Высокоуровневым процессором (High-level language computer) – как раз называют процессоры на основе стека, потому что здесь кратно проще работать с функциями (нам не надо ломать голову с передачей аргументов) мы просто кладем на стек аргументы, вызываем функцию, получаем результат в стеке.

А внутри самой функции она получает произвольное кол-во аргументов (за счет стека), ей не надо париться с тем, чтобы сохранить куда-то состояние процессора (в других процессорах куда-то в память сохраняют состояния регистров, чтобы потом их вернуть), просто считает все что ей надо, кладет результат на стек

Достоинства – простая система команд, высокая производительность

Недостатки – эффективность страдает при большом количестве данных на стеке, сложно положить динамические данные на стек

91. Каковы источники роста производительности процессоров?

Рост частоты процессора - чаще тактовые импульсы -> чаще выполняем инструкции -> быстрее работаем. В два раза больше частота - в два раза больше скорость (с поправкой на скорость работы памяти)

Специализация аппаратуры - аппаратные вычислители, например, аппаратное умножение будет куда быстрее, чем если мы будем через сложение и сдвиги реализовывать умножение

Специализация системы команд — это вот история про CISC, где команды эффективнее за счет специализации

Параллелизм уровня бит, инструкций, задач:

На уровне бит - увеличиваем разрядность, и теперь мы можем большие объемы памяти обрабатывать за меньшее количество инструкций

Инструкций - конвейеры всякие там

Задач - когда у нас есть несколько ядер/потоков и мы параллелим именно на этом уровне, на уровне прикладной задачи

Адаптация структуры вычислителя под задачу – жертвуя гибкостью вычислителя, допиливаем его строго под нашу задачу, получая прирост производительности

Динамическая адаптация - специализация аппаратуры, "перепрошивка" вычислителя (его Control Unit) на лету

92. Какие законы и ограничения влияют на производительность современных процессоров?

см вопросы 92–96, просто кратко описать законы

93. Что представляют собой закон Мура и закон Деннарда?

Закон Мура - каждые ~2 года количество транзисторов в процессоре увеличивается вдвое, а производительность растет в 1.5 раз. Сегодня он уже не работает, так как простое наращивание транзисторов на чипе уже практически ничего не дает. Можно конечно сделать такие чипы, но проблема в том, что мы не сможем их загрузить. Самый простой способ загрузить большое количество ядер - выполнять задачи в параллель, и в итоге мы наталкиваемся на закон Амдала. Это эмпирический закон – наблюдение

Закон Деннарда - уменьшая размеры транзистора и повышая тактовую частоту процессора, возможно пропорционально повышать производительность. Можно уменьшать размер чипа и за счет этого получать большую производительность (во-первых, электрическим сигналам нужно меньше пространства, за счет этого можно поднимать частоту, во-вторых, уменьшение чипов позволяет уменьшить энергопотребление), но есть но!

Это дорого + физически невозможно. И чем меньше транзистор, тем больше утечки тока

94. Что представляет собой закон Амдала? На каком уровне параллелизма он работает?

Закон Амдала - если мы решаем задачу параллелизма, увеличивая его уровень в два раза, нам по сути надо площадь процессора увеличить в два раза (построить еще такой же вычислитель, которые будет вычислять вместе с первым), что очень накладно, а еще с каждым шагом увеличения параллелизма время, которое мы

выигрываем, будет уменьшаться. Ну а еще он говорит о том, что даже если мы распараллелим половину программы и возьмем бесконечность вычислителей, то все равно ускорение будет в два раза в пределе. Для 95% распараллеливания это будет всего 20 раз, это при теоретически бесконечном числе вычислителей, не впечатляет.

Походу работает на уровне параллелизма инструкций, потому что мы рассматриваем, насколько параллелятся наш поток инструкций, который решает одну прикладную задачу (поэтому это не уровень параллелизма задач, а именно инструкций)

95. Что такое Power Wall и тёмный кремний?

Power Wall: увеличивая частоту, мы квадратично увеличиваем потребление, а значит и тепловыделение, значит есть предел повышения частоты, после которого мы уже не сможем эффективно и в нужном объеме отводить тепло от чипа

Темный кремний: если мы в некотором процессоре будем повышать плотность транзисторов, то в какой-то момент мы не сможем больше отводить от него тепло, его будет выделяться слишком много. Таким образом, существует некоторое ограничение объема вычислений на единицу площади, даже если мы сможем уместить туда гораздо больше транзисторов.

96. Какие подходы существуют к решению проблемы Memory Wall в рамках процессоров семейства фон Неймана?

Memory Wall - проблема заключается в том, что есть разрыв между скоростью выполнения инструкций процессором и скоростью доступа к памяти. Раньше этого разрыва не было, но со временем процессора стали ускоряться сильно быстрее чем память, в итоге мы можем создать довольно быстрый вычислитель, но не сможем загрузить достаточное количество данных. В итоге получаем, что мы как бы можем процессоры сделать быстрее, но есть ли в этом смысл, когда мы не можем загрузить необходимые данные

Решать сложно, частично решается с помощью кэшей, количества каналов доступа к памяти (в Гарвардской архитектуре, например, их два, что несколько решает проблему)

97. Что такое RISC? Каковы особенности и предпосылки появления (ПО и аппаратура)?

RISC (Reduced Instruction Set Computer) - подход к проектированию процессоров, где быстродействие увеличивается за счет простого кодирования упрощенного набора инструкций

Основная идея RISC процессора заключается в том, что лучше сделать минимальный набор простых команд, ориентированных на быстрое исполнение, а сложные случаи реализовывать их последовательностями.

Предпосылки:

Появление языков высокого уровня, сложно компилировать код для CISC процессоров. Раньше люди писали на ассемблере и CISC для них был просто удобнее, но сейчас ассемблер генерируется, и проще генерировать под RISC

Простота декодера инструкции — это удобно. Единый формат инструкций — это удобно

Можно ускорить и оптимизировать каждую инструкцию

Появляется параллелизм уровня инструкций

Место микрокоманд и декодера можно пустить на более полезные задачи в духе кэша или регистров

98. Как сравниваются CISC и RISC с точки зрения архитектуры?

В CISC, по сравнению с RISC:

очень мощные, комплексные программы

есть операции memory-to-memory и всякие такие истории, тогда как в RISC строго Load/store

меньше регистров, видимо за счет общей сложности других частей процессора

Плавающая длина инструкций, у RISC единый формат и размер инструкций

99. Как сравниваются CISC и RISC с точки зрения микро-архитектуры?

В RISC, по сравнению с CISC:

Гораздо проще декодеры за счет простоты инструкций

Гораздо проще Control Unit, есть место для его оптимизации

Поддерживается конвейеризация

100. Каковы особенности кодирования инструкций при сравнении CISC и RISC?

В CISC разная длина инструкций, тогда как в RISC фиксированная. Формат инструкций в RISC более строгий и единый, не так много режимов адресации к примеру. В CISC формат может плавать, может быть огромное количество разных фич, зашитых уже сразу в инструкции (как вот в m68k адресация с индексами и некоторым смещением)

101. Каковы принципы конвейерного исполнения инструкций? Как это влияет на загрузку и производительность?

Мы получили все инструкции одинакового размера и возможность загружать 1 инструкцию каждый такт => можем исполнять инструкции не последовательно, а

можем шаги выполнения инструкции (fetch, decode, execute, write) вытянуть в конвейер

Принцип работы:

Выполнение каждой инструкции можно разбить на несколько этапов, которые задействуют непересекающиеся части процессора и выполняются за один такт

Обеспечиваем выполнение этапов независимо друг от друга

Как результат мы одновременно можем выполнять независимые друг от друга этапы разных инструкций. Т.е. пока декодируется первая инструкция, мы уже загружаем следующую инструкцию. Потом первая перейдет на выполнение, а вторая на декодирование, будет возможность взять третью инструкцию на загрузку.

По итогу каждый такт мы забираем новую инструкцию вместо того, чтобы ждать, пока прошлая завершится

Для конвейеризации конечно важно, чтобы одна инструкция не зависела от результата другой, иначе все ломается

В идеале производительность увеличивается пропорционально количеству стадий конвейера. В то время как последовательный процессор выполняет одну команду за 5 тактов, конвейерный — выдает по одной команде за такт.

Конвейер позволяет задействовать АЛУ, память каждый такт, предотвращая их простой.

102. Каковы типовые стадии RISC процессора?

1. Instruction Fetch. Процессор считывает инструкцию по адресу в памяти, значение которого присутствует в счетчике команд.
2. Instruction Decode. Команда декодируется и считываются регистры.
3. Instruction Execute. Выполняются операции. Операции можно разделить на:
 - Сложение, вычитание, сравнение и логические операции.
 - Получение данных из памяти и их загрузка в регистры.
 - Целочисленное умножение и деление и все операции с плавающей запятой.
4. Memory Access. Чтение/запись операндов из памяти/в память
5. Write Back. Полученное значение записывается обратно в регистр.

103. Какие проблемы и архитектурные ограничения связаны с конвейеризацией?

Инструкции должны быть независимыми друг от друга. Это достаточно сложно, потому что часто они используют, например, общие ресурсы процессора. Из этого вытекают:

Структурные конфликты (например, оба этапа хотят обратиться к памяти). По сути это проблема разбиения на этапы, потому что они получаются не независимыми друг от друга (например, чтение инструкции и чтение операндов используют память с одним каналом)

Конфликт по данным. Одна инструкция еще не успела посчитать, а другой уже нужен результат

Конфликт по управлению. Условные переходы порождают проблему того, что нам вообще не ясно, какую следующую инструкцию загружать в конвейер после этого перехода, надо ждать

104. Как разрешаются структурные конфликты в конвейере?

На уровне архитектуры:

Двухпортовая память (несколько каналов доступа к памяти, можно брать свободный)

Гарвардская архитектура (чтение инструкции и чтение операнда теперь не конфликтуют)

На уровне выполнения:

Разрешение конфликта пузырьком. Это холостая "инструкция", которая проходит по этапам, заставляя их простаивать. Это похоже на пор, но НЕ пор, потому что пор тоже надо читать и декодировать. Пузырек не надо, он ничего не использует из ресурсов, они просто простаивают, обрабатывая его. Результат - разносит во времени конфликтующие этапы

105. Как разрешаются конфликты по данным в конвейере?

Виды таких конфликтов:

Read after write (основной, самый простой конфликт): Когда за счет параллельного исполнения одна инструкция уже читает то, что другая инструкция еще не успела записать

Можно разрешить при помощи пузырька опять же.

Можно исполнять инструкции не по порядку (суть такая же, как у пузырька, в том плане, что нам надо разнести инструкции по времени, только вместо бесполезного пузырька мы пускаем полезную инструкцию), для этого надо как-то по-умному их перетасовать.

Другие, более специфичные конфликты:

Write after read - могут быть проблемы с кешами, в целом не очень уж и проблема

Write after Write - когда важна последовательность записи, например, в управляющие регистры

Read after Read - тут вообще проблем нет, не важно, в каком порядке куда читать

Иногда, при ложных конфликтах типа WAR и WAW можно переименовать регистры и это не отразится на логике программы. Тоже способ разрешить конфликт

106. Как происходит разрешение конфликтов при помощи пузыря в конвейере?

Разрешение конфликта пузырьком. Это холостая "инструкция", которая проходит по этапам, заставляя их простаивать. Это похоже на пор, но НЕ пор, потому что пор тоже надо читать и декодировать. Пузырек не надо, он ничего не использует из ресурсов, они просто простаивают, обрабатывая его. Результат - разносит во времени конфликтующие этапы

Может разрешить и структурные конфликты, и конфликты по данным, и конфликты по управлению

107. Как происходит разрешение конфликтов при помощи проброса операндов в конвейере?

Иногда бывает такое, что у нас есть конфликт по данным, когда одной инструкции нужен результат исполнения другой. Тогда мы можем не ждать пока инструкция запишет результат, а сразу взять его с выхода АЛУ, без надобности ждать, пока инструкция завершит работу.

А когда нам надо пробросить данные из будущего, вместо записи в память мы делаем петлю обратной связи, которая заставит процессор вместо прокидывания данных дальше, прокинуть их назад в выход аргумента предыдущей стадии.

Здесь можно применять пузырек, дабы дождаться хотя бы выполнения инструкции, от которой мы зависимы

10 ЛЕКЦИЯ

108. Какие существуют способы разрешения конфликтов по управлению во время компиляции?

Конфликт из-за операций условного/безусловного перехода. Проблема в том, что в конвейер в момент исполнения инструкции перехода уже будут загружены и декодированы части инструкций, которые возможно ввиду перехода исполнять не надо.

Решения во время компиляции (работаем с тем, что мы загрузим в процессор):

Минимизация кол-ва переходов:

- loop unrolling - сокращение числа переходов за счет разворачивания цикла (увеличивает кол-во инструкций, но уменьшает кол-во переходов и повышает производительность)
- условное перемещение данных — это когда мы используем специальные инструкции вроде cmovle (conditional move if less or equal), которые позволяют вообще обойтись без перехода

Оптимизация работы предиктором переходов (см. вопрос 109-110) за счет того, что на этапе компиляции мы добавляем в инструкцию информацию о том, какой исход наиболее вероятен

109. Какие существуют способы разрешения конфликтов по управлению во время исполнения?

Конфликт из-за операций условного/безусловного перехода. Проблема в том, что в конвейер в момент исполнения инструкции перехода уже будут загружены и исполнены части инструкций, которые возможно ввиду перехода исполнять не надо.

Решения во время исполнения (работаем с тем, как работает процессор):

bubble (пузырек) - как только мы понимаем, что у нас идет инструкция перехода (на этапе Instruction Decode), мы останавливаем загрузку инструкций в конвейер, загружая вместо них bubble (это холостая "инструкция", которая проходит по этапам, заставляя их простаивать. Это похоже на пор но НЕ пор, потому что пор тоже надо читать и декодировать. Пузырек не надо, он ничего не использует из ресурсов, этапы конвейера просто простаивают "обрабатывая" его), делаем так до тех пор, пока мы не поймем, как разрешится наш переход (пока инструкция перехода не будет обработана до конца).

Сброс конвейера - когда мы обнаружили инструкцию перехода, мы продолжаем загружать в конвейер инструкции, но у нас есть возможность их сбросить, т.е. на каком-то этапе прервать выполнение (превратив их в "пузырек"), восстановить состояние измененных регистров и сразу начинать загружать в конвейер новые инструкции. По факту выигрыш перед прошлым пунктом в том, что если вдруг переход не состоится, у нас уже есть частично обработанные инструкции

Предсказание переходов - Если мы видим, что у нас будет инструкция перехода, то мы пытаемся предсказать, произойдет переход или нет и начать выполнять инструкции согласно нашему предсказанию. Угадали = выиграли в производительности (подробно см. вопрос 109)

110. Какие существуют способы предотвращения конфликтов по управлению через branch prediction? Что такое статические предсказатели переходов?

Конфликт из-за операций условного/безусловного перехода. Проблема в том, что в конвейер в момент исполнения инструкции перехода уже будут загружены и исполнены части инструкций, которые возможно ввиду перехода исполнять не надо.

Предсказание переходов (branch prediction) - если мы будем хорошо угадывать результат операций перехода (с вер. >50%), мы сможем частично решить проблему конфликта по управлению, ведь сможем заранее загружать нужные инструкции и выигрывать в производительности

Предсказание может быть статическим и динамическим.

Статические предсказатели переходов содержат в себе некоторые правила, статистически на дистанции помогающие им делать хорошие предсказания. Например, если у нас инструкция безусловного перехода, мы 100% знаем, что надо подгружать инструкции с того места, куда она переходит.

Другой пример, статистические исследования показали, что если у нас идет условный переход назад (условие с `jump` на адрес меньше текущего), то с вероятностью 90% мы все-таки прыгнем назад. Это вызвано исключительно спецификой кода, который пишут люди и связано с наличием большого количества циклов, устроенных таким образом. Тем не менее помогает предсказателю угадывать в 90% случаев в среднем.

`jump` вперед же выполняется 50/50, тут статистически ничего не выиграть, только угадать пальцем в небо.

111. Что представляют собой динамические предсказатели переходов в конвейере?

Конфликт из-за операций условного/безусловного перехода. Проблема в том, что в конвейер в момент исполнения инструкции перехода уже будут загружены и исполнены части инструкций, которые возможно ввиду перехода исполнять не надо.

Предсказание переходов (branch prediction) - если мы будем хорошо угадывать результат операций перехода (с вер. >50%), мы сможем частично решить проблему конфликта по управлению, ведь сможем заранее загружать нужные инструкции и выигрывать в производительности

Предсказание может быть статическим и динамическим.

Динамические предсказатели переходов - когда мы стараемся угадать результат перехода непосредственно во время выполнения. Для этого у предсказателя есть таблица, в которой хранятся адреса инструкций перехода, для каждого адреса есть соответствие-предсказание, куда мы можем перейти после этой инструкции.

Кроме этого, в этой таблице есть счетчик с накоплением из двух бит, который увеличивается или уменьшается каждый раз, когда переход случается или не случается при исполнении инструкции условного перехода (переход случился — значит уверенность в том, что в следующий раз переход случится увеличилась и наоборот). Таким образом, на основе того, произошел ли переход на этой инструкции в прошлый раз процессор предсказывает, произойдет ли переход в следующий раз.

Есть еще многоуровневые динамические предсказатели переходов, которые умеют выявлять повторяющиеся паттерны

112. Как влияют различные виды конфликтов на производительность конвейера?

По сути они короче все снижают производительность/пропускную способность. Снизу нейронка достаточно хорошо описала, почему именно каждый конфликт приводит к этому:

1. **Структурные конфликты:** Снижают **пропускную способность**. Если два этапа не могут поделить ресурс (например, одну шину памяти), один из них замирает. Это буквально «физическое» ограничение железа, которое не дает конвейеру работать на полной проектной скорости.
2. **Конфликты по данным:** Увеличивают **время ожидания (latency)** конкретной инструкции. Чем длиннее конвейер, тем больше тактов он будет ждать «свежий» результат от предыдущей команды. Без механизмов обхода (forwarding) такие конфликты могут замедлить выполнение программы в разы, превращая параллельную обработку в последовательную.
3. **Конфликты по управлению:** Самые «дорогие» потери. При неверном предсказании перехода приходится **сбрасывать (flush)** все инструкции, которые уже успели попасть в конвейер «по ошибке». Чем глубже конвейер, тем больше работы выбрасывается в мусор и тем больше тактов тратится на его повторное заполнение.

113. Как связаны конвейерное исполнение и спекулятивное исполнение инструкций?

Конвейеризация разделяет выполнение инструкции на несколько этапов. Пока одна инструкция находится на этапе выполнения, следующая уже может декодироваться, а третья — выбираться из памяти. Это позволяет обрабатывать несколько инструкций одновременно, увеличивая пропускную способность.

Проблема: если встречается условный переход (branch), процессор не знает, какая инструкция будет следующей, пока условие не вычислится. Это приводит к простоям конвейера (pipeline stalls).

Чтобы избежать простоев, процессор предсказывает результат условного перехода (branch prediction) и начинает выполнять инструкции спекулятивно (то есть в предположении, что предсказание верно).

Если предсказание верное, спекулятивно выполненные инструкции сразу используются, и конвейер работает без задержек.

Если предсказание неверное, результаты отбрасываются (сброс конвейера — pipeline flush), и выполнение продолжается с правильной ветки.

Связь: Конвейер создает проблему, а спекулятивное исполнение решает ее (предсказание ветвлений + выполнение "вперёд" → минимизация простоев).

Таким образом, спекулятивное выполнение улучшает эффективность конвейера, позволяя ему работать с высокой загрузкой даже при наличии условных переходов.

OVERALL по конвейерам:

Преимущества: повышение производительности и уровня утилизации вычислительных ресурсов.

Потенциальные недостатки конвейера:

- снижение скорости исполнения отдельной команды (на отдельную инструкцию появляются дополнительные расходы, чтобы сохранить в промежуточный регистр, передать на след. стадию);
- не все операции могут пройти стадию конвейера за один машинный цикл (деления/умножения);
- необходимость разрешения конфликтов;
- непредсказуемое время исполнения (один код может работать разное время)

114. Каковы шаги типового взаимодействия с устройством ввода-вывода? Каков интерфейс устройства для процессора?

Типовое взаимодействие:

Конфигурация внешнего устройства (объясняем внешнему устройству, что мы будем с ним делать)

Проверка состояния (готово ли оно?)

Отправка данных

Проверка статуса ответа (можно ли читать ответ?)

Чтение ответа

Интерфейс ввода-вывода для процессора почти всегда выглядит как набор ячеек памяти или регистров, с которыми идет взаимодействие. Они подразделяются на:

Регистры данных

Регистры статуса

Над всем этим нам нужен протокол взаимодействия, иначе ничего, конечно, не будет работать.

115. Как реализуется ввод-вывод с точки зрения системы команд в Port Mapped IO?

Port Mapped IO (PMIO) - способ организации ввода-вывода, когда для регистров внешнего устройства у нас есть отдельное адресное пространство и, как правило, даже отдельные инструкции процессора, позволяющие с этими регистрами взаимодействовать. Напр. in 15, out 20

Прямо отвечая на поставленный вопрос: с точки зрения системы команд это реализуется отдельными инструкциями, с отдельной адресацией и вообще полностью изолированно от основной памяти

Достоинства:

- На уровне ассемблера чётко видно, где реализуется доступ к памяти, а где к внешним устройствам (процессор не попытается случайно прочитать инструкцию из порта ввода).
- Адресное пространство однородно и не имеет "запретных мест" (мы всегда знаем, что это ячейка памяти и всё).

Недостатки:

- Усложнение системы команд и процессора, реализующего его.
- Данные ввода-вывода становятся данными второго сорта.

116. Как реализуется ввод-вывод с точки зрения системы команд в Memory Mapped IO?

Memory Mapped IO (MMIO) - способ организации ввода-вывода, когда наше устройство "подключается" по некоторым магическим адресам в RAM, в связи с чем читая или записывая по этим адресам на самом деле мы работаем не с памятью, а с внешним устройством. Для процессора выглядит как работа с памятью

Адресное пространство используется как для доступа к основной памяти, так и для доступа к устройствам ввода-вывода. Это позволяет предоставить доступ к внешним устройствам без доработки процессора с точки зрения системы команд и внутренней организации.

В итоге получаем, что часть адресного пространства резервируется под внешние устройства и обращение к этому устройству выглядит как обычное обращение к памяти.

Прямо отвечая на поставленный вопрос: с точки зрения системы команд это реализуется теми же инструкциями, что и для работы с памятью, никаких дополнительных инструкций не вводится

Ключевые достоинства данного подхода:

- Простота процессора, без изменения микроархитектуры
- Вы можете работать с данными устройств ввода-вывода как с обычными данными, в том числе и операции с автоинкрементом

К недостаткам можно отнести:

- использование единой шины для ввода-вывода и доступа к памяти (нужно выравнивать скорости);
- неоднородность памяти, сложная конфигурация системы (включая инструментарий);

117. Как устроен ввод-вывод в Memory Mapped IO с точки зрения устройства процессора?

Устройства ввода-вывода отображаются в адресное пространство оперативной памяти, и взаимодействие с ними происходит через обычные команды работы с памятью

Устройствам ввода-вывода выделяются фиксированные адреса в общем адресном пространстве и контроллер памяти определяет, куда направлять запрос.

Процессор обращается к памяти, но если это адрес IO, то Address Code Decoder выключает (с помощью cs - chip select) RAM и подключает нужный интерфейс, который выставит на шину нужное значение, а процессор его защелкнет как значение из памяти

11 ЛЕКЦИЯ

118. Как работает программно-управляемый ввод-вывод без прерываний? Каковы его ограничения и типовой сценарий? Как связан с теоремой Котельникова?

Процессор активно опрашивает состояние устройства, проверяя его готовность к обмену данными, вместо использования прерываний, и если надо, то реагируем. Это подход «polling»

Проблемы в общем следующие:

Высокое энергопотребление (процессор постоянно занят периодической проверкой)

тяжело совмещать с другими задачами

тяжело обрабатывать сложный протокол ввода-вывода (типа SPI, например)

! НО, он выигрывает у подхода с прерываниями, когда у нас очень плотный поток обрабатываемых событий. Выигрыш за счет снижения накладных расходов на переключение контекста при прерывании !

Теорема Котельникова — аналоговый сигнал можно полностью восстановить его дискретным представлением, если дискретизировать его с частотой, в два раза БОльшей максимальной частоты его спектра. В итоге опрос устройства о его готовности — это условно дискретизация его состояния. И чтобы не пропустить событие, частота опроса должна быть выше максимальной частоты событий (иначе данные будут потеряны), но высокая частота приводит к перегрузке ЦПУ. Но, говоря про прямоугольный сигнал (когда он идеально прямоуголен), то у него спектр неограничен, то есть теорема Котельникова неприменима, типа нам бы пришлось в бесконечность уходить.

119. Что такое дребезг контактов? В чём разница между уровнем и сигналом применительно к кнопке? Как с дребезгом бороться программно?

Дребезг контактов — явление, при котором кнопки, переключатели, реле при замыкании или размыкании создают серию ложных, неконтролируемых переключений, длящихся несколько миллисекунд

Разница:

Уровень (статика): это текущее состояние контакта в конкретный момент времени — либо «высокий» (лог. 1, питание), либо «низкий» (лог. 0, земля).

Сигнал (динамика): это изменение уровня во времени.

- *Идеальный сигнал*: при нажатии уровень мгновенно переходит из 1 в 0 и остается там.
- *Реальный сигнал (с дребезгом)*: В момент переключения уровень хаотично скачет между 1 и 0, создавая временную пачку импульсов, прежде чем стабилизироваться

Программно можно после зафиксированного изменения состояния подождать, например сделать искусственную задержку. Можно сделать на счетчиках, доверяя уровню только если несколько раз подряд он был считан одинаково. Можно на таймере, не позволяя изменяться состоянию кнопки быстрее заданного порога.

120. Как реализовать псевдо-параллельный программно-управляемый ввод-вывод в одном потоке исполнения (state machine)?

Параллельный программно-управляемый ввод-вывод — это процесс ввода-вывода без прерываний, когда обработчику одного устройства не отдается весь поток управления (не занимает собой все время процессора), он делит этот поток с другими обработчиками.

Этот параллелизм можно сделать с помощью конечных автоматов. На примере сброса кнопок: у кнопки может быть состояние "нажата", и "не нажата". Чтобы избежать дребезга контактов (потому что реальная кнопка не переключается четко из 0 в 1 и из 1 в 0, она быстро-быстро колеблется при переключении), переходы между этими состояниями могут осуществляться только после задержки.

И вместо того чтобы занимать весь процессор тем, чтобы обрабатывать одну кнопку, мы делаем неблокирующую функцию, которая проверяет и актуализирует состояние кнопки (актуализирует состояние конечного автомата). Дальше в цикл мы можем понаставить кучу таких обработчиков разных кнопок и все они будут обрабатываться "одновременно".

Хоть и на самом деле при такой организации процессор в один момент времени работает только с одной кнопкой, за счет того что процессор очень быстро переключается между ними пользователю кажется что все кнопки обрабатываются одновременно.

121. Какие существуют уровни параллелизма: бит, инструкций и задач? Приведите примеры.

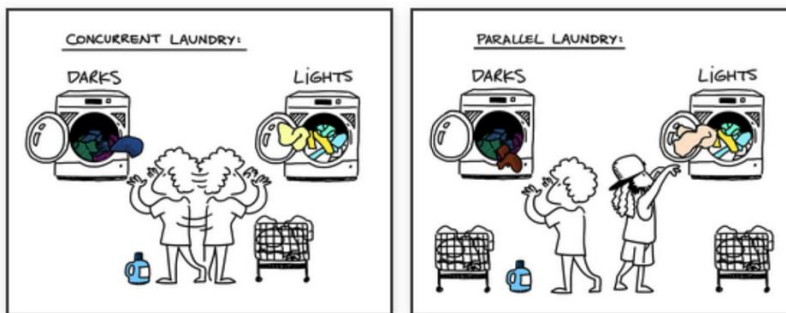
Виды параллелизма:

- **Уровень битов.** Мы обрабатываем не по одному биту, а сразу несколько. Например, складываем 8-битные числа, проводим битовые операции сразу

для 32-битного слова, пересылаем 32-битные регистры в память. Все это делается не для одного бита, а сразу для всех, получается параллельно

- Уровень команд. Когда мы можем выполнять сразу несколько инструкций параллельно. Например, с помощью конвейера, суперскалярности (это архитектура, в которой процессор имеет несколько исполнительных блоков (например, два АЛУ, два блока работы с памятью) и может запускать несколько инструкций одновременно за один такт).
- Уровень задач. Когда мы выполняем несколько (в общем случае независимых) алгоритмов одновременно, параллельно

122. В чём разница между параллелизмом и конкурентностью (concurrency)? Приведите примеры.



Слева конкурентность (concurrency), человечек (исполнитель) один, но он по очереди выполняет две разные задачи (закидывает вещи в машинку), что при быстрой работе человечка можно считать параллельным наполнением обеих машинок.

Справа "реальный параллелизм", когда у нас два человечка (исполнителя), которые реально параллельно наполняют две машины (выполняют две задачи параллельно)

То есть: конкурентность - 1 процессор, который переключается между задачами.
параллельность - несколько процессоров, которые выполняют разные задачи

123. В чём заключается проблема реализации параллелизма уровня задач в архитектуре фон Неймана?

Основная проблема - архитектура фон Неймана не предназначена для такого параллелизма (она рассчитана на последовательное выполнение инструкций и в едином потоке исполнения). У нас один поток инструкций, он не может "приостанавливаться", он работает пока не встретит halt, счетчик команд всегда идет вперед

Поэтому приходится делать всякие костыли, а объединение двух вычислителей архитектуры фон Неймана сразу порождает огромное количество проблем с синхронизацией, объединением памяти, тактовыми сигналами, конфликтами и прочим

Кооперативная многозадачность соответствует фон Нейману

124. Сравните кооперативную и вытесняющую многозадачность: достоинства, недостатки и область применения каждой.

Кооперативная многозадачность - когда у нас несколько процессов, но они все "дружат" между собой (учитывают тот факт, что они не единственные процессы) и готовы к кооперации, никто не будет делать нехорошие вещи и занимать поток надолго или блокировать его. Многозадачность, при которой следующая задача выполняется, когда текущая задача явно объявит о готовности отдать процессорное время

Достоинства:

Атомарность задач (никто не прервет задачу, у нас сразу получается атомарность)

Контроль за ресурсами со стороны задачи - задача сама решает, сколько ей надо взять ресурсов

Легко реализовать (программная реализация достаточно проста и эффективна, как в примере с кнопками)

Недостатки:

Контроль за ресурсами со стороны задачи - если задача случайно или специально зависнет, другие не получают выполнения

Сбой одной задачи может стать глобальным сбоем

Применяется кооперативная многозадачность в основном во встроенных системах, где зачастую нет проблемы доверия к выполняемым задачам в плане расходуемых ресурсов, где мы уверены, что задачи будут кооперироваться и не жадничать. Ну и ограниченность ресурсов таких систем делает кооперативную многозадачность (которая проще в реализации) более выгодным вариантом.

А еще применяется как основа системы асинхронности языков программирования (например, `asyncio` в Python, где можно передать управление с помощью `await`)

125. Какие механизмы необходимы для реализации кооперативной многозадачности?

Нужно реализовать механизмы:

остановки: задача добровольно передает управление диспетчеру

сохранения состояния задачи: все состояние всего ее окружения (регистры, стек, состояния сопроцессоров, память, ввод-вывод, кеш, состояние предсказателя переходов, и т.п.), а затем как-то уметь восстановить

планирования: диспетчеризация порядка выполнения задач (кто будет выполняться следующим?)

возобновление: восстановить состояние, на котором задача прервалась, передать ей управление

Далее есть опциональные механизмы:

изоляция задач: независимое выполнение, безопасность

взаимодействие между задачами: передача данных и сигналов, общие ресурсы (целостность данных)

126. Как взаимодействуют кооперативная многозадачность и система прерываний?

Взаимодействие строится примерно так: прерывание устанавливает флаги или добавляет события в очередь, потом основная обработка происходит, когда задача добровольно получает управление. Сами обработчики прерываний сохраняют данные, устанавливают флаги событий и пробуждают ожидающие задачи через механизм событий.

127. Как реализуется кооперативная многозадачность на уровне приложения через Event-loop?

По сути Event Loop - один из главных механизмов Node.js и один из способов реализации кооперативной многозадачности.

По центру у нас есть цикл событий, который проверяет нет ли новых запросов. Как только в очереди появляется запрос, он начинает его выполнять и выполняет, пока не нужно будет использовать что-то внешнее (диск, базу данных, еще что-то). Когда мы натыкаемся на это, мы создаем заявку в операционную систему (делаем системный вызов, который ОС выполнит впоследствии), в очереди регистрируем колбэк и переключаемся на другую задачу.

Когда ОС (или какая-то внешняя система) даст нам ответ, event-loop это увидит при ближайшей проверке и сможет продолжить выполнение с того места, с которого надо

128. Каковы достоинства и недостатки вытесняющей многозадачности?

Вытесняющая многозадачность - когда у нас несколько процессов, которые "думают" что они самые главные, а значит могут занимать весь поток управления, блокировать его и так далее. Для обеспечения многозадачности таких процессов нужен внешний механизм, который будет распределять ресурсы между этими задачами, приостанавливать одни и запускать другие.

Переключение процессов может происходить буквально между любыми двумя инструкциями

Распределение процессорного времени осуществляется планировщиком

Достоинства:

Простота разработки ПО. Нам не надо думать, когда отдавать управление диспетчеру, ОС сделает все сама

Ресурсами рулит операционная система. Мы изолированы от других задач и защищены от недобросовестного использования ресурсов другими задачами

Недостатки:

"Лишние" переключения. Мы всегда распределяем время между задачами, это накладные расходы на переключение. Хотя иногда нам без разницы и было бы эффективнее выполнить сначала одну задачу целиком, затем другую.

Непредсказуемое переключение - нас могут прерывать в любой момент, повлиять мы на это не можем. Проблемы синхронизации и гонок из-за этого (нарушается атомарность операций)

Непредсказуемая длительность работы из-за непредсказуемых прерываний (их обработка будет занимать какое-то, в общем случае непредсказуемое время)

Применяется в основном как основа операционных систем

129. Какие механизмы необходимы для реализации вытесняющей многозадачности?

Нужно реализовать механизмы:

прерывания: способ забрать поток управления у задачи, независимо от желания задачи (например, по сигналу от таймера или внешнего устройства)

сохранения состояния задачи: все состояние всего ее окружения (регистры, стек, состояния сопроцессоров, память, ввод-вывод, кеш, состояние предсказателя переходов, и т.п.), а затем как-то уметь восстановить

планирования: диспетчеризация порядка выполнения задач (кто будет выполняться следующим?)

возобновление: восстановить состояние, на котором задача прервалась, передать ей управление

Далее есть опциональные механизмы:

изоляция задач: чтобы процессы не портили друг друга, независимое выполнение, безопасность (с точки зрения выделяемой памяти - каждый процесс может захватить столько ресурсов, сколько ему надо)

взаимодействие между задачами: очень часто стоит задача взаимодействия между процессами, они связаны, передача данных и сигналов, общие ресурсы (целостность данных)

130. Что даёт переход от программно-управляемого ввода-вывода (polling) к вводу-выводу через систему прерываний с точки зрения эффективности и параллелизма уровня задач?

Преимущество, что нам не надо постоянно опрашивать состояние передачи и ждать готовности данных. Мы просто занимаемся своими делами, а когда данные будут готовы, обрабатываем их в обработке.

Также и при передаче: загружаем данные, дальше передача идет "сама", прерывая основной поток иногда на очень короткое время, чтобы выставить новые уровни на шине.

Так мы можем максимально быстро обрабатывать приходящие данные (не надо ждать следующую итерацию polling'a) и не занимать основной поток ожиданием, этим самым polling'ом. За счет прерываний можно очень четко выдерживать тайминги передач (чтобы следовать условиям протоколов и внешних устройств)

131. Как по шагам происходит обработка прерывания?

У нас есть основной поток, где у нас выполняются инструкции. У нас есть система прерываний, которая позволяет процессору регистрировать какое-то событие (например, сработал аппаратный таймер).

Когда это происходит и событие зарегистрировано, мы сохраняем текущее значение Program Counter (адрес возврата) и переходим по вектору прерывания (это адрес процедуры-обработчика прерывания, зачастую хранящийся где-то в отдельном месте в памяти), начинаем выполнять эту процедуру.

После ее завершения, если не произошло еще одного прерывания (да, во время обработки одного прерывания может прилететь еще одно), мы возвращаемся туда, где прерывались. Прерывание обработано.

1. Выполнение "основного" потока.
2. Запрос прерывания (HW).
3. Сохранение адреса возврата.
4. Вызов обработчика прерывания (ISR) — обычного кода в специальном месте.
5. Завершение ISR.
6. Восстановление адреса возврата.
7. Продолжение основного потока.

132. Почему появление системы прерываний приводит к автоматическому появлению параллелизма уровня задач? Охарактеризуйте его.

Системы прерываний без многозадачности существовать не может.

Как только появляется система прерываний, сразу же появляется разделение - основная программа и программа обработчик прерываний. И мы переключаемся между этими "задачами", обеспечивая многозадачность.

В целом это можно так не рассматривать, но в подавляющем большинстве случаев основная программа и обработчик прерываний – две абсолютно разные задачи, несинхронизированные между собой, еще и выполняющиеся параллельно независимо друг от друга. А значит многозадачность

В итоге такой параллелизм характеризуется:

это аппаратно-программный уровень

порядок прерываний зависит от внешних событий.

ресурсы общие: регистры, память.

133. Как классифицируются прерывания по источнику (синхронные/асинхронные, аппаратные/программные)? Их отличительные особенности.

Глобально есть два источника прерываний:

Аппаратные (вызывается железом):

- Внешние или асинхронные (асинхронен относительно инструкций процессора, параллельность). Переполнение таймера, нажатие клавиши, приняли пакет по сети
- Внутренние или синхронные (синхронные относительно инструкций процессора, очередь). Деление на ноль, ошибка доступа к памяти, какая-то критическая ошибка

Программные (вызывается инструкцией):

Взаимодействие программы и операционной системы. Например, можно использовать для кооперативной многозадачности, вызывая диспетчер через программное прерывание

Аппаратные прерывания можно использовать для системы ввода-вывода, реализованной с прерываниями.

134. Каковы задачи контроллера прерываний? Что такое маскируемые и немаскируемые прерывания? Относительный и абсолютный приоритеты?

Контроллер прерываний обычно реализовывается аппаратно (можно достичь очень высокой скорости обработки прерываний). Его задача - фиксация события, триггерящего прерывание, переключение контекста на его обработчика и возврат по завершению обработки.

Прерывания можно разделить на:

Маскируемые. Есть возможность запретить прерывание, установив, например, соответствующий бит в регистре разрешения прерываний

Немаскируемые. Соответственно те прерывания, которые нельзя запретить. Обычно они ставятся на какие-то глобальные сбои аппаратуры, например сбой памяти.

Относительные приоритеты --- при одновременном поступлении нескольких прерываний выбирается запрос, имеющий высший приоритет, но после начала его

обработки даже при поступлении прерывания с более высоким приоритетом его обработка не прерывается.

Абсолютные приоритеты --- при каждом поступлении прерывания происходит сравнение с уже исполняющимся (при его наличии) и если приоритет поступившего выше текущего, процессор переходит к обработке нового запроса, а по завершению обработки вернется к предыдущей процедуре.

135. Какие виды событий существуют в системе прерываний (по фронту, по уровню, по сообщению, doorbell)? В чём особенности каждого?

Виды событий:

- По фронту. Прерывание генерируется в момент изменения уровня сигнала на линии (из 0 в 1 или из 1 в 0)
- По уровню. Прерывание активно до тех пор, пока сигнал на линии активен. Требуется сброс: источник прерывания или обработчик прерывания должен сбросить сигнал после обработки.

Преимущество с предыдущим такое: если приходит несколько прерываний, то по фронту, надо будет сохранять этот сигнал в какой-нибудь буффер, иначе будет утерян. По уровню надо будет просто притупить сигнал, чтобы показать, что прерывание обработано

- По сообщению. У нас происходит прерывание и его может вызвать несколько источников. А информация о том, какой именно источник вызвал это прерывание находится в определённом заранее известном месте.
- По дверному звонку (Doorbell). Одно устройство инициирует событие у другого устройства записью в его регистр. Запись в регистр является сигналом к действию, но сами данные для обработки находятся в другом месте памяти, инициирующее устройство только уведомляет об их наличии. Используется для уведомления устройств или других процессоров о наличии готовых к обработке задач.

136. Что такое сторожевой таймер и каков принцип его работы?

Сторожевой таймер – аппаратно-реализованная схема контроля от зависания системы.

Как это устроено:

Есть система, которая имеет возможность перезагрузить весь процессор (Reset) и имеет возможность получать сигналы от процессора. Система (Watchdog Chip) внутри имеет таймер, который сбрасывается по сигналу от процессора. Если таймер превышает некоторый порог, то система считает, что процессор завис и перезагружает процессор нафиг.

В результате: пока все работает штатно, все классно. Таймер периодически сбрасывается, сторожевой таймер не трогает процессор. Как только процессор

завис и перестал сбрасывать таймер - сторожевой таймер жестко сбрасывает нас аппаратно, так куда больше шансов поддержать работоспособность системы в целом.

НО! это не является системой прерываний в классическом понимании, так как мы не передаем управление другой задаче, но это является системой прерываний, так как поток выполнения нарушается, если что-то пошло не так.

12 ЛЕКЦИЯ

137. Какие подходы существуют к решению проблемы изоляции регистров и инструкций в памяти при многозадачности?

Параллелизм требует:

Обеспечить взаимодействие между задачами

Совмещение задач по используемым ресурсам: по процессорному времени, по памяти

Изоляцию между задачами

Нам надо совместить/изолировать:

Регистры

Адреса инструкций(переходы)

Адреса данных(переменных)

Динамическую память(куча)

Автоматическую память(стек)

Когда надо совмещать/изолировать:

Compile-time(состав задач фиксирован)

Run-time(состав задач динамический)

Сами подходы:

1. Инструкции

Размещаем инструкции в разных областях памяти

Корректируем все абсолютные адреса переходов

2. Данные:

Размещаем данные в разных областях памяти

Корректируем все абсолютные адреса

3. Регистры:

Формируем соглашение о вызове процедур внутри задач

Сохраняем и загружаем регистры между задачами

4. Автоматическая память:

Оцениваем потребности в памяти

Выделяем подходящие диапазоны (фрагментация)

5. Динамическая память: выделяем диапазон

Статика - она же статическая память секция .data

Куча - она же динамическая память. Можно выделить там себе место, можно освободить. Находится под управлением выполняемой задачи

Стек - он же автоматическая память. Выделяется и освобождается автоматически в зависимости от того, насколько глубоко мы ушли во вложенные процедуры

138. Какие подходы существуют к решению проблемы изоляции данных в памяти (статика, куча, стек) при многозадачности?

Стоит задача изоляции, чтобы одна задача не имела доступа к памяти другой задачи, иначе мы получаем уязвимости безопасности и ситуации, когда одна задача по глупости программиста крашит другую / крашит систему.

Есть три основных подхода к решению проблемы изоляции памяти, все они работают в runtime:

Банки памяти. Сегодня уже мертвая технология. Классически это вообще не средство изоляции. Идея: давайте подключим к процессору несколько микросхем памяти. Например повесим их на одну шину и будем переключаться между ними, выбирая с кем именно мы сейчас работаем. Это можно использовать для аппаратной изоляции задач: задача физически не сможет прочитать что-то из другой микросхемы, ведь процессор позволяет ей читать только из своей.

Сегментация (сегментная память). Сегодня уже умирающая технология. Описана в вопросе 140.

Виртуальная память. Самая применяемая на данный момент. Описана в вопросе 141.

139. Как банки памяти используются для расширения адресного пространства?

Банки памяти — это буквально блок ячеек памяти, к которому процессор обращается как к единому целому. Это нужно для увеличения объема доступной памяти без усложнения адресации. Или другими словами: есть несколько чипов памяти к которым мы можем ходить связано или независимо друг от друга.

Работает он так (на примере процессора, которому надо больше памяти, чем он может адресовать): память разбивается на банки, в каждый момент времени активен только 1 банк, но его можно переключить. Процесс обращается к одному и

тому же адресу, но в зависимости от выбранного банка адрес указывает на разные физические ячейки.

Идея банков памяти: давайте подключим к процессору несколько микросхем памяти. Тогда мы сможем, например, перевести процессор в определенный режим, режим записи в определенную микросхему памяти на шине и даже за счет этого получить возможность рулить бОльшим количеством памяти, чем позволяет процессор (=адресное пространство).

140. Как банки памяти используются для расширения машинного слова?

Идея банков памяти: давайте подключим к процессору несколько микросхем памяти. Условимся, что в одном банке будут храниться младшие биты машинного слова, а во втором - старшие. Тогда при чтении каждый банк выставит на шину свою часть слова, а процессор будет иметь возможность прочитать информацию по объему бОльшую чем машинное слово каждого банка по отдельности.

Здесь есть еще приколы с трансляцией адреса для каждого банка.

Представим, что наше машинное слово - 16 бит, банки памяти хранят по 8 бит. Чтобы получить слово по адресу 0b10100 надо условно говоря прочитать байт по 0b10100 и байт по 0b10101 и склеить их. Поэтому можно считать что первый банк хранит четные адреса, другой банк - нечетные. По факту - в нашем примере каждый банк игнорирует младший бит и выдает байт по 0b1010.

141. Что такое сегментная память? Как происходит трансляция адресов? Каковы достоинства и недостатки?

Сегментная память - идея в том, чтобы разделить физическую память на сегменты, каждый из которых будет характеризоваться его номером, адресом начала и размером сегмента. При доступе в память мы будем указывать: какой сегмент нам нужен, какое смещение относительно него нам нужно сделать. По таблице сегментов по номеру сегмента ищем адрес начала сегмента в памяти. Проверяем смещение - если смещение больше размера сегмента - выдаем ошибку.

Более того, можно делать сегменты только на чтение или запретить исполнение инструкций по определенным адресам, храня метаинформацию об этом в таблице сегментов. В общем получаем много плюшек, которые помогут обеспечивать безопасность. Таким образом можно изолировать память задач при многозначности: каждую задачу кладем в свой сегмент, итог - никакая задача не сможет обратиться к памяти другой задачи, потому что получит ошибку адресации. Также таблицы сегментов относительно малы, таблицы сегментов просты в обработке и перемещении, отсутствует внутренняя фрагментация

Недостатки:

Это очень сложно, особенно сложно написать компиляторы, которые будут хорошо делить все на сегменты и поддерживать. Максимум что можно - разделить

на какие-то большие сегменты, но теряется часть классной задумки где мы максимально дробим все на сегменты.

Внешняя фрагментация. Когда при выделении/освобождении сегментов мы получаем маленькие кучки памяти, которые не используются, потому что сильно меньше, чем нам нужны для новых сегментов.

142. Что такое виртуальная память? Как происходит трансляция адресов? Каковы достоинства и недостатки?

Сегментная память не получила распространения потому, что становится сложной. Получается нам надо придумать какой-то интерфейс, который бы сделал все гораздо проще.

Сделаем так, будто каждой задаче предоставлено все адресное пространство и она тут вообще одна и делает что хочет. Получается нам надо сделать какое-то отображение из адресов, которыми оперирует каждая задача в реальные адреса в памяти.

Делается это так - разбиваем всю память на небольшие кусочки, назовем их страницы. И каждая страница адресного пространства задачи будет отображена на такую же страницу куда-то в памяти. Получается чтобы обеспечить задачу памятью нужно, чтобы было понятно, куда в память будет отображаться каждая страница. Это дает возможность разбросать страницы в физической памяти как угодно, при этом для задачи память будет выглядеть целостной и линейной.

А еще такой подход позволяет хранить некоторые страницы не в оперативной, а в любой другой памяти. Например, на жёстком диске. Так работает swap

Чтобы это все работало нам нужна Page table (таблица страниц), которая по номеру страницы будет выдавать адрес начала этой страницы в памяти. К этому адресу мы прибавляем смещение и читаем, все.

Итог: задача думает, что у нее есть все адресное пространство, хотя по факту она делит физическую память с другими задачами, не имеет доступ к памяти других задач, память этой задачи может быть собрана из кусочков, лежать отдельными кусками в физической памяти и т.д.

Отдельное большое преимущество, что отсутствует внешняя фрагментация памяти: мы выделяем память по страницам и нет требования на непрерывность выделяемых кусочков памяти, так что можно заполнять ее плотно, без промежутков.

Недостатки:

Page table быстро становится просто огромной, и по ней долго искать адреса страниц. Надо городить какие-то кэши и все связанные с этим страдания

Нет изоляции внутри задач. Задача может сломать сама себя, перезаписав свои инструкции, не очень хорошо, в сегментах мы могли это настроить.

143. Что такое внутренняя и внешняя фрагментация в контексте сегментной памяти?

Внутренняя фрагментация - когда у нас появляется неиспользуемое пространство внутри страницы, потому что выделили больше чем реально надо (данные размещаются в блоках фиксированного размера).

Внешняя фрагментация - когда у нас появляется неиспользуемое пространство между выделенными сегментами. За счет требования к непрерывности выделяемого куска памяти маленькие кусочки, которые уже свободны, могут оказаться не востребуемыми и простаивать (данные размещаются в блоках произвольного размера).

144. Что такое внутренняя и внешняя фрагментация в контексте виртуальной памяти?

См. 142

145. Как появление сегментной памяти улучшило пользовательский опыт?

В части пользовательского опыта (опыта человека, который получает возможность запускать программы и получать от этого бенефиты):

Теперь стало возможным запускать программы, которые не оглядываются на другие и используют все адресное пространство. Ранее такие программы ломали бы другие программы, сейчас они спокойно уживаются

п.1 за счет изоляции памяти программ, ошибки одной программы в части менеджмента памяти перестали быть проблемой других запускаемых программ

Появилась возможность создавать shared сегменты - подключить один и тот же сегмент (например, код библиотеки) к нескольким программам

146. Как появление виртуальной памяти улучшило пользовательский опыт?

В части пользовательского опыта (опыта человека, который получает возможность запускать программы и получать от этого бенефиты):

Теперь стало возможным запускать программы, которые не оглядываются на другие и используют все адресное пространство. Ранее такие программы ломали бы другие программы, сейчас они спокойно уживаются

п.1 за счет изоляции памяти программ, ошибки одной программы в части менеджмента памяти перестали быть проблемой других запускаемых программ

Появилась возможность расширить память программ за счет того, что страницы можно хранить где угодно, например на диске

147. Какие уровни (виды) задач существуют: основной поток, прерывание, процессы, потоки, зелёные потоки?

1. Main/Kernel/основной поток - первый поток, который автоматически создается при запуске процесса
2. Прерывания. Это самый высокий приоритет. Прерывания не являются «задачами» в привычном смысле, это сигналы от оборудования (таймер, клавиатура, сеть) или программные исключения
3. Процессы. Изолированные адресные пространства. Нет прямого доступа (их задача - решать прикладные задачи, с ограниченным доступом)
4. Потоки. Работают в адресном пространстве процесса. Прямой доступ ко всем его данным (поток исполнения, те последовательности инструкций, которые будут выполняться процессором, которые будут исполняться в одном адресном пространстве)
5. Зеленые потоки. Создаются и управляются средой выполнения (виртуальной машиной), обычно используют кооперативную многозадачность (сами передают управление) и хорошо подходят для задач ввода-вывода (I/O)

**148. Как могут взаимодействовать процессы через основной поток?
Какие бывают разделяемые ресурсы?**

Иногда процессам надо взаимодействовать (они же все-таки на одной машине выполняются, а значит потенциально могут решать схожие задачи, требующие обмена информацией между потоками).

Но тут проблема - каждый процесс изолирован по памяти и имеет свое адресное пространство.

Пути решения:

Shared Memory - когда некоторые страницы доступны сразу для всех процессов

Signals - один процесс может вызвать "прерывание" в другом процессе. Например один процесс может попросить ОС вызвать функцию exit в другом процессе

IO - можно вместе писать в файлы, через сеть, через трубы (pipes)

Организацию всех этих путей обеспечивает и менеджерит непосредственно ОС/ядро/основной поток

149. Что такое процессоры ввода-вывода? Каковы их назначение, интерфейс, достоинства и недостатки?

Процессор ввода вывода - отдельный процессор, стоящий рядом с основным. Ему во время работы основного процессора отправляется программа заточенная под задачи ввода/вывода, которую он начинает выполнять, а основной процессор занимается своими задачами. Это основная задумка и преимущество - разгружает процессор от рутинных задач ввода/вывода

Сейчас почти не применяется ввиду того что это оч сложная и специфичная конструкция. Заменена DMA с похожей концепцией но сильно проще.

150. Что такое контроллер прямого доступа к памяти (DMA)? Каковы его назначение, интерфейс, достоинства и недостатки?

DMA - Direct Memory Access - контроллер прямого доступа к памяти - некоторое устройство, заточенное под то, чтобы работать с внешними устройствами, но главная его задача - получить блок данных от внешнего устройства, разместив его где скажут в памяти, или выдать блок и попросить отправить его внешнему устройству.

По сути упрощенный Channel IO (процессор ввода/вывода) с тем же назначением - освободить процессор от рутинных задач ввода/вывода

Достоинства:

Интерактивность, мы разгружаем процессор от задач ввода/вывода, они идут полностью автономно независимо от процессора

Параллелизм. DMA может работать сразу с несколькими внешними устройствами параллельно

Недостатки:

Сложности при непоследовательном доступе к памяти – DMA заточен под линейные непрерывные блоки

Процессор перестает в полной мере контролировать системную шину в части доступа к памяти, надо ломать голову, как их синхронизировать

151. Какие существуют способы интеграции и взаимодействия контроллера прямого доступа к памяти (DMA) с процессором?

Способы интеграции:

1. Third-party: Управление DMA осуществляется процессором (есть специальное отдельное устройство, которое висит на шине рядом с памятью и мы с ним можем взаимодействовать. НО в таком случае контроллер dma тупеньким становится, он не может обрабатывать прерывания, может только переносить данные)

2. Bus mastering: Управление DMA может осуществляться и устройствами ввода-вывода (на шину ставим самостоятельное устройство, которое имеет свою систему прерываний, информировать процессор о готовности данных к чтению только после прочтения всех данных. Кароче весь ввод-вывод делегировать на DMA устройство)

Способы взаимодействия:

1. Пакетный режим. Приоритет DMA. Передача данных осуществляется единой операцией, которая не может быть прервана процессором.

2. Циклический режим: Приоритет конфигурируется. Для процессора и DMA выделяется фиксированный слот времени в рамках цикла

3. Прозрачный режим: Приоритет процессора. Передача данных, когда процессор не взаимодействует с памятью.

152. Что такое гонка состояний (race condition)? Почему она возникает при вытесняющей многозадачности?

Гонка состояний - когда результат выполнения программы зависит от порядка выполнения задач. Этот порядок в случае вытесняющей многозадачности непредсказуем.

Возникает преимущественно именно при вытесняющей многозадачности, потому что именно этот подход предполагает, что задача может быть приостановлена вообще между двумя любыми инструкциями.

153. Как синхронизируются потоки? Что такое атомарная операция, мьютекс и семафор?

Синхронизация потоков — это механизм, управляющий порядком выполнения потоков для предотвращения конфликтов при доступе к общим данным

С целью избежать гонок состояний были придуманы разные ухищрения, в частности:

Атомарная операция - операция, выполнение которой не может быть прервано. Если она начала выполняться, она выполняется полностью

Мьютекс - некая программная сущность, которую одновременно может захватить только один поток. Реализована может быть разными способами, например с использованием разделенной памяти

Семафор - общий случай мьютекса. Некая программная сущность, которую одновременно может захватить не более N потоков. Реализована может быть разными способами, например с использованием разделенной памяти

13 ЛЕКЦИЯ

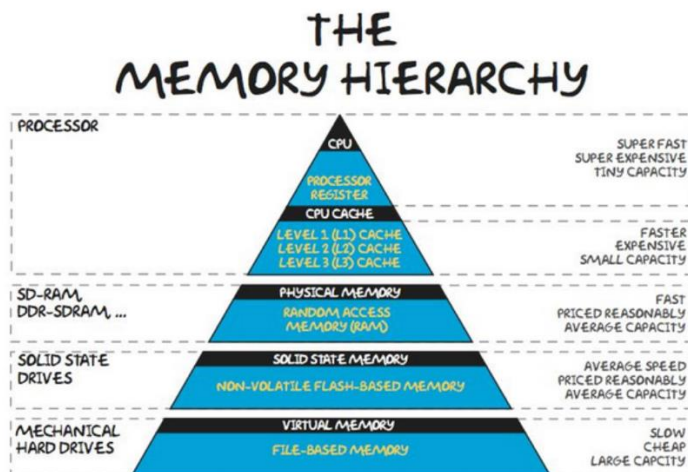
154. Почему необходима иерархия памяти в современных компьютерных системах?

Иерархия памяти возникает естественным образом из такого свойства памяти - чем память быстрее, тем она дороже. Из-за этого получается так, что у нас есть очень ограниченное количество быстрой памяти и очень много медленной. Возникает иерархия, когда мы комбинируем разные виды памяти, чтобы эффективнее решать свои задачи

155. Какие основные виды памяти входят в иерархию памяти и каковы их характеристики?

Процессор, его регистры -> его кэши -> физическая память -> ssd диски -> механические жесткие диски -> всякие сетевые диски, магнитные ленты и т.д.

Чем выше – тем меньше и дороже



1. Регистры цпу: их мало, они быстрые и управляются напрямую процессором и работают на той же частоте, что и процессор. Хранение текущих инструкций и промежуточных данных, обрабатываемых ЦП прямо сейчас
2. кеши L1-L3 (SRAM) - буферизация данных между медленной оперативной памятью и быстрыми регистрами для уменьшения времени ожидания процессора:
L1: вообще разделен на L1i(instruction) и L1d(data). Тоже маленькие и быстрые.
L2: как L1, только чуть больше и медленнее
L3: общий для всех ядер
3. Оператива(DRAM): Динамическая память. Основная рабочая память системы для хранения выполняемых программ и данных.
4. ssd: быстрая энергозависимая память, живут еще долго, так как в них нет движущей части
5. hdd: медленный, но дешевый и емкий, используется для долговременного хранения больших объемов данных
6. магнитные ленты, сетевые хранилища. Очень большой объем, очень низкая скорость доступа. Долгосрочное архивирование данных, резервное копирование.

156. В чём различие между явной иерархией памяти и скрытой? Приведите примеры.

Есть две иерархии памяти: явная (управляется программистом) и скрытая (управляется аппаратно/автоматически)

Явная иерархия - мы четко можем определить, какой тип памяти в какой момент времени для хранения чего конкретно будет использован. С расчетом на то, что мы как программисты лучше знаем, что лучше положить в SRAM что в DRAM что на HDD.

Неявная иерархия - идея в том, что программист будет работать как с одной памятью, не различая ее, он будет просто обращаться к данным и получать их. Главное только реализовать этот слой абстракции, что программист чаще будет

обращаться к самой быстрой памяти и дальше по мере частоты все медленнее и медленнее.

В современном мире на уровне работы с памятью чаще всего применяется скрытая иерархия, потому что виртуальная память объединяет в себе и кэши, и основную память, и память жестких дисков. Это и есть скрытая иерархия. При этом частичные признаки явной иерархии итак или иначе тоже есть, например при работе с вводом-выводом, регистрами там и так далее

157. Какие типы доступа к памяти существуют (произвольный, последовательный, гибридный)? Приведите примеры.

С произвольным доступом. RAM. Мы можем адресовать любой байт информации и мы можем сделать это бесплатно, то есть скорость доступа к памяти не зависит от того, куда мы обращались в прошлый раз. Эта история ломается об кэш

С последовательным доступом. Жесткие диски, магнитные ленты. Хранилище устроено таким образом, что мы вынуждены завязаться на какой-то внешний физический процесс, чтобы попасть в нужную ячейку памяти. Длительная задержка при смене адреса

Гибридная. Для хранения огромного объема данных, когда стоит огромный шкаф полностью автоматизированный, перед этим катается штука, которая выхватывает нужную ленту, кидает ее в ридер и потом можно прочесть эти данные. Тут нет никакого практического смысла

158. Каково устройство памяти с произвольным доступом (decoder, wordline, bitline)? Как устроена ROM-ячейка и как происходит чтение?

RAM – это энергозависимая память (сохраняет данные только при наличии электрического питания), позволяющая читать и записывать данные в произвольном порядке

Чаще всего устроено так - мы берем простейшие ячейки, которые умеют хранить один бит и собираем их в матрицу. Число столбцов соответствует числу бит в машинном слове, число строк - тому, сколько машинных слов мы сможем хранить. Сбоку подключается декодер, на который приходит адрес. Как только адрес приходит, декодер включает определенную строчку, заставляя ячейки, расположенные по этому адресу выставить на шину соответствующий бит.

В целом это работает только для небольших матриц, банально потому, что сделать большой декодер достаточно сложно, что линии становятся очень большими для больших матриц и все это начинает работать медленно и плохо. Поэтому в больших чипах все устроено чуть сложнее, но базово именно так.

Ячейка ROM (Read Only Memory) - ячейка памяти для постоянного хранения информации, базово позволяет только читать из нее.

Есть два типа таких ячеек - ячейка которая хранит "1" и которая хранит "0". Которая хранит "1" не делает ничего, потому что и так по умолчанию на шину

выставляется "1". Которая хранит "0" содержит транзистор - если данная ячейка выбрана декодером, то ячейка начинает замыкать линию в "0".

Чтение такой памяти происходит так: есть wordline (выбирает строку в памяти), bitline (считывает бит), транзистор. Если мы на w1 подаем сигнал, то транзистор открывается и соединяет bl с землей (то есть выставляет 0) или с питанием (выставляется 1). В итоге память представляет из себя наличие или отсутствие транзистора.

159. Каково устройство многопортовой памяти? Как связана площадь с количеством портов?

Многопортовая память - это память, которая позволяет нескольким устройствам (процессорам) одновременно читать и/или писать данные в несколько ячеек памяти.

Используется это в кеш-памяти CPU/GPU, сетевых процессорах

В целом память с произвольным доступом можно сделать многопортовой. Для этого нам надо поставить второй декодер, проложить вторые линии wordline и bitline и дублировать обвязку «сердца» ячейки памяти (в случае SRAM "сердце" - это два инвертора, а обвязка - два access FET'a (транзистора))

Ну и чем больше будет портов темкратно дороже будет память, тем больше площади на чипе все это будет занимать. Применяется крайне редко, потому что количество памяти остаётся таким же, а ее стоимость умножается кратно

160. Сравните технологии реализации битовых ячеек (Flip-flop, ROM, SRAM, DRAM) по числу транзисторов, скорости, плотности и стоимости. В какой области иерархии памяти каждая из них применяется?

Для реализации памяти нам полезно уметь делать битовые ячейки - некоторые конструкции, которые могут быть в двух устойчивых различных состояниях. Именно они и хранят нужную нам информацию

Тип ячейки	Число транзисторов	Задержка	Типовой объем	Цена за Гб
Flip-Flop (это вот как раз триггеры всякие)	~20	ооочень маленькая, 20ps	1-5кБ	Оч много
ROM	0-1	Чтение быстро, запись часто очень затруднена	-	В целом недорого, но depends

Тип ячейки	Число транзисторов	Задержка	Типовой объем	Цена за Гб
		или невозможна		
SRAM	6	наносекунды	10кБ-10МБ	~1000\$
DRAM	1	Десятки наносекунд	16ГБ-64ГБ	~10\$

1. Flip-flop (Триггеры). Используются для создания регистров внутри процессора. Основаны на перекрестно-связанных инверторах, что обеспечивает максимальную скорость. Требуют много площади, поэтому плотность минимальна.
2. SRAM (Static RAM - Статическая ОЗУ). Очень высокая скорость, высокая стоимость, низкая плотность по сравнению с DRAM. Используются в кэш-памяти процессора (L1, L2, L3)
3. DRAM (Dynamic RAM - Динамическая ОЗУ). Меньше скорость (из-за регенерации), но высокая плотность и низкая цена за бит. Применение: Основная оперативная память (RAM) компьютера
4. ROM (Read-Only Memory - ПЗУ). Энергонезависимая, очень дешевая в производстве, но не позволяет быструю запись (или вообще не позволяет). Для хранения микрокода

161. Каковы технологии реализации SRAM ячеек? Как происходит чтение и запись?

SRAM - это статическая память с произвольным доступом, которая хранит данные в виде состояний триггера. Static значит, что данные записанные в такую ячейку памяти будут там храниться произвольно долго (до отключения питания)

Триггер — устройство, способное хранить 1 бит информации, которое обычно собирается из нескольких транзисторов.

Реализуется с помощью 6 транзисторов. Из них по два уходит на каждый из двух инверторов; и два для обеспечения доступа к ячейке. Инверторы, они же логическое "НЕ".

Такая комбинация двух инверторов заставляет их поддерживать одно из двух устойчивых состояний, которые и интерпретируются как сохраненный бит

Транзистор в компьютере — полупроводниковый переключатель, который управляет электрическим током, выступая в роли «включателя/выключателя». Миллиарды таких транзисторов, размещенных на процессоре, формируют двоичный код (0 и 1), обеспечивая вычисления и логические операции.

Чтение происходит следующим образом: когда декодер выставляет "1" на wordline, боковые транзисторы (access FETs) подключают инверторы к bitline которые по умолчанию оба подключены к питанию. И соответственно в зависимости от того, в каком состоянии сейчас инверторы у нас на линиях будет либо "1,0" либо "0,1" и специальное устройство на выходе интерпретирует эти состояния как 1 или 0 и выдает его на шину.

Запись происходит следующим образом: вместо того, чтобы оба bitline подключать к питанию, мы одну линию подключаем к питанию, а другую к нулю. По итогу инверторы выбранной ячейки перестроятся под заданное состояние и примут новое устойчивое состояние.

162. Каковы технологии реализации DRAM ячеек? Как происходит чтение и запись? Что такое регенерация и зачем она нужна?

DRAM - динамическая память, которая хранит данные в виде заряда на конденсаторе. В отличие от sram она требует периодического обновления, но зато дешевле.

Dynamic - сохраняем данные не в стабильном состоянии, а в конденсаторе, то есть какое-то время там пролежит.

Состояние в ячейках DRAM хранится в конденсаторе, и 1 или 0 определяется тем, заряжен конденсатор или нет.

Из-за такого упрощения возникают проблемы:

Заряд конденсатора утекает, его надо перезаряжать постоянно

Считать значение можно только один раз, так как чтение бита разряжает конденсатор

Нужен специальный контроллер, который постоянно будет перезаряжать нужные конденсаторы

Запись происходит так: Декодер выбирает ячейку через wordline. На bitline выставляем или 1 или 0. Если 1 - конденсатор зарядится, если 0 - разрядится. Ячейка записана

Чтение происходит так: выставляем на битовую линию половину напряжения, которое считается единицей. Активируем wordline. Если там была 1, то на линии мы увидим всплеск, пока конденсатор не разрядится до 0.5. Если там был 0, то линия просядет пока конденсатор не зарядится до 0.5. Процесс чтения ломает

заряд, который был в ячейке, поэтому после чтения надо восстановить его "записью"

Регенерация - процесс, когда контроллер памяти периодически производит процесс "чтения" памяти, обновляя заряды конденсаторов, чтобы они не утекали (точнее утекать они все равно будут, просто обновление не дает этому повлиять на нас).

163. Что такое кеш? Каково его назначение, место в иерархии памяти и основные метрики?

Кеш - промежуточный буфер с быстрым доступом, содержащий информацию, которая может быть запрошена с наибольшей вероятностью

Назначение - уменьшить задержки доступа к информации и ускорить работу систему (если по пунктам, то сократить время доступа к памяти, снизить нагрузку на RAM, оптимизировать пропускную способность).

Кеш – чип, место в иерархии памяти между процессором и основной памятью. Кеш состоит из набора кеш-линий (блоков кеша, записей). Каждая запись ассоциирована с элементом данных или блоком данных, который является копией элемента данных в медленной памяти. Каждая запись имеет:

- Индекс --- указывает ячейку в кеш-памяти;
- Тег --- идентификатор блока основной памяти, который отображён в кеш;
- Данные --- содержимое, которое сохраняется и извлекается, но не анализируется самим кешем.

Метрики:

Hit Rate - доля запросов, которые кеш смог обработать без обращения к RAM.

Miss Rate - уже наоборот

Latency (Задержка доступа): время на чтение и запись данных

Ассоциативность: определяет, сколько мест в кеше может хранить данные по одному адресу

Размер строки кеша: объем данных, загружаемых за 1 раз

Кеш - это память (практически всегда это SRAM, быстрая удобная, не надо регенерировать) и логика

Логика должна уметь:

1. Искать кэш-линию (машинное слово например) по тегу
2. Понимать какая кэш-линия сейчас самая бесполезная, уметь замещать ее новой информацией
3. Предзагружать вероятные данные, инструкции

4. Менеджерить работу с памятью (группировать операции и все такое прочее)
5. Взаимодействовать с кешами других уровней

164. Что такое принцип локальности и каковы его виды (временная, пространственная) применительно к механизму кеширования?

Принцип локальности - идея о том, что по факту в прикладных задачах чаще всего доступ к памяти происходит неравномерно, т.е. в некоторой степени предсказуемо из-за локальности данных. Например, процессор последовательно читает инструкции по последовательным адресам. Этот принцип лежит в основе эффективной работы кеш-памяти, позволяя хранить часто используемые данные ближе к процессору для ускорения доступа.

Временная локальность: если программа обращается к данным или инструкциям в определенный момент времени, высока вероятность, что они понадобятся снова в ближайшем будущем.

Пример: Циклы, повторное использование переменных.

Применение: Кеш сохраняет недавно использованные данные, чтобы сократить время доступа при повторном обращении.

Пространственная локальность: если программа обращается к данным по определенному адресу, высока вероятность, что вскоре потребуются соседние данные.

Пример: Последовательный перебор элементов массива

Применение: Кеш загружает не только запрошенные данные, но и соседние блоки, чтобы уменьшить количество промахов.

165. Как пошагово происходит процесс чтения данных через кеш-память процессора?

Ситуация: мы хотим прочитать значение по некоторому адресу в памяти.

Сначала наш запрос попадает в кеш, кеш начинает поиск на совпадение тега какой-либо строки с запрошенным адресом

Если кеш находит адрес это называется *cache hit*, данные из внутренней таблицы кеша попадают сразу в процессор

Если кеш не находит адрес это называется *cache miss*, данные запрашиваются извне (из кеша другого уровня либо непосредственно из основной памяти). Время доступа становится непредсказуемым - память может быть заблокирована кем-либо, кеш может иметь произвольное количество уровней. Кеш выделяет строку в своей внутренней таблице (свободную, если есть; либо перезаписывает какую-то из имеющихся). Записывает в эту строку данные, помечает их тегом (адресом, с которого они были взяты). Отдает данные процессору

166. Как пошагово происходит процесс записи данных через кеш-память процессора? Каковы политики записи при попадании (write-through, write-back) и при промахе (write-allocate, no-write-allocate)?

Ситуация: мы хотим записать данные по некоторому адресу в памяти.

Наш запрос попадает в кеш и дальше есть два варианта:

В кэше уже лежит эта ячейка памяти, cache-hit:

Тут есть несколько вариантов, их называют политиками записи, применяется одна из перечисленных:

- Немедленная запись (write-through) - обновляем ячейку в кеше, передаем изменение дальше. Таким образом значение будет обновлено на всех уровнях кэширования и в памяти. Происходит это все долго
- Отложенная запись (write-back) - мы обновили ячейку в кеше и все. В случае, если когда-то эту ячейку потребуется удалить из кеша, обновление пойдет на следующий уровень кэширования/в основную память

В кэше нет этой ячейки памяти, cache-miss:

Тут тоже несколько вариантов, а именно два:

- Запись с размещением (write-allocate) - ячейка размещается в кеш, дальше все по сценарию cache-hit
- Запись без размещения (no-write-allocate) - запись производится напрямую в память, кеш не трогаем. Минус в том, что возможно было бы эффективно разместить ячейку в кеше, что позволило бы следующие операции записи выполнять куда быстрее

167. Что такое кеш-промах? Как промах по-разному влияет на чтение данных, запись данных и чтение инструкций?

Кеш-промах (cache-miss) - когда мы обращаемся к кешу с запросом определенного адреса (на чтение/запись), а этой ячейки в кеше не оказалось. Плохая ситуация, если случается часто и систематически, то может полностью убить все преимущества кеша

"Типы" кеш промахов:

Чтение инструкций - Большая задержка. Если процессор не может получить следующую инструкцию из кеша, он вынужден приостановить выполнение текущего потока команд, поскольку не знает, какую операцию выполнять дальше.

Чтение данных - Средняя задержка. Может компенсироваться параллелизмом уровня инструкций (в случае суперскалярности или спекулятивного исполнения, которое заранее выполняет вероятные операции). Когда процессор обнаруживает, что требуемые данные отсутствуют в кеше, он инициирует запрос к основной памяти. Но процессор не обязан полностью останавливать работу --- он может продолжать выполнять другие, независимые инструкции, пока ожидается

поступление данных. После того как информация наконец поступает из памяти, возобновляется выполнение инструкций, которые от неё зависели.

Запись данных - Минимальная задержка. Запись может быть отложена, например в буфер, процессор спокойно работает дальше

14 ЛЕКЦИЯ

168. Что такое ассоциативность кеша? Какова "структура" адреса с точки зрения кеша?

Ассоциативность кэша - способ выбора кеш линии под конкретную ячейку памяти. Т.е. нам приходит информация, у нее есть тег. В какую ячейку кеша ее сохранить? На этот вопрос отвечает ассоциативность. Ассоциативность кеша определяет как данные из оперативной памяти распределяются и хранятся в кеш памяти.

Выбор ассоциативности кеша может сильно повлиять на скорость поиска в кеше. Пример:

Если мы кладем в первую попавшуюся ячейку, то чтобы найти какую-то ячейку в кеше придется проверить их все в худшем случае. $O(N)$

Можно выстроить что-то поинтереснее, хранить по определенным правилам, тогда и поиск будет быстрее (как в программировании: бинарное дерево, поиск $O(\log(N))$ / хешмап вообще за $O(1)$)

Структура адреса с точки зрения кеша (а надо ли нам сохранять весь адрес как тег хэш линии?):

Полный адрес. Слишком дорого: требует много памяти.

Младшие биты. Можно отбросить, потому что хеш оперирует блоками а не байтами. Например, если блок хеша 64 байта то можно спокойно отбросить 6 бит адреса

Средние биты. Циклически повторяются, когда мы читаем память последовательно.

Старшие биты. +- уникальны для большого куска памяти

Адрес памяти обычно делится на:

Тег. Уникальный идентификатор блока, хранящегося в строке – старшие биты

Индекс. Номер строки кэша – средние биты

Смещение. Смещение внутри блока данных – младшие биты

169. Каковы особенности реализации полностью ассоциативного кеша?

Решение "в лоб" - любой блок памяти может попасть в любую кеш линию. Для каждой кеш-линии стоит компаратор, который сравнивает пришедший адрес с

тегом кеш-линий. Приходит адрес - компараторы сравнивают, если кто-то нашел, данные выставились, пошел доступ

Из адреса отбрасываются только младшие биты, в итоге по старшим битам ищем физические вхождения.

Особенности:

Очень много логики, все эти компараторы — это достаточно дорого и сложно. Логика вытеснения может быть произвольной сложности

Но достаточно эффективен

170. Каковы особенности реализации кеша с прямым отображением?

Кеш с прямым отображением = Каждая ячейка может быть отображена только в 1 кеш-линию

Возьмем адрес, отрежем у него младшие биты потому что мы храним блоки. Возьмем столько средних битов, сколько кеш-линий у нас есть (это будет номер нашей кеш-линии. Эквивалентно остатку от деления адреса на размер кеша). А в тег запишем только оставшиеся старшие биты.

Тогда при поиске мы точно будем знать, в какой ячейке кеша искать запрошенный адрес (просто берем средние биты, это и будет номер кеш-линии). Берем оттуда тег, если он совпадает с запрошенным адресом, значит мы попали в кеш (cache-hit), пошел доступ к данным

Особенности:

В теге хранится куда меньше информации

Упрощение за счет того, что компаратор теперь один

Нет вариативности логики вытеснения - такая ассоциативность четко задает куда мы должны записать очередной блок

171. Что такое множественно-ассоциативный кеш и что определяет уровень множественности?

Множественно-ассоциативный кеш - гибрид прошлых двух.

Он сочетает преимущества обоих подходов:

- Внутри — это несколько кешей с прямым отображением, работающих параллельно;
- Снаружи --- действует как ограниченно ассоциативный кеш: каждый блок памяти может быть размещён в одном из нескольких вариантов.

Как ищем - берем адрес и ищем его в каждом подкеше с прямым отображением как обычно, получаем несколько блоков и смотрим с кем у нас совпадает тег.

Уровень множественности (степень ассоциативности) - сколько вариантов размещения для одного адреса существует в кэше. Если $N = 1$, это кэш прямого

отображения. Если N равно количеству строк в кэше, это полная ассоциативность. Пример: 8-way set associative: Блок может разместиться в любой из 8 строк в наборе.

172. Как связана ассоциативность кеша с механизмом вытеснения и замещения?

Механизм вытеснения и замещения нужен нам тогда и только тогда, когда для одного блока памяти по определенному адресу может быть несколько возможных мест в нашем кеше. Например для кеша с прямым отображением не нужна логика вытеснения, замещения, потому что там одному адресу строго соответствует одно место в кеше

Чем выше ассоциативность, тем:

Меньше принудительных вытеснений, так как у блока памяти больше возможных мест в кеше.

Но сложнее механизм замещения, потому что надо выбирать одну строку из N кандидатов в наборе

В процессорах применяются алгоритмы:

LRU (least recently used)

Pseudo LRU.

173. Как работает механизм вытеснения и замещения LRU в кеше?

Когда нам надо понять, какую кеш-линию можно наименее болезненно удалить из кеша возникает проблема как понять, какая именно кеш-линия самая ненужная сейчас.

Есть тип такого механизма как LRU (last-recently-used), когда мы считаем что наверное самая ненужная линия это та, к которой уже давно никто не обращался. Как понять какая? Для этого мы заводим глобальный счетчик доступов к кешу (это будет наш "timestamp") и при чтении/записи в кеш-линию будем записывать текущее значение счетчика. Если требуется вытеснить какую-то линию, то вытесняем ту, у которой записанное значение минимально.

Звучит неплохо, на деле очень сложно реализовать сортировку этих значений на лету, плюс у счетчика должна быть как-то ограничена разрядность, а это проблемы с переполнением и короче сейчас такая схема используется очень редко из-за сложностей в реализации.

174. Как работает механизм вытеснения и замещения PLRU в кеше?

Гениальный способ определения самой ненужной ячейки – псевдо LRU. Как он работает:

Мы создаем бинарное дерево, где его листья - наши кеш-линии, а в любой другой вершине либо 0 либо 1. "0" означает стрелку влево, "1" означает стрелку вправо.

При доступе к какой-либо ячейке мы меняем все 1 на 0, а 0 на 1 по пути к этой ячейке через дерево.

Таким образом получается, что дерево всегда будет указывать на "самую ненужную" ячейку, которую можно перезаписать. Так как это а-ля бинарное дерево, число листьев (кеш-линий) должно быть степенью двойки, правда в кешах это так и есть зачастую. Число ячеек для реализации такого дерева - $N-1$ бит

175. Почему необходимо использование многоуровневой иерархии кеш-памяти в современных процессорах?

Мы вынуждены использовать многоуровневую систему, потому что мы одновременно хотим большой и быстрый кеш, но при этом не хотим отдавать много денег/места/энергии.

Поэтому придумали сделать многоуровневую структуру, которая является компромиссом по объему/скорости/стоимости.

Классифицировать многоуровневые кеши можно по 3 признакам:

Разделенный/унифицированный

Включающий/исключающий

Частный/общий

Выбирая разные характеристики кешей, мы будем получать огромную разницу в их размере.

Для реальной эффективной работы нам нужно выстраивать несколько кешей с разной специализацией (для данных и инструкций, специализированный для работы одного процессора или группы процессоров, для оптимизации с работой памяти).

176. В чём разница между разделённым и унифицированным кешем и где они применяются?

Разделенный кеш: Мы создаем два физически разных кеша, один будет отвечать за данные, другой за инструкции. И получается что мы на уровне кешей имитируем гарвардскую архитектуру, хотя память и канал доступа к памяти один! Получаем возможности конвейеризации без конфликтов по доступу к памяти. Лучше работают каждый из кешей за счет бОльшей локальности обрабатываемых данных

Унифицированный кэш — это единый блок памяти, который хранит и инструкции, и данные совместно.

Разница: у разделенного скорость выше, пропускная способность выше, по сложности проще (он проще для конвейера процессора. А унифицированный сложнее, требует более сложного контроллера)

Где они применяются

Разделённый кэш: практически во всех современных микропроцессорах (Intel, AMD, ARM) на уровне L1. Так как, процессор часто выполняет конвейерную обработку: пока один блок берет следующую команду (инструкцию), другой блок работает с данными (чтение/запись). Разделённый кэш позволяет делать это одновременно, без конфликтов за память.

Унифицированный кэш: в качестве L2 и L3 кэша в процессорах, так как на уровне L2 и L3 критична не скорость (как у L1), а «попадание» в кэш. Унифицированный кэш лучше использует общий объём: если программе нужно больше данных, она займет ими больше места. Если нужно больше инструкций — она займет ими

177. В чём разница между включающим и исключаящим кешем и где они применяются?

Включающий кэш гарантирует, что все данные, находящиеся в кэше более высокого уровня (L1), обязательно дублируются в кэше более низкого уровня (L2 и L3), L2 дублируется в L3.

Исключающий кэш гарантирует, что данные находятся только на одном уровне кэша: если они есть в L1, их нет в L2, и наоборот.

Разница: про когерентность - для проверки, нет ли каких-то данных, во включающем кеше надо опросить только общий кеш L3, а в исключаящем требует спроса всех кешей, что увеличивает задержки. Во включающем кеше минус – дублирование данных, объем кеш-памяти равен размеру L3. В исключаящем суммарный объем кеш-памяти процессора равен $L1 + L2 + L3$, ничего не дублируется. Вытеснение из L1: включающий - просто удаляются (или остаются в L2 при наличии места), исключаящий - перемещаются в L2/L3. Вытеснение из L3: включающий - удаляются из L2/L1, исключаящий - просто удаляются (не влияют на L1/L2)

Включающий кэш: используется, когда критически важна простота реализации когерентности кэша, например, в многоядерных системах, где L3 является общим для всех ядер.

Исключающий кэш: используется, когда важнее максимизировать эффективный объем памяти, например, в системах с ограниченным объемом кэша.

178. В чём разница между частным и общим кешем и как они применяются в многоядерных процессорах?

Частный кэш: кэш-память, выделенная для каждого отдельного ядра (доступ только у одного ядра). То есть ядро хранит там только свои инструкции и данные.

- Преимущества:

Низкая задержка: Ядро мгновенно получает доступ к своим данным

Отсутствие конкуренции: Другие ядра не забивают этот кэш своими данными.

- Недостатки:

Если ядрам нужна одна и та же информация, она дублируется в разных кэшах. При изменении данных одним ядром, копии в других нужно аннулировать, что создает нагрузку на шину.

Общий кэш: кэш-память, разделяемая между всеми (или группой) ядрами. Хранит данные, к которым часто обращаются несколько ядер одновременно или последовательно.

- Преимущества:

Эффективность объёма: Данные не дублируются, что позволяет хранить больший объём уникальной информации.

Гибкость: если одно ядро простаивает, другое может использовать его часть общего кэша.

- Недостатки:

Конкуренция: Ядра могут «вытеснять» данные друг друга, если объём данных превышает размер кэша.

Применение: современные процессоры используют иерархическую структуру:

L1 + L2: Частные для каждого ядра (активная работа).

L3: Общий для всех (синхронизация и хранение больших данных).

179. Что представляет собой типовой многоуровневый кеш: L1, L2, L3, L4? Каковы их назначение, тип ячеек, классификация?

L0 (опция) - специальный кеш для: стека, int/float чисел. Обычно доступен за такт. Чаще всего даже не выносится как отдельный кеш

L1. Самый быстрый буфер. Загружает данные, необходимые процессору «прямо сейчас». Объём — мал (обычно 32-128 КБ на ядро), разделен на L1i (инструкции) и L1d (данные). SRAM (Static RAM), очень быстрая.

L2. Буфер для L1. Снабжает ядра данными, если их нет в L1 (cache miss). Больше L1 (от 256 КБ до нескольких МБ), обычно индивидуален для ядра, но медленнее L1. SRAM.

L3. Общий буфер для всех ядер процессора. Обеспечивает когерентность данных между ядрами. Самый большой (десятки МБ), медленнее L2, но всё же в разы быстрее ОЗУ. SRAM.

L4. Редкий уровень. Используется в высокопроизводительных системах или при объединении ЦП+ГПУ для хранения больших объёмов данных. Очень большой, но медленнее L3. eDRAM (Embedded DRAM) — более плотная, чем SRAM, но медленнее

180. Что такое когерентность кеш-памяти и каково её значение для многоядерных систем? Какие существуют возможные состояния кеш-линий?

Когерентность - целостность данных в кешах процессора между собой и в памяти. Т.е. ситуация, когда все ядра при чтении будут получать одинаковые и более того актуальные данные, согласованные с RAM. В многопроцессорных системах без когерентности возникли бы проблемы, когда одно ядро изменяет данные, а другое продолжает использовать устаревшую копию из своего кеша.

Для решения этой проблемы придумали хранить состояния кеш линий, ну и целый протокол на его основе - MOESI:

Modified - текущее ядро изменило эту кеш линию, здесь свежие данные и они есть только в этом ядре. Память об этом еще не знает

Owned - текущее ядро является владельцем данных и готово выдать другим ядрам корректные данные, причем другие могут хранить копию только в состоянии Shared

Exclusive - данные в линии есть только у текущего ядра и они актуальны, совпадают с тем что в оперативной памяти. Ядро может спокойно туда писать.

Shared - данные актуальны, но свободно их можно только читать. Если хочется изменить, придется сначала инвалидировать их в других ядрах, а свое перевести из Shared в Modified или Exclusive

Invalid - Данные невалидны или пустая строка

181. Как происходит обмен информацией о кеш-линиях через справочник? Как этот подход масштабируется?

Один из способов реализации обмена информацией это через справочник. Т.е. появляется некоторый центральный блок, который подключен к кешу каждого ядра, и внутри себя хранит таблицу-справочник. Для каждой кеш-линии в системе он хранит:

Modified bit. Изменена ли эта линия по сравнению с тем, что есть в основной памяти (RAM)

Presence bits. Для каждого ядра здесь есть отдельный бит, который показывает, есть ли копия этой кеш-линии в кеше конкретного ядра.

Процесс записи:

1. Процессор P1 хочет записать данные. Он отправляет запрос «запись» справочнику
2. Справочник проверяет, у кого есть эта строка.
3. Справочник отправляет сообщения инвалидации (удаления) только тем процессорам (P2, P3) у которых эта строка находится в кэше.
4. P2, P3 инвалидируют свои копии и отправляют подтверждение справочнику.
5. Справочник обновляет состояние строки (устанавливает владельцем P1 и дает P1 разрешение на запись).

Если же процессор хочет прочитать данные, то справочник просто помечает это в presense bits.

Масштабируемость: вместо того чтобы оповещать все ядра в системе, запросы отправляются точно только тем ядрам, у которых реально есть копия данных. Это устраняет бутылочное горлышко в виде общей шины

Бутылочное горлышко (bottleneck) — это образное выражение, обозначающее «узкое место» в системе, которое ограничивает производительность всего процесса

182. Как происходит обмен информацией о кеш-линиях через отслеживание? Как этот подход масштабируется?

Идея обмена информацией через отслеживание в следующем - давайте сделаем так, что общение кешей с памятью будет проходить по одной общей шине. Получится так, что каждый кеш сможет слушать, что остальные делают с памятью и в соответствии с этим обновлять там свои статусы, помечать данные и так далее.

Если один процессор пишет в кеш-линию, он отправляет сигнал недействительности (Invalidate) на шину. Все остальные контроллеры кеша видят этот сигнал, проверяют наличие этой линии у себя и, если она есть, помечают её как недействительную (Invalid).

Если процессор читает данные, он запрашивает их. Если другой кеш имеет эту линию в "грязном" состоянии (Modified), он либо обновляет основную память, либо передает данные напрямую читающему процессору

Проблема масштабирования тут стоит очень остро, потому что поскольку все запросы должны транслироваться всем процессорам, пропускная способность общей шины становится «узким местом». Чем больше ядер, тем выше нагрузка на шину и тем чаще возникают конфликты доступа

ВАЖНО, что здесь мы говорим о том, что прокладываем общую шину адреса, и кеши опираются только на это ("о, другой кеш записал в память адрес, который есть и у меня. Инвалидирую свой адрес")

183. Как происходит обмен информацией о кеш-линиях через перехват? Как этот подход масштабируется?

Если в случае подхода через отслеживание мы прокладываем только шины адреса, то здесь мы прокладываем еще и шину данных. Теперь кеши могут видеть, кто что отправляет в память и не только обновлять состояния своих ячеек, но и сразу обновлять их значения! Это сильно оптимизирует всю систему, ну и помогает в полной мере реализовать MOESI (потому что честно говоря в логике "через отслеживание" у кешей нет вариантов обмениваться между собой данными, и как тогда работает логика Owned-Shared непонятно. в случае перехвата все супер)

Если Core A хочет изменить строку, он отправляет сигнал аннулирования (Invalidate). Все остальные контроллеры видят этот сигнал и помечают свои копии этой строки как "Invalid" (недействительные).

Если Core A читает, а строка изменена в кеше Core B, то Core B «подслушивает» запрос, блокирует передачу данных из основной памяти, и сам предоставляет актуальные данные, обычно обновляя их в памяти

Проблема масштабирования тут стоит очень остро, потому что поскольку все запросы должны транслироваться всем процессорам, пропускная способность общей шины становится «узким местом». Чем больше ядер, тем выше нагрузка на шину и тем чаще возникают конфликты доступа

184. Какие существуют типы кеш-промахов (3C + 1: cold, capacity, conflict, coherence)? Какие способы смягчения характерны для каждого?

Cold - происходят при первом обращении к блоку данных. Данных еще нет в кеше, так как он «холодный» (пустой). Смягчается предвыборкой - аппаратное или программное заблаговременное чтение данных в кеш перед их фактическим запросом. А также прогревом кеша - предварительной загрузкой популярных данных при старте приложения.

Capacity - кеш слишком мал, чтобы вместить все рабочее множество данных, необходимых программе. Данные вытесняются и затем снова загружаются, даже если структура кеша идеальна. Смягчается увеличением размера кеша - физическое увеличение объема кеш-памяти (например, использование L3 вместо L2).

Conflict - данные вытесняются, потому что слишком много блоков памяти отображаются на один и тот же набор в кеше, хотя в других частях кеша есть свободное место. Характерны для кешей с прямым отображением или низкой ассоциативностью. Смягчается повышением ассоциативности - увеличение количества путей в ассоциативном кеше (например, с 2-way на 4-way).

Coherence - в многопроцессорных/многоядерных системах, когда одно ядро изменяет данные, копия в кеше другого ядра объявляется недействительной (invalidated). Следующее обращение второго ядра вызывает промах. Смягчается оптимизированием обмена данными - минимизация общего доступа к данным между ядрами.

185. Что такое CAP-теорема? Какова область её применения и как она применяется к когерентности кешей?

CAP-теорема — это фундаментальный принцип в проектировании распределенных систем, который утверждает, что распределенная база данных или хранилище данных может обеспечить одновременно не более двух из трех следующих гарантий:

Consistency (согласованность) - если вы записали данные на один узел, то любой последующий запрос на чтение с любого другого узла вернет это новое значение (или произойдет ошибка, если данные еще не синхронизировались). Пример: В банковской системе, после того как вы перевели деньги, баланс на всех репликах базы данных должен стать одинаковым немедленно.

Availability (доступность) — каждый запрос к распределенной системе завершается корректным откликом (успешным или с ошибкой), без гарантии того, что он содержит самую последнюю версию данных. Пример: Социальная сеть. Если часть серверов упала, вы все равно видите ленту новостей (даже если она устарела), а не ошибку 500.

Partition Tolerance (устойчивость к разбиению) — если связь между двумя узлами пропадает, система не «падает», а продолжает работать, разделяясь на две части, каждая из которых функционирует независимо.

Ну и в области когерентности кешей у нас нет требования к "Устойчивости к разбиению", так что теорема применяется в формате "возможно обеспечить Согласованность и Доступность"

186. Что означают термины "Consistency", "Availability" и "Partition tolerance" в CAP-теореме? Опишите их.

См. выше

187. Как кеш влияет на производительность памяти? Какие существуют способы оптимизации доступа, включая AoS и SoA?

Кеш кардинально повышает производительность, устраняя разрыв в скоростях. Так как: снижение задержек - доступ к L1-кешу занимает 1-4 такта, к L2 — 10-20, а к оперативной памяти — 200-400+ тактов. Кеш позволяет процессору получать данные почти мгновенно, не простаивая; увеличение пропускной способности, потому что процессор загружает данные не побайтово, а целыми строками (cache lines, обычно 64 байта). Но при кеш-промах процессор ждет загрузки из ОЗУ, что резко снижает производительность

Способы оптимизации:

1. Array of Structures (AoS) — Массив структур. Традиционный объектно-ориентированный подход: массив объектов, где каждый объект содержит все свои поля.

Удобно при работе со всеми полями одного объекта одновременно.

Но если нужно обновить только одно поле для всех, в кеш загружаются ненужные остальные, забывая его

2. Structure of Arrays (SoA) — Структура массивов. Подход, ориентированный на данные - структура содержит отдельные массивы для каждого поля.

Максимальная производительность при линейном обходе. В кеш загружаются только нужные данные, что увеличивает интенсивность попаданий.

Но менее удобно для работы с конкретным объектом, сложнее код.

3. Блокировка. Разбиение больших матриц/массивов на блоки, которые целиком помещаются в кеш, чтобы избежать повторного обращения к ОЗУ.

188. Как кеш работает с виртуальными и физическими адресами?

Есть момент с тем, что в реальных системах на процессорах запускается несколько задач-процессов. Чаще всего у каждого процесса своя виртуальная память, и ответственна каждый процесс запрашивает данные в виртуальной адресации, которая затем уже транслируется в физическую (что занимает время).

Так вот вопрос - кешу следует опираться на виртуальный адрес (это будет быстрее, но между процессами будет коллизия адресов (разные виртуальные адреса указывают на один физический), а значит при переключении между процессами надо будет его сбрасывать, ведь у каждого своя память) или все-таки на физический (но нам надо будет тратить время на трансляцию виртуального адреса в физический, а кеш хочется быстрый)?

Решается компромиссом - очень быстрый кеш (L1) делают на виртуальных адресах, а уже другие уровни (L2/L3), которым не так критична скорость, опираются на настоящий физический адрес

189. Что такое HyperThreading? Как два логических ядра делят физический кеш и как это влияет на производительность?

HyperThreading - подход который не дает настоящих нескольких процессов, но за счет двух наборов основных регистров позволяет одному физическому ядру обрабатывать два потока вычислений одновременно, переключаясь между ними за счет отдельных комплектов регистров. Переключение например может происходить в момент cache-miss на одном ядре и пока ждем ответа переключаемся на другое ядро.

Такие логические ядра делят непосредственно одни и те же кешы, поэтому если оба потока словят cache-miss на производительности это скажется плохо - физическое ядро будет простаивать. Но в среднем HyperThreading по сравнению с одним ядром позволяет выполнять несколько больше инструкций и "имитировать" два ядра гораздо дешевле, чем действительно построить два ядра

190. В чём суть уязвимостей Meltdown и Spectre? Как кеш и спекулятивное исполнение создают канал утечки данных?

Обе уязвимости основываются на том, что мы начинаем выполнять инструкции, которые не должны быть выполнены. Например потому что у нас длинный конвейер, или потому что branch-predictor сказал, что надо идти туда

Meltdown: позволяет прочитать данные, которые запрещены для чтения данному процессу.

Spectre: основан на злоупотреблении branch predictor, заставляя его ошибиться в предсказании перехода и выполнить то, что выполнять было нельзя.

Ключевая проблема в том, что хотя результаты расчетов отбрасываются, данные, загруженные в кэш-память процессора во время спекуляции, не удаляются оттуда.

15 ЛЕКЦИЯ

191. Что представляет собой концепция уровневой организации компьютерных систем и каковы её преимущества?

Уровневая организация - способ организации вычислительных систем, где система разделена на группы компонентов (называемых уровнями), которые имеют некоторую иерархию и относительную независимость друг от друга.

Каждый уровень решает конкретную задачу, предоставляя конкретный интерфейс для других уровней, скрывая внутренние детали реализации.

Это позволяет разделять команды разработчиков, снижая общую сложность разработки, делает системы модульными и вариативными, позволяя заменять реализацию некоторых уровней целиком

192. Какие примеры уровневой организации вычислительных систем вы знаете? Чем они отличаются друг от друга?

Lava Flow - антипаттерн проектирования (как делать не надо), когда проект можно представить как извергающийся вулкан. Т.е. во время разработки проекта вулкан периодически извергается (приходит новый разработчик, появляется новое требование, меняется подход), далее из него изливается лава (пишется новый код там и так далее) вот только вся эта лава (и вся работа над проектом) накладывается поверх предыдущих, уже окаменевших слоев. Никто не подвергает сомнениям старый код, никто его не рефакторит, просто фигачат сверху больше и больше.

Layered style (уровневый архитектурный стиль) - такой подход к организации компьютерной системы, который строится на том, что мы выделяем в нашей системе отдельные слои. Каждый слой предоставляет набор сервисов, модулей, набор компонентов, которые можно вызывать на вышестоящем уровне.

Вот еще пара приколов про уровневый архитектурный стиль:

Модифицируемость и переносимость

Управление сложностью и структурой кода разработчиком

Разделение интересов

OSI - идея в том, что есть 7 уровней, через которые должен пройти любой сетевой пакет, от приложения до физического уровня. Уровни эти чтобы обеспечить модульность и вариативность:

1. Application Layer (прикладной)

2. Presentation Layer (уровень представления)
3. Session Layer (сеансовый)
4. Transport Layer (транспортный)
5. Network Layer (сетевой)
6. Data Link Layer (канальный)
7. Physical Layer (физический)

Уровни организации вычислительного процесса: внизу полупроводники, цифровая схемотехника, узлы процессора, его управляющие сигналы и микроархитектура, система команд процессора, программный код (наша 3 лаба), языки высокого уровня и модели вычисления

193. Что такое явление разделения на уровни (disaggregation) и каково его современное положение? Что было до него?

Изначально каждая коммерческая компания стремилась покрыть все уровни:

Application Software	Программы
Операционные системы	Драйверы устройств
Архитектура	Регистры инструкций
Микро - архитектура	Контроллеры каналы передачи данных
Логика	Сумматоры память
Цифровые схемы	AND вентили NOT вентили
Аналоговые схемы	Усилители фильтры
Устройства	Транзисторы Диоды
Физика	Электроны

Они разрабатывали свои процессоры, свои чипы, собирали свои устройства, писали под них свои ОС, под эти ОС писали свое ПО.

Затем произошла реорганизация всего этого (disaggregation) и сейчас уже очень мало компаний, которые покрывают настолько много уровней. По большей части сейчас у нас есть компании, которые производят конкретно процессоры и

специализируются на этом. Есть компании, которые пишут и распространяют свои ОС.

Это огромный шаг навстречу прогрессу, потому что позволяет каждому заниматься своим делом и делать его хорошо. При этом на процессорах разных производителей мы можем запустить одну и ту же ОС, на разных ОС можем запустить одно и то же ПО.

Все это позитивные последствия разделения на уровни.

194. Почему необходимо нарушение уровневой иерархии в современных системах? Приведите примеры.

Разделение на уровни – это классно ровно до того момента, пока у нас не появляются строгие требования реального времени, или мы становимся очень чувствительны к скорости выполнения или энергопотреблению, стремимся оптимизировать. В таких задачах мы не можем игнорировать внутренние уровни

Чтобы обеспечить выполнение этих наших задач мы вынуждены нарушать уровневую иерархию разными способами. Начиная от того, что на верхнем уровне языка мы задумываемся о том как работают нижние уровни, например кеши. Заканчивая тем, что мы реорганизовываем структуры данных, вмешиваемся в нижние уровни и все такое.

195. Какие проблемы возникают при обеспечении реального времени в современных процессорах?

При обеспечении задач реального времени в современных процессорах возникает куча проблем: спекулятивное исполнение, кеш-память, виртуальная память – все это призвано улучшить производительность, но на практике порождает проблему, когда мы не можем понять, сколько именно будет выполняться конкретная инструкция, сколько именно будет длиться реакция системы на воздействие. Поэтому нам приходится как-то обходить эти все штуки на программном уровне

196. Что такое низкоуровневый параллелизм? Приведите примеры.

Низкоуровневый параллелизм - параллелизм, который достигается внутри процессора на уровне его микроархитектуры. Под низкоуровневым параллелизмом мы имеем в виду те виды параллелизма, которые являются прозрачными для программиста. То есть не тогда, когда программист огромным количеством усилий пытается обеспечить параллельные вычисления, а когда это можно получить относительно "бесплатно".

Примеры организации такого параллелизма:

Конвейеризация то, что рассматривалось подробно ранее. Когда инструкция разбивается на несколько этапов, а разные этапы могут выполняться параллельно

Множественные функциональные узлы, когда у нас есть независимые функциональные блоки для разных операций и мы можем выполнять их одновременно:

- Суперскалярный процессор
- Very Long Instruction Word

197. Что такое суперскалярные процессоры и каковы принципы их работы, достоинства и недостатки?

Суперскалярные процессоры — это тип процессоров, которые поддерживают параллелизм на уровне инструкций, позволяя выполнять несколько команд одновременно в течение одного такта. Они достигают этого за счет включения нескольких функциональных блоков (например, нескольких АЛУ) в состав одного процессорного ядра.

Принципы работы суперскалярных процессоров

1. Процессор анализирует поток команд и выполняет одновременно те, которые не зависят друг от друга.
2. В ядре есть дублированные блоки: АЛУ (ALU), блоки с плавающей запятой (FPU), блоки загрузки/записи (load/store).
3. Процессор «на лету» переупорядочивает команды, отправляя их на выполнение не по очереди, а по мере готовности необходимых данных
4. Спекулятивное выполнение — процессор «угадывает», по какой ветке пойдет программа после условия, и выполняет инструкции заранее, чтобы не простаивать.

Достоинства

- Высокая производительность: Способность выполнять более одной инструкции за такт значительно увеличивает скорость работы.
- Эффективное использование ресурсов: Несколько блоков выполнения позволяют минимизировать простои.
- Программная совместимость: Процессор сам распараллеливает последовательный код, не требуя специальной перекомпиляции.

Недостатки

- Сложность проектирования
- Высокое энергопотребление: Большое количество исполнительных блоков потребляют много энергии

198. Как пошагово выполняются инструкции в суперскалярном процессоре?

1. Выборка (Fetch). На этом этапе процессор выбирает не одну, а группу инструкций за один такт. Благодаря предсказателю переходов (branch predictor) процессор пытается угадать направление перехода (ветвления) и загрузить инструкции заранее, чтобы не простаивать.
2. Декодирование и распределение (Decode & Dispatch). Выбранные инструкции переводятся в микрооперации. Микрооперации направляются в очередь инструкций

3. Очередь инструкций (Instruction Queue) Инструкции ожидают в очереди, пока не будут готовы их исходные данные (операнды). Инструкции не обязательно выполняются в том порядке, в котором они были написаны в программе.

4. Исполнение (Execute). Инструкции, у которых готовы данные, отправляются на исполнительные блоки. Поскольку процессор суперскалярный, несколько инструкций могут выполняться одновременно на разных блоках.

5. Завершение (Commit). После выполнения инструкции результат записывается во временный буфер. Окончательная запись в регистры или память происходит только тогда, когда инструкция успешно прошла стадию выполнения и все предыдущие инструкции также были выполнены. Это гарантирует правильность выполнения программы.

199. Что такое VLIW-процессоры? Какими достоинствами и недостатками они обладают?

VLIW (Very Long Instruction Word) — это архитектура процессоров, в которой компилятор заранее планирует, какие операции будут выполняться параллельно, и объединяет их в одну сверхдлинную команду.

В отличие от суперскалярных процессоров, которые сами решают, какие инструкции запустить одновременно, VLIW-процессор доверяет эту задачу компилятору, что упрощает «железо». В одной инструкции VLIW может содержаться 4–8 и более операций (арифметика, работа с памятью, переходы), выполняемых одновременно.

Достоинства VLIW-процессоров

- Упрощенная аппаратная архитектура.
- Высокая энергоэффективность: благодаря простоте, процессоры потребляют меньше энергии.
- Упрощение декодера. Декодеру не нужно на лету анализировать зависимости. Он берет готовое длинное слово и сразу раскидывает его микрокоманды по исполнительным блокам.
- Компилятор видит всю программу целиком на этапе сборки. У него много времени, чтобы проанализировать код, найти независимые инструкции и идеально распараллелить их.

Недостатки VLIW-процессоров

- Зависимость от компилятора: если компилятор слабый, процессор будет простаивать.
- Низкая плотность кода: если не удастся найти достаточно параллельных операций, «пустые» места в длинной команде заполняются инструкциями NOP (нет операции), что увеличивает размер программы.
- Отсутствие совместимости: Программы, скомпилированные для одной версии VLIW-процессора, часто не работают или работают медленно на другой (из-за разного количества функциональных блоков).

200. Как соотносятся суперскалярные и VLIW процессоры применительно к разным классам задач?

Суперскаляр лучше для задач, где много переходов. Потому что суперскалярный процессор гораздо лучше с этим справляется и может подмешивать разные инструкции разные потоки и все такое. Он обратно совместим, ассемблерный код может запускаться на любом процессоре, который поддерживает ISA, на котором написан этот код

VLIW лучше всего работает в узкоспециализированных задачах, например где надо делать кучу однотипных вычислений. Эти процессоры более энергоэффективны, более предсказуемы и плотнее загружают вычислительные мощности процессора

201. Какие существуют промежуточные варианты между суперскалярным и VLIW процессором? Какие проблемы они решают?

Если смотреть чуть более общим взглядом, то суперскаляр и VLIW это всего лишь крайности, где в одном случае мы поток инструкций в рантайме группируем и пытаемся пережевать, а в другом у нас уже на уровне инструкций все объединено и в рантайме нам надо только выполнять инструкции вообще не задумываясь.

На самом деле можно разбить код на 4 этапа обработки. Любой код должен их пройти, вопрос только в том, что что-то можно делать на этапе компиляции, а что-то делать в самом процессоре во время работы. Вот именно по тому, где какие этапы выполняются и получаются промежуточные архитектуры:

Четыре шага обработки кода

Любой параллельный запуск инструкций требует выполнения четырех шагов:

1. **Code generation:** Генерация базового машинного кода.
2. **Instruction grouping:** Поиск независимых команд и объединение их в группы для параллельного запуска.
3. **Fn. unit assignment:** Привязка конкретной команды к конкретному исполнителю (например, эту — в ALU №1, эту — в FPU №2).
4. **Initiation timing:** Определение точного такта (времени), когда эту команду нужно физически выстрелить в конвейер.

Из этого получаем еще две промежуточные архитектуры:

1. Superscalar (Обычный суперскаляр — x86, ARM)

- **Как работает:** Компилятор делает только первый шаг (`Code generation`). Он просто выдает плоский последовательный поток команд.
- **Железо делает ВСЁ остальное:** Сами кремниевые схемы на лету группируют инструкции, распределяют их по исполнительным блокам (порты ALU/FPU) и решают, в какой такт их запустить (OutOfOrder).
- **Итог:** Железо сложное и горячее, софт простой.

2. EPIC (Explicitly Parallel Instruction Computing — Intel Itanium)

- **Как работает:** Компилятор делает первые два шага. Он генерирует код и **явно группирует** независимые команды в блоки (bundles), подсказывая процессору: *«эти три штуки можно запустить одновременно»*.
- **Железо делает остальное:** Процессор берет эти группы и сам решает, на какие конкретно внутренние ALU их раскидать и в какой микротакт запустить.
- **Итог:** Попытка найти компромисс между VLIW и Суперскаляром.

3. Dynamic VLIW

- **Как работает:** Компилятор берет на себя уже три шага. Он не только группирует команды, но и жестко прописывает, в какой функциональный блок (`Fn. unit assignment`) должна пойти каждая инструкция.
- **Железо делает минимум:** Железо отвечает только за тайминг (`Initiation timing`) — проверяет, не случилось ли задержки памяти (cache-miss), и вовремя дает отмашку на старт.

4. VLIW (Чистый VLIW — Эльбрус, DSP)

- **Как работает:** Компилятор делает **абсолютно всё** (все 4 шага). В кодовой строке (длинном слове) уже жестко зашито: в каком такте, на каком конкретно ALU и какие три команды будут выполнены одновременно.
- **Железо делает НИЧЕГО:** Процессор превращается в «глупый исполнитель». Он просто берет готовые строчки-команды и без всякого анализа раскидывает их по кремниевым блокам. Если компилятор ошибся с таймингом — процессор зависнет.

Важно отметить, что из-за сложности компиляторов и не очень большого роста в производительности EPIC процессоры не получили большого распространения и развития

Еще момент - отличие VLIW и Dynamic VLIW в том, что во VLIW в коде уже четко задано в какой момент он должен какие инструкции на каких АЛУ запустить, а в Dynamic VLIW так - если на каком-то АЛУ инструкции выполняются быстрее чем других, то он не простаивает, а идет дальше по своему потоку инструкций, которые назначены на этот АЛУ.

202. Что такое барьеры памяти и в каких ситуациях они применяются?

Барьер памяти - специальная инструкция, которая предписывает компилятору (при генерации инструкций) и процессору (при исполнении инструкций) не смешивать обращения к памяти. Чтобы обращения после барьера никогда не выполнялись раньше обращений до барьера. Применяется когда нам важна последовательность обращений к памяти (например, барьер по чтению может означать, что все операции чтения, которые были по тексту программы до барьера, должны произойти до него)

Актуально для процессоров, которые могут перемешивать выполнение инструкций (например, для суперскалярных процессоров) и для компиляторов, которые тоже иногда для оптимизации могут переставлять инструкции (чтобы избежать пузырьков конвейера, к примеру)

203. Как суперскалярное исполнение и store buffers влияют на порядок обращений к памяти в многоядерных системах? Что такое барьер памяти (fence) и зачем он нужен?

Суперскалярное исполнение подразумевает несколько исполнительных блоков, умеющих выполнять инструкции. Поскольку поток один, а исполнителей несколько, этот поток распределяется на них, а как именно - только самому процессору известно и выполняются соответственно в хаотичном порядке. Поэтому инструкции чтения могут быть выполнены в произвольном порядке.

Store buffers (буфер между АЛУ's и памятью/кешем) выравнивают эту историю, записывая результаты вычислений в память ровно в том порядке, в котором они шли в исходном потоке. Так что инструкции записи выполняются в том порядке в котором они шли изначально.

И здесь могут возникать проблемы, потому что порядок этих всех обращений/чтений/записей относительно друг друга может быть произвольным вообще

Барьер памяти - специальная инструкция, которая предписывает компилятору (при генерации инструкций) и процессору (при исполнении инструкций) не смешивать обращения к памяти. Чтобы обращения после барьера никогда не выполнялись раньше обращений до барьера. Применяется когда нам важна последовательность обращений к памяти (например, сначала записали новые данные, и только потом установили флаг готовности новых данных)

204. Какие ключевые элементы составляют структурное программирование? В чём отличие от кода на ассемблере?

Язык высокого уровня — это когда мы переходим от инструкций и джампов, ну т.е. вообще от абстракций процессора к структурным блокам и их последовательностям. В особенности так реализован язык Си, в основе которого заложена идея, что любую программу можно представить как структурные блоки.

Их три типа:

Последовательный - просто некоторая совокупность операций

Условный - в зависимости от условия выполняем или одно или другое

Циклический - выполняем некоторый блок пока есть некоторое условие

Ну и можно вызвать отдельные блоки (попадаем во внешний блок).

205. Как реализуются процедуры на уровне ассемблера? Что такое call/return и зачем нужен стек вызовов?

При написании кода нам так или иначе нужны процедуры. Зачем:

с целью переиспользования машинного кода

для оптимизации кеша (инструкции не дублируются по разным адресам в разных местах программы, а лежат все компактно в одном месте)

Есть как минимум три подхода к реализации процедур:

`inline` - мы просто вместо вызова процедуры вставляем все инструкции этой процедуры и все. Да, у нас дублируется код, но формально "вызов" процедуры осуществлен

`goto` - путем инструкций перехода осуществляем вызов и возврат

`call/return` - аппаратная поддержка процедур с сохранением адреса возврата в стек вызовов (он нужен для реализации вложенности и реентерабельности)

206. Что такое реентерабельность и почему она важна? Приведите примеры.

Реентерабельность — это свойство процедуры, позволяющее ей быть вызванной повторно до того как завершился предыдущий ее вызов. Т.е. если мы можем безопасно прервать ее выполнение ее же вызовом.

Рекурсия например активно использует это свойство, когда функция вызывает сама себя. Либо прерывание, которое может прерывать собственный обработчик.

207. Как реализуется механизм рекурсии?

Механизм рекурсии реализуется за счет того, что функция вызывает сама себя, а операционная система использует структуру данных «стек» для хранения информации о каждом вызове. Рекурсивный процесс состоит из двух основных частей: «прямого хода» (спуск), когда задача дробится до достижения базового случая, и «обратного хода» (возврат), когда происходит вычисление результатов на основе накопленных данных.

Ключевые элементы реализации рекурсии:

- **Стек вызовов (Call Stack):** при каждом рекурсивном вызове в памяти создается новый кадр стека (stack frame). В нем хранятся локальные переменные, параметры функции и адрес возврата. Когда функция завершается, ее кадр удаляется из стека.
- **Базовый случай:** Условие завершения рекурсии. Это простейшая версия задачи, которая решается без очередного вызова функции. Без него возникает бесконечная рекурсия, приводящая к переполнению стека (stack overflow).
- **Прямой и обратный ход**

208. В чём заключается проблема распределения регистров и как работает алгоритм раскраски?

Эта проблема возникает на этапе компиляции, когда язык высокого уровня нам надо перевести в инструкции. И нам бы очень хотелось, чтобы код выполнялся быстро, в частности с минимальным числом обращений в память. Т.е. было бы классно, чтобы у нас все крутилось и вычислялось в регистрах. Но регистров у нас не всегда так много, как хотелось бы и более того не всегда они одинаковы, значит надо как-то распределять этот ограниченный ресурс.

Один из возможных подходов — это раскраска регистров. Идея вот в чем: наша задача отобразить в виде графа все регистры, которые есть в нашей программе, после этого наша задача ребрами связать те регистры которые должны присутствовать в памяти одновременно и после этого предложить такую раскраску этих вершин, чтобы ни один цвет не был в соседних вершинах. Если удалось добиться такой раскраски, то если количество цветов $\leq$ количество регистров, то наша команда может эффективно работать.

209. Как условный оператор может быть реализован через полиморфизм и замыкания?

В Smalltalk условный оператор if-else полностью удален из синтаксиса языка. Вместо него логика ветвления реализуется за счет динамического вызова методов и упаковки кода в анонимные функции.

Классы вместо булевых типов: Истина (True) и Ложь (False) — это два самостоятельных класса-наследника, унаследованных от общего абстрактного класса Bool.

Полиморфные методы: В обоих классах определен метод ifTrue:ifFalse:, но их внутренняя реализация отличается:

В классе True метод жестко запрограммирован выполнить только первый переданный блок кода (замыкание) и проигнорировать второй.

В классе False метод наоборот — выполняет только второй блок кода.

Роль замыканий: Код каждой ветки («тогда» и «иначе») оборачивается в анонимные функции, чтобы изолировать инструкции и не выполнять их раньше времени.

210. Что такое ленивые вычисления и как они связаны с реализацией условного оператора?

Ленивые вычисления (Lazy Evaluation) – это стратегия вычисления, при которой выражение вычисляется не в момент его создания, а только тогда, когда его результат реально потребовался для работы программы. Например не вычислять аргумент функции, пока к нему не обратились внутри функции

Условный оператор по своей сути всегда должен быть ленивым, потому что выполняется или одна ветка или вторая. Нельзя пройти по обеим веткам, а оставить только результат одной из них. Поэтому без ленивых вычислений условный оператор теряет смысл

211. Как async/await связан с кооперативной многозадачностью на уровне языка программирования?

Механизм async / await — это синтаксический инструмент языка программирования, созданный специально для реализации кооперативной многозадачности.

Главный принцип кооперативной многозадачности — добровольная передача управления. В отличие от вытесняющей многозадачности (где планировщик жестко прерывает поток в любой такт времени), в кооперативной модели задача работает до тех пор, пока сама не скажет, что отдает процессорное время другим». Задача обязана сама передать управление через await

Кооперативность означает, что если async-функция начнет тяжелые вычисления, не используя await (не отдавая управление), она "заморозит" весь Event Loop и другие задачи не выполнятся

16 ЛЕКЦИЯ

212. Что такое FPGA и каковы его преимущества и недостатки?

FPGA - он же ПЛИС - программируемая логическая интегральная схема – это вычислитель, являющийся промежуточным решением между ASIC и CPU

В отличие от обычных цифровых микросхем, логика работы ПЛИС не определяется при изготовлении. Для формирования конфигурации используется специальное ПО и Hardware Description Language (специальные языки аппаратуры): Verilog, VHDL

В чем концепция - сри это супер универсальный вычислитель, реализующий временные вычисления. Он относительно медленный, содержит много логики для интерпретации и выполнения команд и вычислительный процесс в нем представлен как последовательность команд. Это одна крайность. Другая крайность это asic - супер специализированная под задачу огромная комбинационная схема, за счет того, что ей не надо поддерживать универсальность она может быть реализована супер эффективно.

Но подход asic это очень сложно, схему надо спроектировать, произвести и это все нетипичное и это все под заказ. Поэтому есть компромисс - fpga. Это такая схема, которая может быть сконфигурирована под задачу, затем реконфигурирована и поэтому нам не надо производить ничего с нуля. Покупаем готовый типовой чип, настраиваем его под работу с конкретной задачей и радуемся как быстро все выполняется (чуть медленнее все же чем asic но СИЛЬНО быстрее сри)

Преимущества - быстрее сри на порядки, на порядки же дешевле asic (для небольших партий)

Недостатки - если запускать огромное производство на 100500 миллионов штук, в пересчете на штуку дешевле все-таки выйдет asic чем fpga.

213. Какие этапы включает в себя синтез для FPGA?

Пишем на языке описания аппаратуры, где описываем картинку нашей схемы. Потом она транслируется в принципиальную электрическую схему (где нас не волнуют тайминги, длины проводов, нас интересует только то, что мы описали корректную схему). После чего вся эта конструкция транслируется в конкретный вычислительный базис аппаратуры, на которой может быть запущен.

214. Как FPGA устроены внутри?

ПЛИС состоит из множества маленьких логических блоков, собранных в матрицы. Каждый блок умеет реализовывать небольшие таблицы истинности. Блоки мы можем произвольно соединять между собой. Также есть блоки памяти (RAM) и подобие регистров + специальные блоки под частые задачи (например сумматоры, делители), чтобы не тратить на них кучу блоков. Каждый логический блок мы конечно можем настроить, загрузив туда нужную таблицу истинности

215. Что такое HDL и RTL? Чем описание аппаратуры на HDL отличается от программирования на языках высокого уровня?

HDL: Это специализированный язык программирования (самые популярные - Verilog и VHDL), который используется для моделирования, проектирования и верификации цифровых электронных схем (например, для FPGA или ASIC).

RTL: это стиль (или уровень абстракции) написания кода на языке HDL. На уровне RTL разработчик явно описывает схему как совокупность регистров (триггеров для хранения состояния) и комбинационной логики (логических вентилях И, ИЛИ, НЕ), которая преобразует данные при их пересылке между этими регистрами от такта к такту.

Основные отличия:

1. В программировании параллельность нужно специально создавать (поток, процессы). В HDL все процессы работают параллельно «по умолчанию».
2. В HDL вы описываете, *как схема должна выглядеть*, а не то, *что она должна делать*.

216. Как соотносятся CPU и FPGA с точки зрения модели программирования, модели вычислений, модели исполнения и элементов микроархитектуры?

1. Модель программирования.

- CPU — Императивное программирование: Код пишется на классических языках (C/C++). Программа представляет собой последовательность инструкций, пошагово меняющих состояние памяти. Мы явно описываем как и в каком порядке процессор должен выполнять шаги.

- FPGA - Декларативное программирование: Код пишется на языках описания аппаратуры (VHDL/Verilog). Разработчик описывает саму цифровую схему: как физически соединены между собой провода, регистры и логические элементы.

2. Модель вычислений.

- CPU — Временные вычисления: Одно АЛУ выполняет разные задачи последовательно во времени. Сначала АЛУ считает одну операцию, в следующем такте на этом же АЛУ считается другая операция.
- FPGA - Пространственные вычисления: Каждая операция из алгоритма получает свой собственный физический кусок железа в пространстве чипа. Все эти вычислительные узлы работают одновременно и параллельно.

3. Модель исполнения.

- CPU — Динамическое планирование и последовательное исполнение: На вход подается один последовательный поток команд. Процессор внутри себя на ходу (динамически) решает, как распределить команды по конвейеру, пытается угадать переходы (Branch Prediction).
- FPGA — Статическое планирование и потоковое исполнение: Никакой динамики на ходу нет. Вся схема жестко спланирована заранее, еще на этапе компиляции. Данные поступают на входы и волной идут через логические элементы

4. Микроархитектура.

- CPU: Состоит из универсальных фиксированных блоков: ALU. Register File (регистровые файлы для хранения промежуточных данных). Branch Predictors. Cache
- FPGA: Состоит из огромного массива мелких настраиваемых блоков: CLB (Configurable Logic Blocks) - те самые блоки, внутри которых сидят таблицы истинности (LUT), триггеры и мультиплексоры. Switch box — ключи, которые соединяют CLB между собой. Block memory (BRAM) - встроенные мелкие блоки памяти.

217. Что представляет собой классификация Флинна и какие классы архитектур она выделяет? Примеры.

Классификация Флинна - попытка классифицировать параллельные процессоры.

Каждому процессору сопоставляется два параметра:

Количество потоков инструкций

Количество потоков данных

Выделяются 4 группы: Single/Multiple Instruction Simple/Multiple Data (соответственно SISD, SIMD, MISD, MIMD)

SISD – простой последовательный процессор. Пример: Фон-Нейман, CPU

SIMD - когда одна операция запускается параллельно сразу для нескольких наборов данных. Подходит для однотипной обработки: для нейросетей, для графики, для матриц. Пример: GPU

MISD - не получил широкого распространения, потому что очень мало есть задач когда для одних данных нужно посчитать несколько разных инструкций. Мооожет применяться в системах с супер требованием к надежности управления, когда одни данные прогоняются через несколько инструкций

MIMD - практически все процессора попадают сюда, они могут обрабатывать много потоков инструкций и у каждой свои входы по данным. Такие сильные и независимые отдельные блоки процессорные. Пример: Intel, Ryzen

218. Что такое SIMD и SIMT архитектуры? В чём их различия? Что такое lockstep execution?

SIMD (Single Instruction, Multiply Data): одновременное выполнение несколькими наборами данных одной инструкции. Назначение – однообразная обработка множества наборов данных. Ключевые ограничения: операции ветвления

SIMT (Single Instruction Multiply Threads): расширение SIMD. Широко применяется в современных GPU. Включает механизмы синхронизации потоков между собой для достижения lockstep execution. Одна инструкция посылается не напрямую в векторный блок (как в SIMD), а множеству отдельных потоков (threads), которые выполняют её синхронно. Эти потоки объединены в группы warps. Если потоки в одном «warp» идут по разным путям if-else, они выполняются последовательно, что значительно замедляет работу

Lockstep execution - много потоков данных, они работают строго синхронно и выполняют одно действие

219. Что представляет собой классификация Дункана и чем она отличается от классификации Флинна? Какие проблемы она решает? Что включает верхний уровень классификации?

Дункан решил что низкоуровневый параллелизм никому не интересен, потому что внешне оно выглядит будто инструкции выполняются линейно. При этом базировался он все равно на Флинне, но расширил и включил новшества в отрасли.

Разбил MIMD на составные части. Это решает проблему излишней простоты классификации Флинна, которая не погружается глубоко в детали реализации процессоров.

Он делит архитектуры на три большие группы (это верхний уровень классификации):

Синхронные архитектуры. Параллельные архитектуры с единым механизмом управления

MIMD архитектуры. Параллельные архитектуры, которые могут исполнять независимые потоки инструкций. Потоки инструкций исполняются автономно, требуется динамическая синхронизация.

MIMD парадигма. Архитектура MIMD процессора определяется моделью вычислений. МоС определяет характер потоков данных, принципы взаимодействия и синхронизации.

220. Какие основные типы синхронных архитектур выделяются по классификации Дункана?

Векторные архитектуры предназначены для высокоэффективной обработки больших массивов данных (векторов) с использованием одной команды. В основе их работы лежат принципы параллелизма и конвейеризации, что позволяет выполнять одну и ту же операцию над множеством элементов данных одновременно

SIMD (процессорные и ассоциативные массивы) - все процессорные элементы выполняют одну инструкцию одновременно в тактовом режиме. Ассоциативные процессоры работают на ассоциативной памяти, которая может быстро искать данные по определенным параметрам

Систолические архитектуры – у нас есть вычислительные блоки, которые по внешнему общему такту друг с другом общаются, передают по цепочке значения, и так у нас распространяются вычисления. Минус в том что это не для каждой задачи так просто сделать. Зато для умножения матриц это сууупер эффективно.

Совмещение памяти и вычислений - когда мы решаем, что упираться в скорость доступа к памяти это грустно и мы прямо в память вшиваем вычислители, которые прямо в памяти по команде процессора будут делать какие-то простые вычисления. Например, при перемножении матриц нам не надо будет тыщу раз ходить туда обратно в память.

221. Какие принципы лежат в основе работы векторных архитектур и где они применяются?

См. билет 220

Применяют в системах, где очень актуальны векторные операции, для ускорения графики, обработки сигналов, спектрального анализа

222. Как работают ассоциативные массивы и где они применяются?

См. билет 220

Применяют в сетевом оборудовании (задача поиска в таблицах маршрутизации там очень актуальна)

223. Как работают систолические архитектуры и где они применяются?

См. билет 220

Применяют для получения аппаратного очень быстрого перемножения матриц (задачи машинного обучения, цифровая обработка изображений и видео)

224. Какие основные типы MIMD-архитектур выделяются по классификации Дункана?

Разделяемые - в этих системах все процессоры имеют доступ к единому общему адресному пространству.

Распределенные – в этих системах каждый процессор имеет собственную локальную память, недоступную напрямую другим процессорам. Взаимодействие происходит через передачу сообщений

225. Что из себя представляют архитектуры с распределённой памятью? Каковы их особенности?

У каждого процессора своя отдельная память, а общаются они явно через сообщения между процессорами. Это кратно все усложняет, потому что надо выстраивать всю эту коммуникацию между ними, заморачиваться с протоколами и сообщениями. А еще есть огромное количество топологий подобной сети процессоров, у каждой свои особенности. Зато здесь можно обеспечить масштабирование, процессор никогда не конкурирует за память

226. Что из себя представляют архитектуры с разделяемой памятью? Каковы их особенности?

Память общая, у каждого процессора может быть свой кеш, но все вместе они делят одно общее пространство памяти. Соответственно возникает куча проблем того, что процессоры начинают конкурировать за память, кеши надо как-то согласовывать. Однако в такой системе проще координировать работу процессоров потому, что они могут общаться через память. А еще это удобно для программистов

227. Какие основные типы MIMD парадигм выделяются по классификации Дункана?

Выделяются следующие основные парадигмы и архитектуры:

1. Гибридные MIMD/SIMD архитектуры: Системы, способные реконфигурироваться, разделяя процессоры на группы, работающие в режиме SIMD.
2. Архитектуры, управляемые потоками данных (Dataflow): Вычисления инициируются готовностью данных, а не последовательностью команд.
3. Редукционные архитектуры (Reduction): Основаны на принципе уменьшения выражений (редукции).
4. Волновые архитектуры (Wavefront): Специализированные структуры для специфических вычислений.

228. Что представляет собой MIMD/SIMD парадигма и где она применяется?

Система состоит из нескольких независимых процессоров, работающих по принципу MIMD (каждый выполняет свою программу), при этом каждый из этих процессоров или отдельные специализированные блоки внутри них работают по

принципу SIMD (одна инструкция обрабатывает несколько данных одновременно).

MIMD-уровень обеспечивает гибкость, позволяя системе выполнять разные задачи параллельно, а SIMD-уровень ускоряет векторные и матричные вычисления, обрабатывая большие массивы данных за один такт.

Гибридные системы MIMD/SIMD применяются там, где требуется высокая производительность при обработке больших массивов данных, а также гибкость управления вычислительными процессами:

Обработка изображений и видео

Искусственный интеллект и машинное обучение

229. Как работает архитектура потоков данных (dataflow)? Подходы к реализации.

Вместо центрального счетчика команд узлы вычислений активируются, когда все необходимые входные данные для них готовы.

Как работает Dataflow-архитектура

1. Программа представляется в виде направленного графа, где узлы — это функции (операции), а дуги — пути передачи данных.
2. Операция запускается только тогда, когда на всех ее входах присутствуют операнды.
3. Если несколько узлов получили данные одновременно, они выполняются параллельно, что обеспечивает высокую производительность.
4. Данные передаются от узла к узлу

Аппаратные подходы:

- Статический Dataflow: на дугах графа может находиться только один элемент данных в один момент времени. Подходит для простых, предсказуемых вычислений.
- Динамический Dataflow: позволяет хранить несколько данных на дуге, пометая их тегами (tags) для различения, например, итераций цикла. Это сложнее, но эффективнее для сложных алгоритмов.

230. Как работает редукционная архитектура? Каким образом она обеспечивает параллелизм?

Как работает редукционная архитектура

1. Программа представляется в виде графа, где узлы — это функции, а связи — аргументы.
2. Вычислительные элементы (процессоры) ищут «красные» узлы, которые можно вычислить
3. Выражения вычисляются только тогда, когда их значение действительно необходимо для получения конечного результата. Это позволяет избежать лишних расчетов (ленивые вычисления)

4. После вычисления узла он заменяется на результат, что сокращает граф. Процесс продолжается, пока граф не превратится в единственное значение.

Параллелизм достигается за счет:

- Узлы графа, не зависящие друг от друга, могут вычисляться одновременно на разных процессорах.
- Древовидная структура

231. Как работает wavefront архитектура? В чём её отличия от систолической архитектуры?

Wavefront-архитектура представляет собой двумерный массив процессорных элементов, которые работают асинхронно, обмениваясь данными только с ближайшими соседями

Основные отличия заключаются в следующем:

1. Синхронизация и время

- Систолическая архитектура: Синхронная. Данные перемещаются по массиву процессоров в строгом темпе
- Wavefront: Асинхронная. Элементы не привязаны к жесткому глобальному такту. Каждый элемент активируется, как только получит данные от соседа.

2. Управление потоком данных

- Систолическая архитектура: Данные передаются от соседа к соседу. Часто они используются для конвейерной обработки регулярных задач (матричное умножение).
- Wavefront: Работает по принципу dataflow (поток данных). Вычисления происходят, когда «волна» данных доходит до ячейки.

232. Что такое CGRA-процессоры и какие возможности они предоставляют?

CGRA - семейство процессорных архитектур, находящееся посередине между FPGA и DSP процессорами. В CGRA единицей конфигурирования является более крупный вычислительный блок (например: целый АЛУ или небольшой функциональный узел). Идея состоит в том, чтобы адаптировать процессор под конкретную задачу на уровне конфигурации: настроить доступные функциональные блоки и их межсоединения для эффективного выполнения определённого класса алгоритмов, сохраняя при этом программируемость.

Гибридная организация пространственного и временного параллелизма – ключевое свойство CGRA, позволяющее эффективно ускорять алгоритмы, сочетающие независимые параллельные участки и последовательные ветви исполнения.

Отличие от ПЛИС в том, что ПЛИС заточен под реализацию примерно любой цифровой схемы, а здесь более крупноячеистая структура с большими блоками,

заточенными под какие-то прикладные задачи. Более того, можно комбинировать подход ПЛИС, который занимается пространственными вычислениями, и подход CPU, который занимается временными вычислениями.

Получается часть задачи посчитать как "комбинационную схему", а часть посчитать через парадигму инструкций.

Тут немного страдает универсальность из-за того, что вычислительные блоки достаточно большие и достаточно сильно заточены под узкие задачи

233. В чём различие между пространственными (spatial) и временными (temporal) вычислениями на примере CGRA?

См. в билете 232

234. Какие подходы существуют к решению проблемы Memory Wall в рамках ASIC и CGRA?

Проблема «Memory Wall» — это разрыв между высокой скоростью вычислений ASIC/CGRA и низкой скоростью/пропускной способностью памяти.

1. Ассоциативные массивы

Этот подход переносит вычисления внутрь памяти или использует память как процессор, устраняя необходимость пересылки данных. То есть вместо поиска данных по адресу (как в RAM), система ищет данные по их содержанию

2. Пространственно-временные вычисления

- Пространственная часть: Данные размещаются на вычислительных элементах рядом с памятью, образуя массивы, что минимизирует глобальные перемещения.
- Временная часть: используется глубокий конвейер или пошаговое выполнение, когда данные обрабатываются «на лету» по мере поступления.

3. Распределенная память

Вместо одного большого централизованного блока памяти данные распределяются по маленьким, быстрым блокам, размещенным непосредственно рядом с вычислительными блоками. Повышает скорости доступа.